

Threshold Public-Key Encryption: Definitions, Relations, and CPA-to-CCA Transforms

Chris Brzuska¹, Michael Klooß², and Ivy K. Y. Woo¹

¹ Aalto University, Espoo, Finland

{chris.brzuska,ivy.woo}@aalto.fi

² Karlsruhe Institute of Technology, Karlsruhe, Germany

KASTEL Security Research Labs, Karlsruhe, Germany

klooss@mail.informatik.kit.edu

Abstract. Threshold public-key encryption (TPKE) allows t out of k parties to jointly decrypt a ciphertext, while ensuring confidentiality against any coalition of $t - 1$ parties. Despite its long history and ongoing standardisation efforts, there has not been a dedicated study on its basic security notions, and a handful of variations are currently in use.

We initiate the systematic study of TPKE confidentiality and develop relations between notions contrasting indistinguishability (IND) vs. simulatability (SIM), passive (CPA) vs. active (CCA) attacks, and static vs. adaptive corruptions. One of our insights is that security under maximal corruptions does not imply security under fewer corruptions when the adversary has access to partial decryptions on challenge ciphertexts. Maximal corruption was adopted by a significant portion of prior works, and this calls for cautious interpretation when using such a notion.

We complement our study by providing two generic CPA-to-CCA transforms for TPKE. The first is effectively the Naor–Yung transform, for which we fix a gap in prior work by requiring the underlying TPKE to achieve *semi-malicious* CPA security, where the adversary can choose randomness for non-challenge ciphertexts. Our second transform applies to any CPA secure TPKE in the random oracle model. We abstract the underlying technique as a standalone novel primitive called non-interactive proof of randomness (NIPoR), and we provide a simple construction from straightline extractable non-interactive zero-knowledge proofs and commitments, which we consider of independent interest.

Keywords: threshold PKE · separations · CPA-to-CCA transforms · non-interactive proof of randomness · partial decryption queries

1 Introduction

Threshold public-key encryption (TPKE) allows decryption of a ciphertext when threshold t out of the total $k \geq t$ parties come together, and preserves confidentiality of the underlying message so long as less than t parties contribute partial decryption shares. TPKE is a fundamental building block for multi-party computation [AJL⁺12], and also a classic example under the umbrella of threshold cryptography, the latter being recently recognised as a standardisation area by the National Institute of Standards and Technology (NIST) [BP23].

Despite the conceptual simplicity of TPKE, a handful of security notions can be considered and the relations between the different definitions are intricate. Our key contributions are two-fold: First, we clarify the impact of definitional choices on security notions and highlight aspects important to TPKE security. Second, we provide two generic CPA-to-CCA transforms of TPKE, easing the task of achieving and/or arguing CCA security for future works.

1.1 TPKE: Syntax and Security

Similar to standard public-key encryption (PKE), the key generation algorithm KGen of a TPKE outputs a public key pk for encryption, but it differs from PKE by returning a set $(\text{sk}_j)_j$ of secret key *shares*, each

of which is distributed to a user j . Given a ciphertext ct and a key share sk_j , a TPKE’s partial decryption algorithm ParDec derives a partial decryption pd_j .³ Given $(\text{pd}_j)_j$ from threshold t many users, the recovery algorithm Rec returns the plaintext.

Toy Example: Threshold Signatures from TPKE. To motivate the impact of TPKE security notions in applications, we consider a simple threshold signature scheme based on a standard signature scheme with deterministic signing algorithm and a fully-homomorphic TPKE⁴ [BGG⁺18]: The dealer encrypts the signing key ssk under TPKE to obtain ct_{ssk} , and shares the partial decryption keys. Now, every party P_j with a key share sk_j offers a partial signing service:

- A user U submits a message m to P_j and P_j decides, according to its policy, whether U should obtain a (partial) signature or not, e.g. each teacher P_j decides whether a thesis should be graded pass or fail.
- In the positive case, P_j homomorphically evaluates $\text{Sign}(\text{ssk}, m)$, using ct_{ssk} , and obtains the encrypted signature ct_σ . It sends the partial decryption $\text{pd}_j = \text{ParDec}(\text{sk}_j, \text{ct}_\sigma)$ back to the user U .

After obtaining sufficiently many partial decryption shares, user U reconstructs the signature σ on m via Rec . This simple protocol is *round-optimal*, since pd_j is obtained in 2 moves, and it requires *no interaction* between secret key holders.

Turning to the security of the protocol, CPA security for TPKE suffices, since an adversary can only choose the to-be-signed message, and ct_σ is computed by honest parties. Now, intuitively, TPKE confidentiality⁵ should directly imply that a signature is not forgeable for any message m which was queried to an insufficient set of parties. Surprisingly, this is *not* the case under security notions which the majority of prior works on TPKE consider, as we explain below.

Security. To see the potential (in)security of the above threshold signatures due to the underlying TPKE, we have to dive deeper into TPKE security models. Below, we restrict to *static* corruption, i.e. the adversary initially specifies the set of corrupted parties. We discuss the impact of adaptive corruption on our security notions later on.

Corruption. A main motivation of threshold primitives including TPKE and threshold signatures is to tolerate some level of corruption of the shared secrets. In a t -out-of- k TPKE, an adversary $\mathcal{A}$ may corrupt a set $\mathcal{C}$ of up to $t - 1$ parties and obtain their secret key shares. Referring to Table 1, a major portion of prior works only consider $\mathcal{A}$ which corrupts a *maximal* set of $t - 1$ parties. While it is common (for other cryptographic primitives) that security against maximal corruption trivially implies security against fewer corruptions, this simplification turns out to be problematic for TPKE.

In the threshold signatures example, suppose we have a $\frac{1}{2}$ recovery threshold (i.e. $\frac{t}{k} \geq \frac{1}{2}$). That is, a majority of parties P_j must decide to issue a signature on m for the user U to recover it. If we face an adversary who can at most corrupt a fraction of $\frac{1}{4}$ users, this is not modelled by maximal corruption at all.

Partial Decryptions. In our threshold signatures example, naturally an adversary $\mathcal{A}$ will see partial decryptions of ciphertexts. This is modelled in TPKE security experiments by giving $\mathcal{A}$ access to a partial decryption oracle ParDecO . One can naturally expect ParDecO to answer two types of partial decryption queries:

- (I) queries on *non-challenge* ciphertexts (i.e. those not depending on the challenge bit), where ParDecO may return its partial decryptions w.r.t. all k parties;

³ Alternatively, one can consider, e.g. interactive decryption, or decompose KGen into two algorithms which generate pk and $(\text{sk}_j)_j$ separately. See Sec. 1.3 for a discussion of syntax variations. The syntax considered in this work is one of the simplest and the most common, arguably also with the longest history.

⁴ This leads to threshold fully-homomorphic encryption (TFHE), although we will only focus on TPKE security, i.e. without taking homomorphic computation into account. See Sec. 1.3 for recent results on subtleties of (non-threshold) FHE security.

⁵ and circuit privacy for FHE so that ciphertext ct_σ contains no information about ssk .

Works	Security							Assumptions
	IND/SIM	Corr.	$ \mathcal{C} $	ParDecO		Strength	rnd	
				Type I	Type II			
[SG98]	IND	Q-stat.	$t - 1$	Yes	-	CCA	Yes [#]	CDH/DDH + RO
[BBH06]	IND	Q-stat.	$t - 1$	Yes	-	CCA	Yes [#]	DBDH
[AT09]	IND	Static	$t - 1$	Yes	-	CCA	Yes [#]	DBDH/DLIN
[BD10]	SIM	Static	$t - 1$	-	-	CPA	No	LWE
[BGG ⁺ 18]	SIM	Q-stat.	$t - 1$	-	-	CPA	No	LWE
[PS24]	SIM	Q-stat.	$t - 1$	-	-	CPA	No	LWE
[MS25]	SIM	Static	$t - 1$	-	-	CPA	No	LWE [‡]
[BS23]	IND	Q-stat.	$\leq t - 1$	Yes	No	CPA	No	LWE
[LY12]	IND	Adapt.	$\leq t - 1$	Yes	Yes [†]	CCA	Yes [#]	DLIN
[DLN ⁺ 21]	IND	Adapt.	$\leq t - 1$	Yes	Yes	CCA	Yes [#]	LWE(+DCR)
[CLW25]	SIM ⁻	Static	$\leq t - 1$	Yes	Yes	CPA [*]	Yes	LWE
Sec. 4	SIM	Static	$\leq t - 1$	Yes	Yes	CCA	Yes [#]	DDH

Table 1: Prior works on TPKE⁷. [BGG⁺18, BS23, PS24] constructed TFHE, the above considers their schemes as TPKE. **rnd**: Oracle **EncO** releases encryption randomness. **Q-stat.**: Quasi-static (Remark 2). **Adapt.**: Adaptive. **#**: Implied by CCA (Remark 1). **†**: Implied by adaptive corruption (Thm. 1). *****: [CLW25] proved CPA security and relied on our NIPoR result (Sec. 5) to achieve CCA. **SIM⁻**: A version more restrictive than Definition 3. **‡**: In the ring setting, also assuming known-covariance LWE.

- (II) queries on *challenge* ciphertexts (i.e. whose underlying plaintexts depend on the challenge bit), where **ParDecO** may return its partial decryptions w.r.t. some subset $S \subseteq [k] \setminus \mathcal{C}$ of parties, so that $\mathcal{A}$ cannot trivially decrypt the challenge, that is, $|S \cup \mathcal{C}| < t$.

Type I queries were commonly taken into account in prior works. However, Type II queries were rarely considered (see Table 1).

We observe that Type II queries will be crucial in our example scenario: We want the signature σ in ct_σ to be hidden until t partial decryption shares have been queried. Since we assume that $\mathcal{A}$ only corrupts $\frac{1}{4}$ th of the parties, it is allowed to request partial signatures from less than $\frac{1}{4}$ th more parties.⁶

A Separating Example. In general, security models without Type II queries seem weaker and may be insufficient for TPKE applications. As a simple example, consider a $(k/2)$ -out-of- k TPKE where the encryption algorithm encrypts normally, but additionally also picks a smaller random subset S of $k/4$ parties and $(k/4)$ -out-of- $(k/4)$ encrypts to them. Using Type II queries, the adversary $\mathcal{A}$ can query $k/4 < t$ partial decryptions of the challenge ciphertext on the set S of users and recover the message. Thus, such TPKE should be deemed insecure, as the chosen $k/4$ parties can jointly decrypt without meeting the threshold $t = k/2$. However, in models without Type II queries, the set S may be exploited only when all its parties are corrupt, which happens only with negligible probability $\leq (1/2)^{k/4}$ when k is sufficiently large.

The above example highlights the phenomenon that security can trivially break even for a non-corrupting adversary, although security against a maximal-corrupting adversary $\mathcal{A}$ still holds, since under maximal corruptions, allowing or disallowing Type II queries is equivalent. Therefore, for TPKE, the claim that “security under maximal corruption implies the same notion under fewer corruptions” does not hold in general. Of course, the validity of this argument also depends on other aspects of the security model, e.g. adaptivity of corruption and the adversary’s queries, the number of challenge queries allowed etc. This raises the question of how different TPKE security models relate and which models considered in existing works provide sufficient security guarantees.

⁶ We note that the distinction between Type I and Type II queries is obscured by *maximal* corruption, as any additional **ParDec** query on a challenge leads to a trivial win under maximal corruption.

⁷ We list the works whose TPKE syntax is consistent with that considered in this work (Definition 1), which is also the most common in the literature. See Section 1.3 for more related works and some possible variants in syntax.

Indistinguishability versus Simulation. Assuming that Type II queries for TPKE are at hand, we attempt to make a security reduction for our toy threshold signature. With indistinguishability-based (*IND*-CPA⁸) security for TPKE, we run into a problem: An adversary $\mathcal{A}$ against the threshold signature may request $\leq t - 1$ partial signatures for random parties on many distinct messages, e.g. 2^{30} many. Then, it may pick some, e.g. $2^{15} \gg \lambda$ out of 2^{30} , and request the t -th share. In the *IND*-CPA game of TPKE, the reduction cannot request the t -th partial decryption for any challenge ciphertext; it also cannot guess the 2^{15} messages and let these be the queries for non-challenge ciphertexts, since the guessing would succeed with negligible probability. Thus, we need an alternative reduction strategy⁹ – or better a stronger security for TPKE.

Consider a simulation-based notion for TPKE (*SIM*-CPA), where the simulator does not learn the encrypted message of a challenge ciphertext until the t -th partial decryption share is requested, which ensures confidentiality. With such a notion, a reduction may then delay the request for a signature σ from the EUF-CMA game, until it needs to produce the t -th partial decryption, i.e. until $\mathcal{A}$ learns σ anyway (which does not constitute a forgery).

To sum up the learnings from the above:

- A TPKE with *SIM*-CPA security supporting Type II queries and allowing non-maximal corruption is convenient and ideal to instantiate our toy example.
- With an *IND*-CPA secure TPKE, smarter reductions are needed⁹.
- Without Type II queries, the threshold signature is completely broken.

This work focuses on TPKE and subtle details may not directly generalise to TFHE. Bridging the gap between TPKE and TFHE is a task that requires additional care (see Section 1.3).

1.2 Our Contributions

Security Models for TPKE. We formulate two basic security notions for TPKE, one as indistinguishability games and one in simulation style, respectively. Our models are designed to take care of various natural security aspects of TPKE, including corruption and partial decryptions as discussed in Section 1.1. The models are formalised in Section 2, capturing the setting with:

- *adaptive* partial decryption queries (both Types I and II),
- *static* corruption, and
- both chosen-plaintext attacks (CPA) and chosen-ciphertext attacks (CCA).

By suitably restricting an adversary’s ability, our models recover the majority of adversarial capabilities considered in prior works. The choice of static corruption is for two reasons: We aim to model what we believe are the minimal requirements for a reasonably secure TPKE, and we will see that adaptive corruption is impossible for simulation-based security. Our models can be both strengthened and weakened in the natural ways. To relate notions, we will also consider variants with *selective* partial decryption queries (both Types I and II) and/or *adaptive* corruption (the latter only for *IND* security).

Relations between TPKE Security Notions. We prove implications and separations between different security notions, as depicted in Fig. 1. The results below highlight some aspects of CPA security under static corruption, which should be taken into account when assessing security of a TPKE.

- (Thm. 3) Under adaptive partial decryption queries, security under maximal corruption does not imply security under an arbitrary number of corruptions.
- (Thm. 5) Simulation security (with mild restrictions on the simulator) against an adversary observing only a single challenge ciphertext is strictly weaker than multi-challenge security.
- (Thm. 6) Under adaptive partial decryption queries and CPA-security, learning the encryption randomness of non-challenge (honest) ciphertexts is strictly stronger than hiding it.¹⁰

⁸ The *ParDecO* oracle is present for CPA and CCA. However, for CPA, *ParDecO* is limited to honestly generated ciphertexts. For CCA, any ciphertext is allowed.

⁹ E.g. the reduction may guess which (if any) of the 2^{30} messages will be the forgery. This works, but the reduction suffers from a 2^{30} factor reduction loss.

¹⁰ Not applicable to CCA, as there is no meaningful non-challenge encryption oracle.

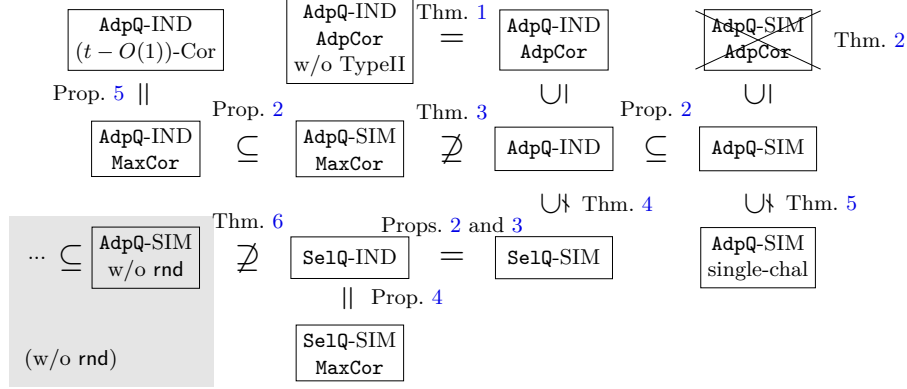


Fig. 1: Selected and simplified implication and separation results. (For a comprehensive version see Fig. 29.) Setting: CPA, ParDecO answers both Types I, II queries (AdpQ for adaptive oracle queries, SelQ for selective), static corruption unless otherwise specified. Our main results are stated for the setting where the non-challenge encryption oracle EncO also outputs the encryption randomness *rnd*. The same set of relations again holds for the setting where EncO does not output *rnd*, depicted by the “w/o *rnd*” zone.

To prove our separations, we construct (contrived) counterexample TPKE schemes. While the counterexamples for Thm. 3 and Thm. 6 are very simple, the one for Thm. 5 is quite sophisticated. It is an interesting question whether *natural* counterexamples exist. Other relations and separations include:

- (Thm. 1) Adaptive corruption automatically implies the security with Type II partial decryption queries.
- (Thm. 2) Simulation security with adaptive corruption is not achievable, assuming adaptive partial decryption queries.
- (Thm. 4) Security against selective partial decryption queries does not imply that against adaptive queries.

Thm. 2 is a simple adaptation of Nielsen’s non-committing encryption impossibility result [Nie02]. The above results raise an interesting tension between the choice of game- and simulation-based security: Game-based security with adaptive corruption and simulation-based security with static corruption are incomparable, and the suitability of either of them likely depends on applications.

Along the way, we also show other relation results which are (more) commonly expected to hold (Propositions in Fig. 1).

Beyond CPA-security, we also consider the stronger notion of CCA-security, where an adversary can issue partial decryption queries for arbitrary, maliciously generated ciphertexts [SG98, BBH06, AT09], explained next.

CPA-to-CCA Transform I: Semi-Malicious Security. We observe that the classic Naor–Yung transform [NY90] (and straightforward variants) to upgrade CPA to CCA security by attaching a NIZK cannot generically work for CPA secure TPKE.¹¹ Nevertheless, in Section 4 we give a result of similar spirit: When assuming *semi-malicious CPA*, where an adversary can see partial decryptions of ciphertexts generated with *adversarially chosen randomness*, CCA can be achieved by attaching a non-interactive zero-knowledge (NIZK) proof of knowledge, which proves the well-formedness of ciphertexts (Thm. 7).

Attaching a NIZK is a classic technique, but the subtlety lies in *when* it actually applies to a TPKE. Our message is that, to rely on this technique, a stronger baseline security of the underlying TPKE is required.

¹¹ The gap in [FP01, Theorem 1] is, at a high level, that the Naor–Yung transform requires *perfect* correctness, but the natural analogue is not enough for TPKE when partial decryption queries are admitted. See Section 4.

To demonstrate that our security notions are achievable, we show that the natural thresholdised ElGamal PKE (first suggested as a TPKE in [DF90]) achieves simulation semi-malicious CPA security¹². Most of our proof is a direct adaptation from a recent work [CLW25] for which we claim no novelty. Applying the above NIZK-based transform, we obtain a simulation CCA-secure TPKE.

CPA-to-CCA Transform II: Non-interactive Proof of Randomness. To capture a broader class of TPKE schemes which satisfy CPA but not semi-malicious CPA security, in Section 5 we propose a new primitive called *non-interactive proof of randomness (NIPoR)*. A NIPoR may be seen as a variant of NIZK, whose functionality is to generate a proof that an efficient function f has been evaluated on a secret value m and secret high-entropy randomness r , resulting in some (public) value $y = f(m; r)$. By setting f as the encryption algorithm of a TPKE, one can prove that an encryption of a message m has been computed with honestly sampled and fresh randomness r .

We construct a NIPoR in the ROM, assuming the existence of straightline extractable NIZKs and commitments (Thm. 9). Using a NIPoR to attach a “randomness proof” to a ciphertext, we obtain a transform which upgrades CPA to CCA security in the ROM (Thm. 10). The transform might be viewed as a “publicly verifiable Fujisaki–Okamoto (FO) transformation”, and is a mix between NY and FO transforms.

Our NIPoR has already found application in the recent work [CLW25] where it was used to upgrade a lattice-based TPKE scheme from CPA to CCA security. We believe the concept of NIPoR can find applications beyond TPKE and CPA-to-CCA transforms, and is of independent interest.

1.3 Related Works

Variations in TPKE Settings. The field of threshold encryption is vast, a variety of TPKE syntaxes (therefore also further variations in security notions) exists. For example, this work only focuses on *non-interactive* decryption, where partial decryptions are generated asynchronously without interaction between the decryption parties. The setting with interactive decryption is a natural and reasonable extension, considered in e.g. [CG99]. Dynamic key generation was considered in e.g. [DP08, QWZ⁺12], where the trusted party can dynamically generate user secret keys (instead of all at once upfront) and encryption is w.r.t. threshold t dynamically chosen by the encryptor.

There has recently been rapid progress in TPKE constructions targeting improved efficiency and/or more advanced functionalities. A handful of models are proposed, some examples are: batch decryption [CGPP24, BFOQ25, BLT25], i.e. multiple ciphertexts can be decrypted at once (with non-trivial efficiency constraints); silent setup [GKPW24, WW25, HASW25], i.e. users can distributedly generate their own keypair; traceability [BPR24, CPP25], i.e. traitors who leak their decryption ability can be traced; context-dependent decryption [BBN⁺25a], i.e. partial decryptions are bound to a context value which acts as a domain separator; server-aided decryption [PS24, OT25] (proposed for TFHE and also applies to TPKE), i.e. an additional trusted server, possibly knowing the users’ secrets, may pre-process a ciphertext before it is partial-decrypted by users; and combinations of the above [BCF⁺25].

For each of the above, the difference in syntax inherently leads to different formal security definitions. However, we highlight that most of these variations can be seen as add-ons of the base case considered in this work, and we expect that most of our results carry over to these more advanced/complex settings.

Other Results on TPKE Security. This work focuses on *confidentiality* of TPKE, although further properties may be desired. For example, it is arguably important to model non-malleability, also called *robustness* [GRJK00, BBH06, BGG⁺18, BS23], which guarantees that malicious partial decryptions do not allow an adversary to arbitrarily modify the message that an honest user recovers.

In a concurrent work, Boneh, Bünz, Nayak, Rotem and Shoup (BBNRS [BBN⁺25b]) also consider the concept of Types I and II queries and refer to these as low- and high-threshold security respectively. BBNRS

¹² Despite the decades, we are not aware of a proof showing that it meets the security we consider, in particular in presence of Type II partial decryption queries.

[BBN⁺25a, Sec. 8, Eprint 2025-09-09] sketch how their security notions can be generalised to adaptive corruption, simulation-based security, UC-compatibility, and discuss a straightforward IND-CCA to SIM-CCA transform via non-committing encryption in the ROM. In turn, they do not consider CPA-to-CCA transforms, impossibilities, maximal corruption, and relations between various other existing security notions.

Related Notions/Primitives. Li and Micciancio (LM [LM21]) point out that IND-CPA security is too weak for (non-threshold) *approximately correct* FHE, since even the decryption of *honestly* generated ciphertexts might leak additional information. They suggest strengthening IND-CPA with an additional decryption oracle D to which an adversary can query honestly generated ciphertexts. Under their CPA^D model, LM show practical attacks against approximately correct FHE schemes. This observation is analogous to the potential leakage from TPKE Type II queries that we consider. LM also study relations between variants of the definition, later expanded by [CFP⁺24]. Similar to our CPA notions for TPKE, CPA^D security for approximately correct FHE is fragile in that single- and multi-challenge definitions are not equivalent, and likewise, the number of decryption queries strictly increases the strength of the model. [BJSW25] study CPA^D security where (non-challenge) ciphertexts can be generated using adversarially chosen randomness, and show that such model is strictly stronger [BJSW25]; this can be seen analogous to our semi-malicious CPA security (Section 4). Several realistic CPA^D attacks on both approximately correct and exact FHE schemes have been implemented (cf. [GNSJ24, CSBB24, CCP⁺24] and references therein), demonstrating that decryption of honest ciphertexts is a relevant attack angle.

For *threshold signatures*, the work of [BCK⁺22] introduces a fine-grained hierarchy of security notions including implications and separations, focusing on selective corruptions, similar to this work. Here, the difficulty of deciding when to consider a signature a forgery¹³ is similar to the question of when to allow partial decryptions for a challenge ciphertext in TPKE. Separations between different notions for threshold signatures and encryption are conceptually related.

The question of NIZK-based variants of the Naor–Yung transform [NY90] in the ROM is explicitly explored in [BFW15]. The notion of *universal samplers* [HJK⁺16] in the ROM bears similarities to NIPoR, but is stronger and their construction requires trusted setup to realise. Universal samplers force a party to honestly sample from a distribution (specified by a sampling circuit) such that only the output is learned. A NIPoR only asserts the correct distribution of the output, but the sampling party may be aware of the random coins; indeed, this is the case for our constructions and can be a desirable feature. The work [ASY22] introduces the concept of an *anti-rusher* transformation for MPC in the ROM. Both works [HJK⁺16, ASY22] program random oracle queries to inject information, similar to fixing a Blum coin-toss; the technique is called *delayed backdoor programming* in [HJK⁺16].

2 IND and SIM Security for TPKE

Preliminaries. We let λ denote the security parameter and generally, almost all objects and families are implicitly parametrised by it and algorithms receive λ (implicitly) as input. We denote sets and tables by capital letters, e.g. S and T . For $k \in \mathbb{N}$, we write $[k]$ for $\{1, 2, \dots, k\}$. As pseudocode, we write $x \leftarrow v$ to assign value v to variable x , and we write $x \leftarrow \text{algo}(v)$ to run the deterministic algorithm algo on value v and assign the result to x . Similarly, we write $x \leftarrow \$ S$ to sample a uniformly random value from set S and assign the result to x , and we write $x \leftarrow \$ \text{algo}'(v)$ to run the randomised algorithm algo' with fresh random coins on v and assign the result to x . When an algorithm algo' has multiple outputs, we write $(x, y, z) \leftarrow \$ \text{algo}'(v)$ and $(x, _) \leftarrow \$ \text{algo}'(v)$ if we ignore the values of all outputs except for the first.

An *access structure* on k parties is a set $\mathbb{A} \subseteq 2^{[k]} \setminus \emptyset$ of non-empty sets. If a set of parties $A \subseteq [k]$ satisfies $A \in \mathbb{A}$, we say that A satisfies the access structure $\mathbb{A}$. An access structure $\mathbb{A}$ is said to be *monotone* if the following holds for any $A, B \subseteq [k]$: If $A \in \mathbb{A}$ and $A \subseteq B$, then $B \in \mathbb{A}$. For any $t \in [k]$, a monotone access structure $\mathbb{A}_{k,t}$ is called a (t, k) -threshold access structure, if $A \in \mathbb{A}_{k,t}$ if and only if $A \subseteq [k]$ and $|A| \geq t$. Unless

¹³ Important differentiations were already considered in [Sho00].

specified otherwise, we assume that every access structure $\mathbb{A}$ is a (t, k) -threshold structure, i.e. $\mathbb{A} = \mathbb{A}_{k,t}$. This is a simplifying assumption. Many of our results apply to general $\mathbb{A}$ with minor adaptations.

We recall definitions and security properties of pseudorandom functions (PRF), commitment schemes, and non-interactive zero-knowledge (NIZK) proofs in Appendix A.

We provide the TPKE syntax and our security definitions in Section 2.1, then discuss our design choices in Section 2.2. Moreover, we summarise the properties and their variants in Table 2.

2.1 Definitions

Definition 1 (Threshold PKE). Let $\mathfrak{A} = (\mathbb{A}_\lambda)_{\lambda \in \mathbb{N}}$ be a (family of) set(s) of access structures. A threshold public-key encryption (TPKE) TPKE for $\mathfrak{A}$ and a message space $\mathcal{M}$ consists of PPT algorithms (KGen, Enc, ParDec, Rec) with the following syntax:

KGen($1^\lambda, \mathbb{A}$) $\rightarrow$ (pk, (sk_j)_{j ∈ [k]}): The key generation algorithm, on input the security parameter 1^λ and an access structure $\mathbb{A} \in \mathfrak{A}_\lambda$ on k parties, generates the public key pk and a tuple of k secret keys (sk_j)_{j ∈ [k]} for each user $j \in [k]$.

Enc(pk, μ) $\rightarrow$ ct: The encryption algorithm encrypts a message $\mu \in \mathcal{M}$ w.r.t. the public key pk.

ParDec(sk_j, ct) $\rightarrow$ pd_j: The partial decryption algorithm receives the secret key sk_j of a user j and a ciphertext ct, and outputs a partial decryption pd_j.

Rec(pk, (pd_j)_{j ∈ T}, ct) $\rightarrow$ μ' : The reconstruction algorithm gets a tuple of partial decryptions (pd_j)_{j ∈ T} from a set T of users and a ciphertext ct and outputs a message μ' . We omit pk as input to Rec whenever clear from the context.

We require TPKE to be correct, i.e. for any $\mathbb{A} \in \mathfrak{A}$, (pk, (sk_j)_{j ∈ [k]}) $\in$ KGen($1^\lambda, \mathbb{A}$), set $T \in \mathbb{A}$, and $\mu \in \mathcal{M}$, there is a negligible function ε such that

$$\Pr \left[\mu' = \mu \mid \begin{array}{l} \text{ct} \leftarrow \text{Enc}(\text{pk}, \mu) \\ \text{pd}_j \leftarrow \text{ParDec}(\text{sk}_j, \text{ct}) \quad \forall j \in T \\ \mu' \leftarrow \text{Rec}((\text{pd}_j)_{j \in T}, \text{ct}) \end{array} \right] \geq 1 - \varepsilon.$$

We call ε the correctness error. If $\varepsilon = 0$, then TPKE is perfectly correct.

If any choice is valid, we often leave $\mathfrak{A}$ and $\mathcal{M}$ implicit. Below, we define TPKE game-based confidentiality (IND-CPA & IND-CCA) and simulation-based confidentiality (SIM-CPA & SIM-CCA) under static corruption. See Appendix B for a definition under adaptive corruption.

Definition 2 (IND-CPA & IND-CCA). A TPKE scheme TPKE is secure under static corruption, adaptive challenge queries and chosen-plaintext attacks (SCor-AdpQ-IND-CPA) or chosen-ciphertext attacks (SCor-AdpQ-IND-CCA), respectively, if for all PPT adversaries $\mathcal{A}$, the advantage $\text{Adv}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^{\text{IndCPA}}(\lambda) :=$

$$|\Pr[\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^0(1^\lambda) = 1] - \Pr[\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^1(1^\lambda) = 1]|,$$

or the advantage $\text{Adv}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^{\text{IndCCA}}(\lambda) :=$

$$|\Pr[\text{IndCCA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^0(1^\lambda) = 1] - \Pr[\text{IndCCA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^1(1^\lambda) = 1]|,$$

is negligible, respectively, where the security experiments are defined in Fig. 2.

Definition 3 (SIM-CPA & SIM-CCA). A TPKE scheme TPKE is simulation-secure under static corruption, adaptive challenge queries and chosen-plaintext attacks (SCor-AdpQ-SIM-CPA) or chosen-ciphertext attacks (SCor-AdpQ-SIM-CCA), respectively, if there is a PPT simulator $\mathcal{S}$, such that for all PPT adversaries $\mathcal{A}$ the advantage $\text{Adv}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^{\text{SimCPA}}(\lambda) :=$

$$|\Pr[\text{SimCPA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^0(1^\lambda) = 1] - \Pr[\text{SimCPA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^1(1^\lambda) = 1]|,$$

or the advantage $\text{Adv}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^{\text{SimCCA}}(\lambda) :=$

$$|\Pr[\text{SimCCA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^0(1^\lambda) = 1] - \Pr[\text{SimCCA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^1(1^\lambda) = 1]|,$$

is negligible, respectively, where the security experiments are defined in Fig. 3.

IndCCA _{TPKE, A, AdpQ} ^b (1 ^λ)	IndCPA _{TPKE, A, AdpQ} ^b (1 ^λ)	ParDecO (ct, j)
$(\mathbb{A}_{k,t}, \mathcal{C}, \text{st}_A) \leftarrow \mathcal{A}(1^\lambda)$ // Corrupt set $\mathcal{C}$ assert $(\mathcal{C} \subset [k]) \wedge (\mathcal{C} \not\subseteq \mathbb{A}_{k,t}) \wedge \mathbb{A}_{k,t} \in \mathfrak{A}_\lambda$ $(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \mathcal{KGen}(1^\lambda, \mathbb{A}_{k,t})$ $L_{\text{Enc}} \leftarrow \emptyset; L_{\text{Chal}} \leftarrow \emptyset; P[\cdot] \leftarrow \emptyset$ $\text{st}_A \leftarrow (\text{pk}, (\text{sk}_j)_{j \in \mathcal{C}}, \text{st}_A)$ return $b' \leftarrow \mathcal{A}^{\text{EncO}, \text{ChalO}, \text{ParDecO}}(\text{st}_A)$		assert $\text{ct} \in L_{\text{Enc}} \cup L_{\text{Chal}}$ if $\text{ct} \in L_{\text{Chal}}$ then $P[\text{ct}] \leftarrow P[\text{ct}] \cup \{j\}$ assert $P[\text{ct}] \cup \mathcal{C} \not\subseteq \mathbb{A}_{k,t}$ $\text{pd}_j \leftarrow \mathcal{P}(\text{sk}_j, \text{ct})$ return pd_j
EncO (μ)	ChalO (μ ₀ , μ ₁)	
$\text{rnd} \leftarrow \{0, 1\}^{\text{rlen}(\lambda)}$ $\text{ct} \leftarrow \text{Enc}(\text{pk}, \mu; \text{rnd})$ $L_{\text{Enc}} \leftarrow L_{\text{Enc}} \cup \{\text{ct}\}$ return (ct, rnd)	$\text{ct} \leftarrow \mathcal{S}(\text{Enc}(\text{pk}, \mu_b))$ $L_{\text{Chal}} \leftarrow L_{\text{Chal}} \cup \{\text{ct}\}$ return ct	

Fig. 2: $\text{IndCCA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^b(1^\lambda)$ and $\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^b(1^\lambda)$ security experiments for TPKE. The boxed code only applies to $\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \text{X}}^b(1^\lambda)$. The oracle EncO is redundant for CCA, see Remark 1.

SimCCA _{TPKE, A, S, AdpQ} ^b (1 ^λ)	SimCPA _{TPKE, A, S, AdpQ} ^b (1 ^λ)	ChalO (μ)
$(\mathbb{A}_{k,t}, \mathcal{C}, \text{st}_A) \leftarrow \mathcal{A}(1^\lambda)$ // Corrupt set $\mathcal{C}$ assert $(\mathcal{C} \subset [k]) \wedge (\mathcal{C} \not\subseteq \mathbb{A}_{k,t}) \wedge \mathbb{A}_{k,t} \in \mathfrak{A}_\lambda$ if $b = 0$ then $(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \mathcal{KGen}(1^\lambda, \mathbb{A}_{k,t})$ if $b = 1$ then $(\text{pk}, (\text{sk}_j)_{j \in \mathcal{C}}, \text{st}_S) \leftarrow \mathcal{S}(1^\lambda, \mathbb{A}_{k,t}, \mathcal{C})$ $L_{\text{Enc}} \leftarrow \emptyset; L_{\text{Chal}} \leftarrow \emptyset; P[\cdot] \leftarrow \emptyset; M[\cdot] \leftarrow \emptyset$ $\text{st}_A \leftarrow (\text{pk}, (\text{sk}_j)_{j \in \mathcal{C}}, \text{st}_A)$ return $b' \leftarrow \mathcal{A}^{\text{EncO}, \text{ChalO}, \text{ParDecO}}(\text{st}_A)$		if $b = 0$ then $\text{ct} \leftarrow \mathcal{S}(\text{Enc}(\text{pk}, \mu))$ if $b = 1$ then $(\text{ct}, \text{st}_S) \leftarrow \mathcal{S}(\text{st}_S)$ $L_{\text{Chal}} \leftarrow L_{\text{Chal}} \cup \{\text{ct}\}; M[\text{ct}] \leftarrow \mu$ return ct
EncO (μ)		ParDecO (ct, j)
if $b = 0$ then $\text{rnd} \leftarrow \{0, 1\}^{\text{rlen}(\lambda)}$ $\text{ct} \leftarrow \text{Enc}(\text{pk}, \mu; \text{rnd})$ if $b = 1$ then $(\text{ct}, \text{rnd}, \text{st}_S) \leftarrow \mathcal{S}(\mu, \text{st}_S)$ $L_{\text{Enc}} \leftarrow L_{\text{Enc}} \cup \{\text{ct}\}; M[\text{ct}] \leftarrow \mu$ return (ct, rnd)		assert $\text{ct} \in L_{\text{Enc}} \cup L_{\text{Chal}}$ $P[\text{ct}] \leftarrow P[\text{ct}] \cup \{j\}$ if $b = 0$ then $\text{pd}_j \leftarrow \mathcal{P}(\text{sk}_j, \text{ct})$ if $b = 1$ then if $P[\text{ct}] \cup \mathcal{C} \subseteq \mathbb{A}_{k,t} \wedge \text{ct} \in L_{\text{Chal}}$ then $\mu \leftarrow M[\text{ct}]$ $(\text{pd}_j, \text{st}_S) \leftarrow \mathcal{S}(\text{ct}, \mu, j, \text{st}_S)$ else $(\text{pd}_j, \text{st}_S) \leftarrow \mathcal{S}(\text{ct}, j, \text{st}_S)$ return pd_j

Fig. 3: $\text{SimCCA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \text{AdpQ}}^b(1^\lambda)$ and $\text{SimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \text{AdpQ}}^b(1^\lambda)$ security experiments for TPKE. The boxed code only applies to $\text{SimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \text{AdpQ}}^b(1^\lambda)$. The oracle EncO is redundant for CCA, see Remark 1.

Selective Queries. In Appendix B, we define *selective queries* security for all of the above 4 definitional variants, denoted by SCor-SelQ-Y-Z , for $Y \in \{\text{IND}, \text{SIM}\}$ and $Z \in \{\text{CPA}, \text{CCA}\}$ (Definitions 18 and 19). Selective security requires the adversary to declare its queries upfront, at the same time as it chooses the corrupt parties. While selective security is too weak to be considered a reasonable target notion, it is useful to clarify in which cases the adaptivity of partial decryption queries makes one security notion stronger than

another (cf. Propositions 3 and 4 and Remark 8, which are in contrast to the separations for adaptive security in Sections 3.3 and 3.4).

Adaptive Corruption. In Appendix B, we define TPKE security under *adaptive corruption* for both the IND and SIM settings (Definitions 20 and 21), denoted by AdpCor-X-Y-Z , for $X \in \{\text{SelQ}, \text{AdpQ}\}$, $Y \in \{\text{IND}, \text{SIM}\}$ and $Z \in \{\text{CPA}, \text{CCA}\}$. Under adaptive corruption, an adversary $\mathcal{A}$ does not need to declare the corrupt set $\mathcal{C}$ upfront, but is given a corruption oracle CorO which it can query throughout the experiment. Upon querying on an index j , CorO returns the secret key sk_j of j . For IND, we require that $\mathcal{A}$ never corrupts an admissible set of indices; for SIM, as soon as $\mathcal{A}$ corrupts an admissible set T , the simulator $\mathcal{S}$ receives the underlying plaintext for any ct decryptable by the set T . Looking ahead, we will see that adaptive corruption is impossible in the SIM setting (Theorem 2).

Remark 1 (CCA subsumes EncO). For CCA security, the access to EncO in Figs. 2 and 3 is unnecessary, since an adversary $\mathcal{A}$ can query ParDecO on ciphertexts ct's generated by itself for all k parties. (In particular, $\mathcal{A}$ also knows the encryption randomness rnd for these ct's.) We keep EncO in the CCA definitions only for ease of comparison with CPA.

Remark 2 (Quasi-static Corruption). We say that TPKE is IND- resp. SIM-CPA-secure under *quasi-static* corruption and X challenge queries, if security holds w.r.t. a modified experiment $\text{IndCPA}_{\text{TPKE}, \mathcal{A}, X}^b$ resp. $\text{SimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, X}^b$, where $\mathcal{A}$ declares the corrupt set $\mathcal{C}$ after receiving pk , but still before making any oracle query to EncO , ChalO and ParDecO . For SIM-security, correspondingly, in game $b = 1$ the simulator $\mathcal{S}$ simulates pk and $(\text{sk}_j)_{j \in \mathcal{C}}$ separately, the latter after $\mathcal{A}$ declares the set $\mathcal{C}$.

The following restricted variants of security are adapted from prior works.

Definition 4 (Security under Maximal Corruption). Let $X \in \{\text{AdpQ}, \text{SelQ}\}$. A TPKE scheme TPKE is X -IND-CPA under maximal corruption (MaxCor-X-IND-CPA), if for all PPT adversaries $\mathcal{A}$ which declare a corrupt set $\mathcal{C}$ of maximal size, i.e. $|\mathcal{C}| = t - 1$ for threshold t chosen by $\mathcal{A}$, the advantage $\text{Adv}_{\text{TPKE}, \mathcal{A}, X}^{\text{IndCPA}}(\lambda)$ is negligible. X -SIM-CPA under maximal corruption (MaxCor-X-SIM-CPA) and CCA-variants are defined analogously.

Note that maximal corruption only makes sense in the static (but not adaptive) corruption model.

Definition 5 (Security without Type II Queries). Let $X \in \{\text{AdpQ}, \text{SelQ}\}$. A TPKE scheme TPKE is X -IND-CPA without Type II queries, if for all PPT adversaries $\mathcal{A}$ which never query ParDecO on any $\text{ct} \in L_{\text{Chal}}$, the advantage $\text{Adv}_{\text{TPKE}, \mathcal{A}, X}^{\text{IndCPA}}(\lambda)$ is negligible. X -IND-CCA without Type II queries is analogous.

Definition 6 (Single-challenge Security). Let $X \in \{\text{AdpQ}, \text{SelQ}\}$. A TPKE scheme TPKE is single-challenge X -IND-CPA, if every PPT adversary $\mathcal{A}$ which makes only a single ChalO query has negligible advantage $\text{Adv}_{\text{TPKE}, \mathcal{A}, X}^{\text{IndCPA}}(\lambda)$. In this case, we also denote such advantage by $\text{Adv}_{\text{TPKE}, \mathcal{A}, X}^{\text{1Ch-IndCPA}}(\lambda)$. Single-challenge X -SIM-CPA and CCA variants are defined analogously.

Different mixtures of the above variants are possible. We summarise the different properties and the notations used in this work in Table 2.

2.2 Discussion

We highlight a number of security aspects in our models which are tied to TPKE, and draw connections to the models in prior works.

Property	Type	Notation/Abbrev.	Note
Corruption	Maximal	MaxCor	Def. 4
	Static	SCor	<i>Default</i>
	Quasi-Static	–	Rem. 2
	Adaptive	AdpCor	Def. 20, 21
Type II queries	No	“without Type II queries”	Def. 5
	Yes	–	<i>Default</i>
Queries (EncO, ChalO, ParDecO)	Selective	SelQ	Def. 18, 19
	Adaptive	AdpQ	<i>Default</i>
# of challenges	Single-Challenge	1Ch	Def. 6
	Multi-Challenge	–	<i>Default</i>
EncO oracle (only applies to CPA)	Honest randomness	–	<i>Default</i>
	Semi-malicious	SemiMal	Def. 7
ParDecO restriction	CPA	CPA	
	CCA	CCA	
Security Model	Indistinguishability	IND	Def. 2
	Simulation	SIM	Def. 3

Table 2: Summary of properties, their possible types, and notations. Default means the setting considered in Definitions 2 and 3.

Corrupt Parties. Both the IND- and SIM-based (Definitions 2 and 3) models allow an adversary $\mathcal{A}$ to corrupt $|\mathcal{C}| \leq t - 1$ parties, i.e. any $\mathcal{C} \notin \mathbb{A}_{k,t}$. By restricting to the class of adversaries that corrupt a maximal set of $t - 1$ parties (Definition 4), we recover the corruption behaviour in models of a handful of prior works, including that in the seminal work of [BGG⁺18] which constructed TFHE and a universal thresholdiser, among others, e.g. [SG98, BBH06, AT09] (IND-based) and [BD10, BGG⁺18, PS24, MS25] (SIM-based).¹⁴

Looking ahead, we show that Definition 4 implies neither Definition 2 nor Definition 3 (Theorem 3). Concretely, in the case of maximal corruption, partial decryption queries on challenge ciphertexts ct returned from ChalO are disallowed in the IND setting, whereas in the SIM setting the simulator $\mathcal{S}$ always receives the plaintext μ for simulating partial decryptions. As such, maximal corruption does not capture confidentiality of messages for which the adversary obtains $< t - 1$ partial decryptions. We note that while Theorem 3 is stated for static corruption, the same holds for quasi-static corruption (Remark 2).

ParDecO queries for Challenges. The IND-CPA model of [BS23, Def.20] is similar to our AdpQ-IND-CPA model (Definition 2), but with the crucial difference that our oracle ParDecO also answers queries on challenge ciphertexts (called Type II queries in Section 1.1) so long as $\mathcal{A}$ cannot trivially decrypt them. More specifically, in ParDecO in Fig. 2, if the check $\text{ct} \in L_{\text{Chal}}$ passes, it does not abort. If instead ParDecO would abort for such queries, then the maximal corruption IND-CPA definition (Definition 4) implies the model in [BS23].

EncO outputs Randomness. The IND-CPA and SIM-CPA experiments (Definitions 2 and 3) model *semi-honest* security, in that partial decryptions can only be requested for ciphertexts honestly generated by the experiment. However, even an honest-but-curious adversary $\mathcal{A}$ can observe its own randomness, so a realistic model should indeed allow $\mathcal{A}$ to observe it. Thus, for non-challenge ciphertexts (i.e. ct ’s output by EncO), our model also provides the encryption randomness rnd to $\mathcal{A}$. To our knowledge, this aspect has not been explicitly considered by most prior works [BD10, BGG⁺18, BS23, PS24, MS25] for CPA security, with the only exception being [CLW25] (cf. Table 1).

¹⁴ Some with the minor difference between static and quasi-static corruption, cf. Table 1. While corruption behaviour agrees, the behaviour of oracles differs, see below.

Looking ahead, we give a positive result showing that this extra property allows a generic CPA-to-CCA transform for TPKE in the ROM (Theorem 10), and we give a separation result (Theorem 6) showing that `EncO` not outputting `rnd` leads to a strictly weaker model. All other separation results (Theorems 2 to 5) still hold when `EncO` does not return `rnd`. We note again that `EncO` is redundant in the CCA setting (cf. Remark 1), in which case the aspect on `rnd` does not matter [SG98, BBH06, AT09, LY12, DLN⁺21].

Single vs. Multiple Challenges. Both models allow an adversary $\mathcal{A}$ to query arbitrarily many (both challenge and non-challenge) ciphertexts to model realistic applications. On the one hand, Proposition 1 confirms that IND security against a single challenge (e.g. as in [LY12]) implies IND security against multiple ones (e.g. as in [BS23]), via a straightforward hybrid argument. On the other hand, in the SIM-based setting, we show that under mild assumptions, single-challenge-SIM-CPA is strictly weaker than (multi-challenge-)SIM-CPA (Theorem 5).

Simulator Strength. Definition 3 considers a simulator $\mathcal{S}$ which can (statefully) simulate `KGen`, thus endowing it with additional power. In particular, $\mathcal{S}$ is allowed to simulate all secret keys sk_j , including those for the corrupt parties. We note that this suffices for guaranteeing confidentiality of a TPKE, since $\mathcal{S}$ is required to simulate both the challenge `ct`'s and their partial decryptions `pdj` without knowledge of the plaintext μ , so long as the threshold t has not been reached.

Remark 3 (Variants of Simulation Security). It is possible to consider stronger variants of Definition 3 by restricting the ability of $\mathcal{S}$, for example, disallowing $\mathcal{S}$ to simulate `KGen` and/or ciphertexts, forbidding $\mathcal{S}$ to know the corrupt keys $(\text{sk}_j)_{j \notin \mathcal{C}}$, requiring $\mathcal{S}$ to be stateless (overall or between sessions), etc. For example, the variants given in Appendix B Definitions 22 and 23 will be used for proving Theorem 5.

Remark 4 (On Security Model of [BGG⁺18]). In [BGG⁺18] which constructed TFHE (which served as a building block for the classic universal thresholdiser), the security model is split into two experiments: (1) an IND-CPA experiment forbidding any partial decryption query, and (2) a SIM-based experiment against maximally-corrupting $\mathcal{A}$, which involves a simulator $\mathcal{S}$ for partial decryptions but without guaranteeing confidentiality of `ct` (both under quasi-static corruption). This approach has been followed by e.g. [MS25]. These jointly imply `MaxCor-AdpQ-SIM-CPA` under quasi-static corruption (Definition 4 and Remark 2). On the other hand, the joint modelling of confidentiality and simulatability as in Definition 3 seems to enable new flexibility. For instance, a simulator $\mathcal{S}$ which simulates `KGen` would trivialise experiment (2) above¹⁵, but this is not the case for Definition 3.

Remark 5 (Multiple `ParDecO` queries on Same Input). In both Definitions 2 and 3, when $b = 0$ and `ParDecO` is queried on the same (ct, j) multiple times, each time it runs `ParDec` honestly and obtains a (potentially) fresh partial decryption. We remark that this generality was not always directly achievable in prior works (e.g. [CLW25]). For example, for lattice-based constructions, each partial decryption is typically an LWE sample taking the form $\text{s}_j^\top \mathbf{A} + \mathbf{e}_j^\top \bmod q$ for some partial decryption key s_j and fresh partial decryption randomness $\mathbf{e}_j$. Upon being given multiple fresh partial decryptions $(\text{s}_j^\top \mathbf{A} + \mathbf{e}_{j,i}^\top \bmod q)_i$ on the same query, depending on precise parameter choices, one might attempt statistical attacks, e.g. averaging the errors $\mathbf{e}_{j,i}$ out and recovering s_j . Fortunately, given a scheme that only achieves security with a restricted `ParDecO` answering once to each query (ct, j) , one can generically compile such a scheme into one that supports “freshly generated” answers, by using a PRF to derandomise the `ParDec` algorithm. Specifically, let `ParDec` derive its randomness via $r = \text{PRF}(\text{ct}, \text{sk}_{j, \text{PRF}})$, and use this to run the original (potentially randomised) code of `ParDec`, see Appendix C (Lemma 1) for a formal statement and proof.

3 Implications and Separations

We formalise the implications and separations outlined in Section 2.2, Fig. 1.

¹⁵ This is the case of [BGG⁺18, Eprint, Def.5.5], as also observed by [PS24]. Fortunately, its security proof has not exploited this feature of their definition, and would still go through without $\mathcal{S}$ simulating `KGen` (which yields a meaningful notion).

3.1 Preparatory Implications

First we state a few simple implications. These results are relatively standard, but they will be useful for our subsequent separations.

Proposition 1. *Let $\mathcal{C} \in \{\text{SCor}, \text{AdpCor}\}$, $\mathcal{X} \in \{\text{AdpQ}, \text{SelQ}\}$, and let TPKE be a TPKE scheme.*

- (1) *TPKE is single-challenge $\mathcal{C}\text{-}\mathcal{X}\text{-IND-CPA}$. $\Rightarrow$ TPKE is $\mathcal{C}\text{-}\mathcal{X}\text{-IND-CPA}$.*
- (2) *TPKE is single-challenge $\mathcal{C}\text{-}\mathcal{X}\text{-IND-CCA}$. $\Rightarrow$ TPKE is $\mathcal{C}\text{-}\mathcal{X}\text{-IND-CCA}$.*

The proof of Proposition 1 proceeds via a standard hybrid argument, where the reduction forwards the i th ChalO query as its own ChalO query and simulates answers to other ChalO queries via queries to EncO. See Appendix D.1 for details.

For X-SIM-CPA, it is unclear how to transform a single-challenge simulator into a multi-challenge simulator to obtain an analogue of Proposition 1. We give in Theorem 5 a separation for all simulators that do not simulate KGen.

Next we show that SIM-CPA implies IND-CPA. This implication holds for all variations of the definitions that we are aware of. We state and prove it for the variations which we use in other theorems.

Proposition 2 (SIM $\Rightarrow$ IND). *Let $\mathcal{C} \in \{\text{SCor}, \text{MaxCor}\}$, $\mathcal{X} \in \{\text{AdpQ}, \text{SelQ}\}$, and let TPKE be a TPKE scheme.*

- (1) *TPKE is $\mathcal{C}\text{-}\mathcal{X}\text{-SIM-CPA}$. $\Rightarrow$ TPKE is $\mathcal{C}\text{-}\mathcal{X}\text{-IND-CPA}$.*
- (2) *TPKE is $\mathcal{C}\text{-}\mathcal{X}\text{-SIM-CCA}$. $\Rightarrow$ TPKE is $\mathcal{C}\text{-}\mathcal{X}\text{-IND-CCA}$.*

The proof of Proposition 2 is standard, moving from ChalO encrypting μ_0 to simulated ChalO and then to encrypting μ_1 . See Appendix D.2 for details. Under *selective* queries, the converse implication also holds.

Proposition 3 (SelQ-IND $\Rightarrow$ SelQ-SIM). *Let TPKE be a TPKE scheme.*

- (1) *TPKE is SCor-SelQ-IND-CPA. $\Rightarrow$ TPKE is SCor-SelQ-SIM-CPA.*
- (2) *TPKE is SCor-SelQ-IND-CCA. $\Rightarrow$ TPKE is SCor-SelQ-SIM-CCA.*

For Proposition 3, the simulator essentially runs the real game honestly, except that it encrypts zeroes whenever it does not know a challenge plaintext. The reduction to IND-CPA/CCA can forward $(\mu, 0)$ to its ChalO oracle, since $\mathcal{A}$ never makes ParDecO queries for the resulting ciphertext (as else, the simulator would know the message, since Filter does not remove those). We omit the details.

Remark 6. For Proposition 3 to hold, the simulator indeed needs to simulate KGen, else the equivalence fails for pathological schemes.¹⁶

3.2 Difference between IND and SIM: (Im)possibility of Adaptive Corruption

Prior works have provided TPKE schemes that achieve adaptive corruption in the IND setting [LY12, DLN⁺21] (cf. Table 1). Theorem 1 below shows that in the IND setting, adaptive corruption is strong enough to imply Type II queries.

Theorem 1 (IND Adaptive Corruption). *Let $\mathcal{X} \in \{\text{AdpQ}, \text{SelQ}\}$ and let TPKE be a TPKE scheme.*

- (1) *TPKE is single-challenge AdpCor- $\mathcal{X}\text{-IND-CPA}$ without Type II queries. $\Rightarrow$ TPKE is (multi-challenge) AdpCor- $\mathcal{X}\text{-IND-CPA}$ (with Type II queries).*
- (2) *TPKE is single-challenge AdpCor- $\mathcal{X}\text{-IND-CCA}$ without Type II queries. $\Rightarrow$ TPKE is (multi-challenge) AdpCor- $\mathcal{X}\text{-IND-CCA}$ (with Type II queries).*

¹⁶ If pk includes the image of a one-way function and (some) sk_j contains the preimage which is revealed by ParDecO, then simulating ParDecO is impossible, unless the simulator breaks one-wayness.

In other words, under adaptive corruption, omitting Type II queries is without loss of generality [LY12]. Theorem 1 relies on the observation that, in the single-challenge setting, adaptive corruption can be used by the reduction to simulate the adaptive ParDecO Type II queries. Then, by Proposition 1 single-challenge security implies multi-challenge security. We give the proof in Appendix D.3.

Theorem 1 does not carry over to the SIM setting. In contrast, Theorem 2 establishes that adaptive corruption in the SIM setting is impossible. Looking ahead, we will also see that in the SIM setting, even for static corruption, single-challenge security does not imply multi-challenge security (Theorem 5).

Theorem 2 (Impossibility of SIM-CPA under Adaptive Corruption). *Let $X \in \{\text{AdpQ}, \text{SelQ}\}$. No TPKE scheme TPKE can be $\text{AdpCor-}X\text{-SIM-CPA}$ or $\text{AdpCor-}X\text{-SIM-CCA}$.*

The proof of Theorem 2 is in Appendix D.4. It is a simple adaptation of Nielsen’s impossibility result for non-committing encryption [Nie02]. Similar to Nielsen [Nie02], the arguments of our impossibility also carry over to the CRS model and the *non*-programmable random oracle. However, in the *programmable* random oracle model, the impossibility does not apply.

3.3 On Static and Maximal Corruption

Having seen that adaptive corruption in the SIM setting is impossible, we turn to inspect different security notions under static corruption. In this subsection we investigate the setting of maximal corruption considered in a significant number of prior works (cf. Table 1), and show that under adaptive partial decryption queries, this restriction leads to strictly weaker and arguably inadequate security.

We construct a TPKE' that is $\text{MaxCor-AdpQ-SIM-CCA}$ and SCor-SelQ-SIM-CCA (hence also $\text{MaxCor-AdpQ-IND-CCA}$ and SCor-SelQ-IND-CCA by Proposition 2), but neither SCor-AdpQ-SIM-CPA nor SCor-AdpQ-IND-CPA . Towards this, we first construct a TPKE^* that is $\text{MaxCor-AdpQ-SIM-CPA}$ and SCor-SelQ-SIM-CPA but not SCor-AdpQ-IND-CPA , then compile it into the desired TPKE' using a generic CPA-to-CCA transform to be introduced in Section 4.

Let $\text{TPKE} = (\text{KGen}, \text{Enc}, \text{ParDec}, \text{Rec})$ be a TPKE that is SCor-AdpQ-SIM-CCA . We construct $\text{TPKE}^* = (\text{KGen}^*, \text{Enc}^*, \text{ParDec}^*, \text{Rec}^*)$ in Fig. 4.

Overview. The key generation KGen^* of TPKE^* first checks that $k \geq \lambda$ and $t \leq \frac{k}{2} + 1$, i.e. the threshold t (and thus the number $t - 1$ of corrupted keys which the adversary can obtain) is not too high. If so, the public key pk^* consists of two public keys (pk, pk') of TPKE. The 1st pk is for all k parties and under threshold t , the 2nd pk' is only for $k' = \frac{k}{2}$ parties and under threshold $t' = k' = \frac{k}{2}$. KGen^* distributes the k' secret keys w.r.t. pk' to a random size- k' subset of parties $R \subseteq [k]$. Enc^* encrypts w.r.t. both pk and pk' , ParDec^* partial-decrypts w.r.t. both sets of secret keys, and Rec^* only decrypts the partial decryptions w.r.t. pk .

Insecurity. The scheme TPKE^* in Fig. 4 does not achieve AdpQ-IND-CPA nor AdpQ-SIM-CPA under arbitrary (static) corruption. Consider $t := \frac{k}{2} + 1$. An adversary $\mathcal{A}$ can refrain from corrupting anyone, determine the set R by asking partial decryption queries, then make partial decryption queries to R for a challenge ciphertext ct . This allows $\mathcal{A}$ to distinguish challenge ciphertexts in the AdpQ-IND-CPA game. In the AdpQ-SIM-CPA game, since $|R| = \frac{k}{2} < t$, the simulator $\mathcal{S}$ does not know the challenge message $\mu \leftarrow \mathcal{M}$. Hence it fails to emulate the partial decryption queries with high probability of $1 - 1/|\mathcal{M}|$, for $\mathcal{M}$ the message space.

CPA-Security. Under *maximal* corruption, no partial decryption query for challenge ciphertexts is allowed in the AdpQ-IND-CPA game, whereas in the AdpQ-SIM-CPA game the simulator $\mathcal{S}$ gets the challenge message μ for simulating the partial decryptions. For the SelQ-SIM-CPA and SelQ-IND-CPA games (i.e. arbitrary corruption but selective queries), the adversary $\mathcal{A}$ needs to guess the set R upfront, but since R is a set of size $\frac{k}{2}$ contained in $[k]$, $\mathcal{A}$ needs to guess R amongst exponentially many possibilities and is unlikely to succeed.

$\text{KGen}^*(1^\lambda, \mathbb{A}_{k,t})$	$\text{Enc}^*(\text{pk}^*, \mu)$	$\text{ParDec}^*(\text{sk}^*, \text{ct}^*)$
$(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \$ \text{KGen}(1^\lambda, \mathbb{A}_{k,t})$	$(\text{pk}, \text{pk}') \leftarrow \text{pk}^*$	$(\text{sk}, \text{sk}') \leftarrow \text{sk}^*$
if $k < \lambda$ or $t > \frac{k}{2} + 1$ then	$\text{rnd} \leftarrow \$ \{0, 1\}^{\text{rlen}}$	$(\text{ct}, \text{ct}') \leftarrow \text{ct}^*$
return $((\text{pk}, \perp), (\text{sk}_j, \perp)_{j \in [k]})$	$\text{rnd}' \leftarrow \$ \{0, 1\}^{\text{rlen}}$	$\text{pd} \leftarrow \$ \text{ParDec}(\text{sk}, \text{ct})$
$k' \leftarrow \frac{k}{2}; t' \leftarrow \frac{k}{2}$	$\text{ct} \leftarrow \$ \text{Enc}(\text{pk}, \mu; \text{rnd})$	if $\text{sk}' = \perp$ then
$(\text{pk}', (\text{sk}'_j)_{j \in [k']}) \leftarrow \$ \text{KGen}(1^\lambda, \mathbb{A}_{k',t'})$	if $\text{pk}' = \perp$ then $\text{ct}' \leftarrow \perp$	$\text{pd}' \leftarrow \perp$
$\text{pk}^* \leftarrow (\text{pk}, \text{pk}')$	else $\text{ct}' \leftarrow \$ \text{Enc}(\text{pk}', \mu; \text{rnd}')$	else $\text{pd}' \leftarrow \$ \text{ParDec}(\text{sk}', \text{ct}')$
$R \leftarrow \$ \{S \subseteq [k] : S = \frac{k}{2}\}$	return (ct, ct')	return (pd, pd')
$\text{ctr} \leftarrow 0$	$\text{Rec}^*((\text{pd}'_j)_{j \in T}, \text{ct}^*)$	
for $j \in R$:	$(\text{ct}, \text{ct}') \leftarrow \text{ct}^*$	
$\text{ctr} \leftarrow \text{ctr} + 1; \text{sk}_j^* \leftarrow (\text{sk}_j, \text{sk}'_{\text{ctr}})$	for $j \in T$:	
for $j \in [k] \setminus R$:	$(\text{pd}_j, _) \leftarrow \text{pd}'_j$	
$\text{sk}_j^* \leftarrow (\text{sk}_j, \perp)$	return $\mu \leftarrow \$ \text{Rec}((\text{pd}_j)_{j \in T}, \text{ct})$	
return $(\text{pk}^*, (\text{sk}_j^*)_{j \in [k]})$		

Fig. 4: Construction of pathological $\text{TPKE}^* = (\text{KGen}^*, \text{Enc}^*, \text{ParDec}^*, \text{Rec}^*)$ from $\text{TPKE} = (\text{KGen}, \text{Enc}, \text{ParDec}, \text{Rec})$, such that TPKE^* is secure under maximal, but not under static corruption.

Upgrade to CCA-Security. To upgrade TPKE^* to TPKE' which achieves $\text{MaxCor-AdpQ-SIM-CCA}$ and SCor-SelQ-SIM-CCA , we append a simulation-extractable NIZK certifying the computation of the encryption, which is verified before outputting a partial decryption. This can be seen as a variant of the Naor–Yung transform and is an instance of the SMT transformation to be presented in Section 4. The full transformed scheme TPKE' is provided in Appendix D.5 Fig. 19 for completeness.

We state our formal theorems. Theorem 3 states that, in the adaptive queries setting, considering maximal corruption leads to a strictly weaker security notion, under which a scheme can be insecure against adversaries that can make partial decryption queries.

Theorem 3 ($\text{MaxCor-AdpQ} \not\Rightarrow \text{AdpQ}$). *Let TPKE be a TPKE for access structure family $(\mathfrak{A}_\lambda)_{\lambda \in \mathbb{N}}$ such that for every λ , it holds that $\mathbb{A}_{\frac{\lambda}{2}, \frac{\lambda}{2}}, \mathbb{A}_{\lambda, \frac{\lambda}{2}+1} \in \mathfrak{A}_\lambda$.*

If TPKE is SCor-AdpQ-SIM-CCA , then TPKE^ in Fig. 4 is $\text{MaxCor-AdpQ-SIM-CPA}$, but not SCor-AdpQ-IND-CPA .*

If, additionally, NIZK is a simulation-extractable NIZK, then $\text{TPKE}' = \text{SMT}[\text{TPKE}^, \text{NIZK}]$ is $\text{MaxCor-AdpQ-SIM-CCA}$, but not SCor-AdpQ-IND-CPA , where the transformation SMT is defined in Fig. 8.*

We stated Theorem 3 in a style where we say that TPKE' achieves the strongest possible notion (simulation-based CCA security) under maximal corruption, but does not achieve the weakest possible notion (game-based CPA security) when arbitrary (static) corruption is allowed. This separation simultaneously also establishes all immediate separations based on Proposition 2.

Next, Theorem 4 below establishes that allowing adaptive partial decryption queries is strictly stronger than restricting to only selective partial decryption queries. We state Theorem 4 in the same style as Theorem 3.

Theorem 4 ($\text{SelQ} \not\Rightarrow \text{AdpQ}$). *Let TPKE be a TPKE for access structure family $(\mathfrak{A}_\lambda)_{\lambda \in \mathbb{N}}$ such that for every λ , it holds that $\mathbb{A}_{\frac{\lambda}{2}, \frac{\lambda}{2}}, \mathbb{A}_{\lambda, \frac{\lambda}{2}+1} \in \mathfrak{A}_\lambda$.*

If TPKE is SCor-AdpQ-SIM-CCA , then TPKE^ in Fig. 4 is SCor-SelQ-SIM-CPA , but not SCor-AdpQ-IND-CPA .*

If, additionally, NIZK is a simulation-extractable NIZK, then $\text{TPKE}' = \text{SMT}[\text{TPKE}^, \text{NIZK}]$ is SCor-SelQ-SIM-CCA , but not SCor-AdpQ-IND-CPA , where the transformation SMT is defined in Fig. 8.*

The proofs of Theorems 3 and 4 are given in Appendix D.5.

Remark 7. Theorems 3 and 4 require TPKE to support large thresholds $(k, t) = (\lambda, \lambda/2)$ and $(\lambda, \lambda/2 + 1)$. These parameters can be downscaled to $k \in \omega(1)$, since for the separations, it suffices that $k^{-k/2}$ is negligible.

For comparison, we also observe that in the much weaker setting with only selective partial decryption queries, maximal corruption is without loss of generality. This contrasts with Theorem 3 above.

Proposition 4. *Let TPKE be a TPKE scheme.*

- (1) *TPKE is MaxCor-SelQ-IND-CPA. $\Rightarrow$ TPKE is SCor-SelQ-IND-CPA.*
- (2) *TPKE is MaxCor-SelQ-IND-CCA. $\Rightarrow$ TPKE is SCor-SelQ-IND-CCA.*

By Proposition 1, it suffices to prove Proposition 4 for single-challenge SelQ-IND-CPA/CCA under static corruption. In the single-challenge, selective setting, the adversary declares its corrupt set and partial decryption queries for the single challenge ct upfront. Thus, the reduction can corrupt all these parties (which must be $< t$), as well as further parties unrelated to ct until reaching $t - 1$. The proof is given in Appendix D.6.

By guessing over a polynomial-size set, we have Proposition 5 which generalises to the setting with adaptive queries. Its proof is in Appendix D.7.

Proposition 5. *Let $c \in \mathcal{O}(1)$ and TPKE be a TPKE scheme. Let $t' = t - c \geq 0$.*

- (1) *TPKE is MaxCor-AdpQ-IND-CPA. $\Rightarrow$ TPKE is AdpQ-IND-CPA under t' -corruption.*
- (2) *TPKE is MaxCor-AdpQ-IND-CCA. $\Rightarrow$ TPKE is AdpQ-IND-CCA under t' -corruption.*

3.4 Single-challenge AdpQ-SIM $\nRightarrow$ Multi-challenge AdpQ-SIM

We give a separation between single- and multi-challenge AdpQ-SIM-CPA under static corruption. Due to the freedom that our notion offers to a simulator, a separation is hardly straightforward. To this end, we restrict to the following:

1. We assume the existence of a TPKE scheme TPKE whose simulator $\mathcal{S}$ (I) does not simulate KGen, and (II) is *low-state* in the sense that it does not use state for simulating EncO queries and their associated ParDecO queries. We call this 1St-SIM-CPA security (Definition 23).
2. We achieve a separation for a strengthened SIM-CPA notion, called Hkg-SIM-CPA, where the simulator does not simulate KGen (Definition 22).

The notion of 1St-SIM-CPA (Item 1) is achievable whenever one can simulate partial decryptions given only ct and randomness rnd . For example, we will show that the thresholdised ElGamal PKE satisfies this notion (Theorem 8). Below we summarise how we transform a base TPKE scheme TPKE satisfying Item 1 into a scheme TPKE* in our separation. W.l.o.g. we assume that ParDec of TPKE is deterministic (cf. Lemma 1).

Our core idea is to let ParDec* of TPKE* return a NIZK which proves that the partial decryption response is computed correctly, but attaches a “one-time trapdoor” which allows the simulator $\mathcal{S}^*$ to respond arbitrarily once (while still producing a valid NIZK proof). The trapdoor is such that single-use is not noticeable, but double-use easily allows to distinguish simulated pd_j from real pd_j . Thus, $\mathcal{S}^*$ can embed one challenge response pd_j via the trapdoor, but a two-challenge simulation would require breaking either soundness of the NIZK or using the trapdoor twice (and thus be distinguishable).

Executing the above idea is problematic under Definition 3, since $\mathcal{S}^*$ may exploit its ability to simulate KGen* and program the common-reference-string (CRS) required for the NIZK, so that it may use the trapdoor repeatedly. This leads to the above restriction Item 2, i.e. forbidding $\mathcal{S}^*$ to simulate KGen*.¹⁷ However, this also makes simulating ParDecO queries for non-challenge ciphertexts harder, as $\mathcal{S}^*$ needs to attach a valid proof to the partial decryptions without actually knowing the secret key of all parties.

¹⁷ We do provide $\mathcal{S}$ with all secret key shares. But not the randomness of KGen itself. This is sufficient to embed a CRS for the NIZK.

$\text{KGen}^*(1^\lambda, \mathbb{A}_{k,t})$ $(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \text{KGen}(1^\lambda, \mathbb{A}_{k,t})$ $\text{ck} \leftarrow \text{COM.Setup}(1^\lambda)$ $\text{crs} \leftarrow \text{NIZK.Setup}(1^\lambda)$ $\text{pk}^* \leftarrow (\text{pk}, \text{ck}, \text{crs})$ for $1 \leq j \leq k$: $\rho_j^* \leftarrow \{0, 1\}^\lambda; \tau_j^* \leftarrow \{0, 1\}^\lambda$ $k_{\text{PRF},j}^* \leftarrow \{0, 1\}^\lambda$ $\text{sk}_j^* \leftarrow (\text{sk}_j, \tau_j^*, \rho_j^*, k_{\text{PRF},j}^*)$ return $(\text{pk}^*, (\text{sk}_j^*)_{j \in [k]})$ $\text{Enc}^*(\text{pk}^*, \mu)$ $(\text{pk}, \text{ck}, \text{crs}) \leftarrow \text{pk}^*$ $\text{ct} \leftarrow \text{Enc}(\text{pk}, \mu); \text{pad} \leftarrow \{0, 1\}^{2\lambda}$ return (ct, pad) $\text{R}_{\text{NIZK}}^*(\mathbb{x}, \mathbb{w} = (C_j(\cdot), \tau_j^*, \rho_j^*))$ $(\text{ck}, \text{ct}, \text{pd}_j, \tau_j, \text{cm}_j) \leftarrow \mathbb{x}$ $b_0 \leftarrow (\text{cm}_j = \text{COM.Com}(\text{ck}, (C_j, \tau_j^*); \rho_j^*))$ $b_1 \leftarrow (\text{pd}_j = C_j(\text{ct}^*))$ $b_2 \leftarrow \tau_j = \tau_j^*$ return $(b_0 \wedge b_1) \vee b_2$	$\text{ParDec}^*(\text{sk}_j^*, \text{ct}^*)$ $(\text{sk}_j, \tau_j^*, \rho_j^*, k_{\text{PRF},j}^*) \leftarrow \text{sk}_j^*$ $// \text{Assign deterministic circuit}$ $C_j(\cdot) \leftarrow \text{ParDec}(\text{sk}_j, \cdot)$ $// (\text{Re})\text{Generate commitment}$ $\text{cm}_j \leftarrow \text{Com}(\text{ck}, (C_j, \tau_j^*); \rho_j^*)$ $// \text{Compute } \text{pd}_j \text{ and consistency proof}$ $\text{pd}_j \leftarrow C_j(\text{ct}^*)$ $\tau_j = \text{PRF}(k_{\text{PRF},j}^*, \text{ct}^*) \quad // \text{No trapdoor}$ $\mathbb{x} \leftarrow (\text{ck}, \text{ct}^*, \text{pd}_j, \tau_j, \text{cm}_j)$ $\mathbb{w} \leftarrow (C_j(\cdot), \tau_j^*, \rho_j^*)$ $\pi_j \leftarrow \text{NIZK.Prove}(\text{crs}, \mathbb{x}, \mathbb{w})$ return $\text{pd}_j^* \leftarrow (\text{pd}_j, \tau_j, \text{cm}_j, \pi_j)$ $\text{Rec}^*(\text{pk}^*, (\text{pd}_j^*)_{j \in T}, \text{ct}^*)$ $(\text{pk}, \text{ck}, \text{crs}) \leftarrow \text{pk}^*$ $(\text{ct}, \text{pad}) \leftarrow \text{ct}^*$ for $j \in T$: $(\text{pd}_j, \tau_j, \text{cm}_j, \pi_j) \leftarrow \text{pd}_j^*$ $\mathbb{x}_j = (\text{ck}, \text{ct}, \text{pd}_j, \tau_j, \text{cm}_j)$ $\text{assert } \text{NIZK.Verify}(\text{crs}, \mathbb{x}_j, \pi_j) = 1$ return $\mu \leftarrow \text{Rec}(\text{pk}, (\text{pd}_j)_{j \in T}, \text{ct})$
--	--

Fig. 5: $\text{TPKE}^* = (\text{KGen}^*, \text{Enc}^*, \text{ParDec}^*, \text{Rec}^*)$, using $\text{TPKE} = (\text{KGen}, \text{Enc}, \text{ParDec}, \text{Rec})$, $\text{COM} = (\text{Setup}, \text{Com})$, and $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$.

To circumvent this, we rely on the restriction in Item 1 and make *non-black-box* use of the low-state simulator $\mathcal{S}$ of TPKE . Instead of proving that partial decryptions are correctly computed using ParDec^* , $\mathcal{S}^*$ now only proves that they are consistently computed with the *same* deterministic, stateless program $\mathcal{S}(\cdot, \text{init-st}_{\mathcal{S}})$ from $\mathcal{S}$, with some fixed state $\text{init-st}_{\mathcal{S}}$ hardcoded. To fix this program, pd_j contains each time an identical commitment (computed with the same random string) to the program computing pd_j . As such, $\mathcal{S}^*$ can fix the program once to $\mathcal{S}(\cdot, \text{init-st}_{\mathcal{S}})$.

Finally, one-time simulatability is achieved by turning the NIZK into an OR proof: pd_j additionally contains a pseudorandom value τ_j and a (fixed) commitment to a random value τ_j^* (sampled in KGen), and the OR proof establishes that either the committed circuit was executed, or that $\tau_j^* = \tau_j$. Revealing the same randomness τ_j^* twice is easily distinguishable from pseudorandomness and thus, the trapdoor τ_j^* can only be used once. Summarising, we have:

- a trapdoor (namely, the committed code) that allows $\mathcal{S}^*$ to simulate partial decryptions of all non-challenge ciphertexts, and
- a one-time trapdoor that allows $\mathcal{S}^*$ to bypass the consistent computation check in the NIZK once, to embed an answer to a challenge query.

A multi-challenge simulator would need to bypass the consistent computation check in the NIZK at least twice, which either contradicts its soundness or is easily distinguishable due to revealing τ_j^* twice. In the actual proof, the construction TPKE^* preserves simulation-security under honest key-generation, and $\mathcal{S}^*$ inherits the stateless EncO simulation from $\mathcal{S}$.

$\text{Enc}^*(\text{pk}, \mu)$	$\text{ParDec}^*(\text{sk}_j, (\text{ct}', \text{pk}_{\text{PKE}}))$	$\text{Rec}^*((\text{pd}_j, \text{ct}_{\text{PKE},j})_{j \in T}, \text{ct})$
$\text{ct}' \leftarrow \$ \text{Enc}(\text{pk}, \mu)$	$\text{pd}_j \leftarrow \$ \text{ParDec}(\text{sk}_j, \text{ct}')$	$(\text{ct}', \text{pk}_{\text{PKE}}) \leftarrow \text{ct}$
$(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \$ \text{PKE.KGen}(1^\lambda)$	$\text{ct}_{\text{PKE}} \leftarrow \$ \text{PKE.Enc}(\text{pk}_{\text{PKE}}, \text{sk}_j)$	$\mu \leftarrow \$ \text{Rec}((\text{pd}_{k,j})_{j \in T}, \text{ct}')$
return $(\text{ct}', \text{pk}_{\text{PKE}})$	return $(\text{pd}_j, \text{ct}_{\text{PKE}})$	return μ

Fig. 6: $\text{TPKE}^* = (\text{KGen}^*, \text{Enc}^*, \text{ParDec}^*, \text{Rec}^*)$, using $\text{TPKE} = (\text{KGen}, \text{Enc}, \text{ParDec}, \text{Rec})$ and $\text{PKE} = (\text{KGen}, \text{Enc}, \text{Dec})$. $\text{KGen}^* = \text{KGen}$.

Theorem 5 (Single-challenge AdpQ-SIM-CPA). *Let TPKE be a TPKE which is perfectly correct and AdpQ-1St-SIM-CPA (Definition 23) with deterministic ParDec . Let PRF be a secure PRF, COM be a hiding and straightline extractable binding commitment scheme, and NIZK be a SIMEXT NIZK in the CRS model. Then TPKE^* in Fig. 5 is single-challenge AdpQ-1St-SIM-CPA and single-challenge AdpQ-Hkg-SIM-CPA (Definition 22), but not (multi-challenge) AdpQ-Hkg-SIM-CPA.*

The adversary in the proof of Theorem 5 makes no corruption, one EncO and two ChalO queries, and has noticeable advantage. We provide the details in Appendix D.8. Our simplifying assumptions of perfectness and derandomisation can easily be relaxed, e.g. the latter by Lemma 1.

Remark 8. For selective queries and static corruption, single-challenge SelQ-SIM-CPA is equivalent to (multi-challenge) SelQ-SIM-CPA by Propositions 1 to 3.

3.5 On Encryption Randomness

We show that providing randomness for honest encryptions to the adversary constitutes a strictly stronger security model. Similar to Section 3.3, we start with $\text{TPKE} = (\text{KGen}, \text{Enc}, \text{ParDec}, \text{Rec})$ which is an adaptive SIM-CCA TPKE and modify it into a counterexample $\text{TPKE}^* = (\text{KGen}^*, \text{Enc}^*, \text{ParDec}^*, \text{Rec}^*)$. This time, $\text{KGen}^* = \text{KGen}$, Enc^* additionally outputs a pk_{PKE} for an IND-CPA public key encryption scheme, and ParDec^* encrypts its own secret key under pk_{PKE} . We provide the algorithms Enc^* , ParDec^* and Rec^* in Fig. 6.

Insecurity. We observe that, if $\mathcal{A}$ knows the encryption randomness, it knows the secret key for pk_{PKE} and can decrypt, so that the scheme is insecure, because each partial decryption share leaks the partial decryption key. In turn, if $\mathcal{A}$ does not learn the encryption randomness, pk_{PKE} is a securely generated public key and thus, the TPKE scheme is as secure as without the modification. We state Theorem 6 in the same style as Theorem 3.

Theorem 6 (EncO releasing rnd). *Let $\mathcal{C} \in \{\text{SCor}, \text{AdpCor}\}$, $\mathcal{Y} \in \{\text{IND}, \text{SIM}\}$. If TPKE is a TPKE that is $\mathcal{C}$ -AdpQ- $\mathcal{Y}$ -CPA and PKE is a PKE that is IND-CPA, then TPKE^* in Fig. 6 is $\mathcal{C}$ -AdpQ- $\mathcal{Y}$ -CPA against adversaries that do not receive rnd from their EncO queries, but is not SCor-SelQ-IND-CPA .*

The proof of Theorem 6 is found in Appendix D.9.

4 From CPA to CCA Security I: Semi-Malicious CPA

We investigate how to generalise existing CPA-to-CCA transforms for (non-threshold) PKE to TPKE . In [FP01], Fouque and Pointcheval discuss that the classic Naor–Yung transformation with double encryption [NY90] equally applies in the threshold setting, but their argument has a gap (cf. Footnote 11). Indeed, under CPA and CCA security models with partial decryption queries, we observe that such transformation cannot work generically. As an example, think of a pathological scheme where $\text{ParDec}(\text{sk}_i, \text{ct})$ returns its secret key when decrypting a ciphertext ct which was created with some special, unlikely randomness, e.g. the all-zeroes string. For PKE , this issue can be circumvented by assuming perfect correctness [DNR04]. However, for TPKE , perfect correctness is insufficient, since pd_j can contain additional values that are ignored by Rec .

Below, we consider a strengthened security called *semi-malicious CPA*, and show that it suffices for existing Naor–Yung-style transformations to apply.

```

EncO( $\mu$ , rnd)
ct  $\leftarrow$  Enc(pk,  $\mu$ ; rnd)
 $L_{\text{Enc}} \leftarrow L_{\text{Enc}} \cup \{\text{ct}\}$ 
return (ct, rnd)

```

(a) EncO for semi-malicious IND-CPA.

```

EncO( $\mu$ , rnd)
ct  $\leftarrow$  Enc(pk,  $\mu$ ; rnd)
if  $b = 1$  then  $\text{st}_S \leftarrow \mathcal{S}(\mu, \text{rnd}, \text{st}_S)$ 
 $L_{\text{Enc}} \leftarrow L_{\text{Enc}} \cup \{\text{ct}\}$ ;  $M[\text{ct}] \leftarrow \mu$ 
return (ct, rnd)

```

(b) EncO for semi-malicious SIM-CPA.

Fig. 7: Semi-malicious IND- and SIM-CPA security experiments. All algorithms except EncO are identical to those in Figs. 2 and 3 respectively.

Semi-malicious security. Borrowing similar notions for multi-party computation (e.g. [IKSS22, ASY22, FJK23]), we define semi-malicious CPA security of TPKE, which is almost identical to CPA security, except that an adversary $\mathcal{A}$ is further allowed to query EncO for non-challenge ciphertexts generated using maliciously chosen randomness, i.e. rnd is generated by $\mathcal{A}$ itself. Other than guaranteeing stronger security and being a meaningful notion in itself, we show that a semi-malicious CPA-secure TPKE can be generically lifted to a CCA-secure TPKE by attaching a proof of knowledge of ciphertext well-formedness.

Definition 7 (Semi-malicious CPA). Let $\mathbf{X} \in \{\text{AdpQ}, \text{SelQ}\}$. Define the experiments $\text{SemiMalIndCPA}_{\text{TPKE}, \mathbf{A}, \mathbf{X}}^b$ and $\text{SemiMalSimCPA}_{\text{TPKE}, \mathbf{A}, \mathbf{S}, \mathbf{X}}^b$, which are identical to $\text{IndCPA}_{\text{TPKE}, \mathbf{A}, \mathbf{X}}^b$ and $\text{SimCPA}_{\text{TPKE}, \mathbf{A}, \mathbf{S}, \mathbf{X}}^b$ in Figs. 2 and 3 respectively, except that the oracle EncO is replaced by that in Figs. 7a and 7b respectively. A TPKE scheme is $\text{SCor-X-SemiMal-IND-CPA}$ or $\text{SCor-X-SemiMal-SIM-CPA}$, if the advantage $\text{Adv}_{\text{TPKE}, \mathbf{A}, \mathbf{X}}^{\text{SemiMalIndCPA}}(\lambda) :=$

$$|\Pr[\text{SemiMalIndCPA}_{\text{TPKE}, \mathbf{A}, \mathbf{X}}^0(1^\lambda) = 1] - \Pr[\text{SemiMalIndCPA}_{\text{TPKE}, \mathbf{A}, \mathbf{X}}^1(1^\lambda) = 1]|$$

is negligible, or there is a PPT simulator $\mathcal{S}$ such that $\text{Adv}_{\text{TPKE}, \mathbf{A}, \mathbf{X}}^{\text{SemiMalSimCPA}}(\lambda) :=$

$$|\Pr[\text{SemiMalSimCPA}_{\text{TPKE}, \mathbf{A}, \mathbf{S}, \mathbf{X}}^0(1^\lambda) = 1] - \Pr[\text{SemiMalSimCPA}_{\text{TPKE}, \mathbf{A}, \mathbf{S}, \mathbf{X}}^1(1^\lambda) = 1]|$$

is negligible, respectively.

Remark 9. In Definition 7, oracle EncO returns (ct, rnd) to be syntactically close to other definitions. Since the adversary knows rnd and μ , the oracle could actually omit ct in its output.

Semi-malicious security under adaptive corruption ($\text{AdpCor-X-SemiMal-Y-Z}$) and maximal corruption ($\text{MaxCor-X-SemiMal-Y-Z}$) analogously (cf. Definitions 20 and 21).

Remark 10 (CCA $\Rightarrow$ SemiMal). It is easy to see that CCA-security implies semi-malicious CPA-security. More precisely, for $\mathbf{C} \in \{\text{AdpCor}, \text{SCor}, \text{MaxCor}\}$, $\mathbf{X} \in \{\text{AdpQ}, \text{SelQ}\}$, $\mathbf{M} \in \{\text{IND}, \text{SIM}\}$, if TPKE is $\mathbf{C-X-M-CCA}$, then it is $\mathbf{C-X-SemiMal-SIM-CPA}$. The reduction $\mathcal{R}$ can simply generate the responses to EncO itself (via an honest decryption), and still answer using ParDec. Since we assume CCA-security, this perfectly simulates the real $\mathbf{C-X-M-CCA}$ game ($b = 0$). For $\mathbf{M} = \text{IND}$, $\mathcal{R}$ also simulates $b = 1$ perfectly. For $\mathbf{M} = \text{SIM}$, a subtlety is that $\mathcal{S}$ for TPKE learns μ through EncO. But since $\mathcal{S}$ must also work for $\mathcal{R}$, and $\mathcal{R}$ never queries EncO, we can in fact safely “ignore” the EncO oracle for CCA-security both for $\mathbf{M} = \text{IND}$ and $\mathbf{M} = \text{SIM}$ (cf. Remark 1). Thus, the claim also holds for $\mathbf{M} = \text{SIM}$.

Fig. 8 shows how to transform a semi-malicious-CPA-secure TPKE into a CCA-secure $\text{TPKE}' = \text{SMT}[\text{TPKE}, \text{NIZK}]$ by appending a simulation-extractable NIZK. Theorem 7 is the security claim for TPKE' , its proof is in Appendix E.

Theorem 7 (Semi-Malicious CPA + NIZK $\Rightarrow$ CCA). Let $\mathbf{C} \in \{\text{SCor}, \text{AdpCor}, \text{MaxCor}\}$. Let NIZK be a SIMEXT NIZK for the relation

$$\mathbf{R} = \{((\text{pk}, \text{ct}), (\mu, \text{rnd})) \mid \text{TPKE}.\text{Enc}(\text{pk}, \mu; \text{rnd}) = \text{ct}\}$$

and let TPKE be a TPKE, and $\text{TPKE}' = \text{SMT}[\text{TPKE}, \text{NIZK}]$ as in Fig. 8.

$\text{KGen}'(1^\lambda, \mathbb{A})$	$\text{Enc}'(\text{pk}', \mu)$
$(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \$ \text{KGen}(1^\lambda, \mathbb{A})$	$(\text{pk}, \text{crs}) \leftarrow \text{pk}'$
$\text{crs} \leftarrow \$ \text{NIZK.Setup}(1^\lambda)$	$\text{rnd} \leftarrow \$ \{0, 1\}^{\text{rlen}}$
$\text{pk}' \leftarrow (\text{pk}, \text{crs})$	$\text{ct} \leftarrow \$ \text{Enc}(\text{pk}, \mu; \text{rnd})$
return $(\text{pk}', (\text{sk}_j)_{j \in [k]})$	$\mathbb{x} \leftarrow (\text{pk}, \text{ct})$
	$\mathbb{w} \leftarrow (\mu, \text{rnd})$
$\text{ParDec}'(\text{sk}_j, \text{ct}')$	$\pi \leftarrow \$ \text{NIZK.Prove}(\mathbb{x}, \mathbb{w}, \text{crs})$
$(\text{ct}, \pi) \leftarrow \text{ct}'; \mathbb{x} \leftarrow (\text{pk}, \text{ct})$	return (ct, π)
assert $\text{NIZK.Verify}(\mathbb{x}, \pi, \text{crs}) = 1$	
return $\text{pd}_j \leftarrow \$ \text{ParDec}(\text{sk}_j, \text{ct})$	

Fig. 8: Transformation $\text{TPKE}' = \text{SMT}[\text{TPKE}, \text{NIZK}]$ given NIZK for relation $R = \{((\text{pk}, \text{ct}), (\mu, \text{rnd})) \mid \text{TPKE}.\text{Enc}(\text{pk}, \mu; \text{rnd}) = \text{ct}\}$. We set $\text{Rec}' = \text{Rec}$.

- (1) TPKE is $\text{C-X-SemiMal-SIM-CPA}$. $\Rightarrow \text{TPKE}'$ is C-X-SIM-CCA .
- (2) TPKE is $\text{C-X-SemiMal-IND-CPA}$. $\Rightarrow \text{TPKE}'$ is C-X-IND-CCA .

AdpQ-SIM-CCA-secure TPKE. To substantiate that our security notions are meaningful, we show that they are, despite allowing more adversarial capabilities than most prior models, achievable. Concretely, we show that the natural thresholdised ElGamal PKE achieves **AdpQ-SemiMal-SIM-CPA** (Definition 7). In particular, this immediately implies the CPA notions in Definitions 2 and 3.

Theorem 8. *The ElGamal TPKE (given in Appendix G Fig. 24) is $\text{SCor-AdpQ-SemiMal-SIM-CPA}$ secure, if the DDH assumption holds.*

More precisely, for any PPT algorithm $\mathcal{B}$, there is a PPT algorithm $\mathcal{A}$ against the DDH problem with advantage $\text{Adv}_{\text{GGen}, \mathcal{A}}^{\text{DDH}}(\lambda)$, such that $\text{Adv}_{\text{ElGamal}, \mathcal{B}, \text{AdpQ}}^{\text{SemiMalSimCPA}}(\lambda) \leq t \cdot (\text{Adv}_{\text{GGen}, \mathcal{A}}^{\text{DDH}}(\lambda) + \frac{1}{q-1})$, where t is the threshold and prime q is the group order.

The proof of Theorem 8 is given in Appendix G.2. The following is immediate by combining Theorems 7 and 8.

Corollary 1. *There exists a TPKE scheme that is SCor-AdpQ-SIM-CCA secure, assuming that there is a SIMEXT NIZK and that the DDH assumption holds.*

5 From CPA to CCA Security II: NIPoR

We propose a generic CPA-to-CCA transformation for TPKE, which applies also to schemes that achieve plain CPA-security but are not robust under malicious randomness. To achieve this, we present a new transformation in the ROM, which can be seen as a mix of Naor–Yung and Fujisaki–Okamoto (FO) transformations.¹⁸

The solution pursued in this section is based on an extension of non-interactive proofs of knowledge which we call *non-interactive proofs of randomness (NIPoR)*. At a high level, a NIPoR allows to generate a proof that an efficient function f was evaluated on a secret value m and secret high-entropy randomness r , and resulted in the (public) value $y = f(m; r)$. By setting $f := \text{Enc}(\text{pk}, \cdot; \cdot)$, this allows us to prove that an encryption was computed under pk with honestly sampled and fresh randomness. With this, we instantiate a

¹⁸ The FO transform [FO13] and its variants (see [HHK17] and references therein) are well-known CPA-to-CCA transformations for PKE. However, FO-like transforms are in the ROM and generically uninstantiable [BFM15], with few currently known exceptions [MOZ22]. The approach in this section suffers similar limitations.

variant of the Naor–Yung transformation in the random oracle model, which applies to any (T)PKE. We expect that NIPoRs can find applications in other settings as well.

Alternatively, our transformation may be seen as an instantiation of the FO transform, but with a *publicly verifiable* proof that the encryption randomness is truly random (in the sense of a programmable random oracle output).

5.1 Non-Interactive Proof of Randomness

The NIPoR Prove algorithm generates a value y which is generated with uniform randomness r (but a malicious prover will be able to choose a random string among a polynomial number of options), together with a proof π that $y = f(m; r)$ and a **tag**. Conceptually, in a NIZK generated via Fiat–Shamir in the ROM, **tag** would be the commitment, and r would be (roughly) the output of a random oracle evaluation that also takes **tag** as input and thus, any change in **tag** induces sampling of a fresh random value.

We define NIPoRs for function families $(\mathcal{F}_\lambda)_{\lambda \in \mathbb{N}}$ with associated spaces $(\mathcal{M}_\lambda)_{\lambda \in \mathbb{N}}$ for m and $(\mathcal{R}_\lambda)_{\lambda \in \mathbb{N}}$ for r , such that for all $\lambda \in \mathbb{N}$ and all $f \in \mathcal{F}_\lambda$, the domain is $\mathcal{M}_\lambda \times \mathcal{R}_\lambda$. We require each function $f \in \mathcal{F}_\lambda$ to be identified by a function key fk and the function key spaces for different λ to be disjoint.

Definition 8 (NIPoR). A non-interactive proof of randomness (NIPoR) for a function family $(\mathcal{F}_\lambda)_{\lambda \in \mathbb{N}}$ is a tuple $\text{NIPoR} = (\text{Setup}, \text{Prove}, \text{Verify})$ of PPT algorithms with oracle-access to a random oracle RO , such that

$\text{Setup}^{\text{RO}}(1^\lambda) \xrightarrow{\$} \text{crs}$: The setup algorithm generates a common reference string.

$\text{Prove}^{\text{RO}}(\text{crs}, f_{\text{fk}}, m) \xrightarrow{\$} (y, \text{tag}, \pi)$: The prove algorithm takes a **crs**, a function $f_{\text{fk}} \in \mathcal{F}_\lambda$ and a value $m \in \mathcal{M}_\lambda$ as inputs, and returns an output y , a tag **tag** and proof π .

$\text{Verify}^{\text{RO}}(\text{crs}, f_{\text{fk}}, y, \text{tag}, \pi) \xrightarrow{\$} b$: The verification algorithm takes as input the **crs**, a function f_{fk} , a value y , a proof π and a tag **tag**. It outputs a bit $b \in \{0, 1\}$.

We require NIPoR to be perfectly correct, that is, for all $\lambda \in \mathbb{N}$, any choice of random oracle, $f_{\text{fk}} \in \mathcal{F}_\lambda$ and $m \in \mathcal{M}_\lambda$, it holds that

$$\Pr \left[\text{Verify}^{\text{RO}}(\text{crs}, f_{\text{fk}}, y, \text{tag}, \pi) = 1 \mid \begin{array}{l} \text{crs} \leftarrow \$ \text{Setup}^{\text{RO}}(1^\lambda) \\ (y, \text{tag}, \pi) \leftarrow \$ \text{Prove}^{\text{RO}}(\text{crs}, f_{\text{fk}}, m) \end{array} \right] = 1.$$

Remark 11. As an additional property, Prove could allow the prover to recover the randomness r used to compute $y = f_{\text{fk}}(m; r)$, or output r instead of y . This is useful when the prover should get a secret output, e.g. if f_{fk} is a commitment function and the prover should learn the decommitment. However, since a NIPoR should be a proof that y was generated with honest randomness, returning y seems more natural. (A compromise is to return both y and r with a consistency requirement that $y = f_{\text{fk}}(m; r)$.) Jumping ahead, we note that security could be defined w.r.t. returning r to the adversary (instead of y) in ProveO queries, which would yield a slightly *stronger* security notion than the one we define shortly. Our NIPoR construction, however, evidently satisfies this.

We define security of a NIPoR as (weak) simulation extractability, analogously to simulation extractability for NIZK (Definition 17) for the specific relation $y = f(m; r)$. The crucial *proof of randomness property* is captured by VerifyO_1 (cf. Fig. 9), which is best viewed as an ideal functionality. Concretely, it captures that if the verify *algorithm* (which the adversary can run locally) returns 1 on a tuple $(f_{\text{fk}}, y, \text{tag}, \pi)$, then the value r used to generate y was actually the output of a random oracle evaluated on $(f_{\text{fk}}, \text{tag}, m)$ and thus uniformly random. Otherwise, the assertion in VerifyO_1 fails and the adversary wins.

Random oracles. The random oracle **TrueRand** is part of the ideal functionality and thus only available to the simulator, but not to the distinguisher $\mathcal{A}$. However, since NIPoRs are inherently defined in the random oracle model, the NIPoR algorithms **Setup**, **Verify** and **Prove** need access to a random oracle as well. This random oracle (denoted by RO_b in Fig. 9) can be programmed by the simulator (unlike the ideal functionality **TrueRand**). At first glance, this gives the simulator much freedom, but the checks in the ideal oracle VerifyO_1 constrain it to use the output r from **TrueRand**, which is proper randomness, except that the simulator can query **TrueRand** a polynomial number of times and bias the randomness in this way.

$\boxed{W} \text{SIMEXT}_{\text{NIPoR}, \text{Sim}, \mathcal{A}}^b(1^\lambda)$	$\text{ProveO}_1(f_{fk}, m)$
if $b = 0$: $\text{crs} \leftarrow \text{NIPoR.Setup}^{\text{RO}_0}(1^\lambda)$	$r \leftarrow \mathcal{R}$
if $b = 1$: $(\text{crs}, \text{st}_{\text{Sim}}) \leftarrow \text{Sim.Setup}(1^\lambda)$	$y \leftarrow f(m; r)$
return $\mathcal{A}^{\text{ProveO}_b, \text{VerifyO}_b, \text{RO}_b}(\text{crs})$	$((\text{tag}, \pi), \text{st}_{\text{Sim}}) \leftarrow \text{Sim.Sim}^{\text{TrueRand}}(f_{fk}, y, \text{st}_{\text{Sim}})$
$\text{ProveO}_0(f_{fk}, m)$	$\boxed{\mathcal{L} \leftarrow \mathcal{L} \cup \{(fk, y, \text{tag}, \pi)\}}$
$(y, \text{tag}, \pi) \leftarrow \text{Prove}^{\text{RO}_0}(\text{crs}, f_{fk}, m)$	$\mathcal{L} \leftarrow \mathcal{L} \cup \{(fk, y, \text{tag})\}$
return (y, tag, π)	return (y, tag, π)
$\text{VerifyO}_0(f_{fk}, y, \text{tag}, \pi)$	$\text{VerifyO}_1(f_{fk}, y, \text{tag}, \pi)$
return $\text{Verify}^{\text{RO}_0}(\text{crs}, f_{fk}, y, \text{tag}, \pi)$	if $\text{Verify}^{\text{RO}_1}(\text{crs}, f_{fk}, y, \text{tag}, \pi) = 0$ then return 0
$\text{RO}_0(fk, \text{tag})$	$\boxed{\text{if } (fk, y, \text{tag}, \pi) \in \mathcal{L}}$ // simulated proof
if $\text{RO}[fk, \text{tag}] = \perp$ then	$\boxed{\text{if } (fk, y, \text{tag}) \in \mathcal{L}}$ // $(y, _, \text{tag})$ was simulated
$\text{RO}[fk, \text{tag}] \leftarrow \mathcal{R}$	return 1
return $\text{RO}[fk, \text{tag}]$	$(m, \text{st}_{\text{Sim}}) \leftarrow \text{Sim.Ext}^{\text{TrueRand}}(f_{fk}, y, \text{tag}, \pi, \text{st}_{\text{Sim}})$
$\text{TrueRand}(fk, \text{tag}, m)$	$r \leftarrow \text{TrueRand}(f_{fk}, \text{tag}, m)$
if $\text{TrueRand}[fk, \text{tag}, m] = \perp$ then	assert $y = f_{fk}(m; r)$ // no extraction failure
$\text{TrueRand}[fk, \text{tag}, m] \leftarrow \mathcal{R}$	return 1
return $\text{TrueRand}[fk, \text{tag}, m]$	$\text{RO}_1(fk, \text{tag})$
	$(z, \text{st}_{\text{Sim}}) \leftarrow \text{Sim.RO}^{\text{TrueRand}}(fk, \text{tag}, \text{st}_{\text{Sim}})$
	return z

Fig. 9: Experiments for (weak) SIMEXT for NIPoR. Code in dashed and solid boxes is only executed for SIMEXT and weak SIMEXT respectively.

Definition 9 ((Weak) SIMEXT). A NIPoR for a function family $(\mathcal{F}_\lambda)_{\lambda \in \mathbb{N}}$ is (weakly) simulation-extractable (SIMEXT), if there exists a (stateful) PPT simulator $\text{Sim} = (\text{Setup}, \text{RO}, \text{Sim}, \text{Ext})$ such that for all PPT adversaries $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{A}, \text{NIPoR}, \text{Sim}}^{(\text{W})\text{SIMEXT}}(\lambda) :=$

$$|\Pr[(\text{W})\text{SIMEXT}_{\text{NIPoR}, \text{Sim}, \mathcal{A}}^0(1^\lambda) = 1] - \Pr[(\text{W})\text{SIMEXT}_{\text{NIPoR}, \text{Sim}, \mathcal{A}}^1(1^\lambda) = 1]|$$

is negligible, where the experiments $(\text{W})\text{SIMEXT}_{\text{NIPoR}, \text{Sim}, \mathcal{A}}^b(1^\lambda)$ are defined in Fig. 9. Moreover, we say NIPoR has a tag-only extractor, if Sim.Ext ignores y, π (see Fig. 9). NIPoR has a stateless extractor, if Sim.Ext does not modify its state.

5.2 NIPoR Construction

Let $(\mathcal{R}, +)$ be an additive group whose domain is efficiently sampleable, $\text{COM} = (\text{COM.Setup}, \text{COM.Com})$ a commitment scheme, and $(\mathcal{F}_\lambda)_\lambda$ a function family of polynomial-size circuits f_{fk} specified by function keys fk . Further, let $\text{NIZK} = (\text{Setup}, \text{Verify}, \text{Prove})$ be a NIZK for the relation

$$\mathcal{R}^* := \left\{ ((ck, fk, y, \text{tag}, r''), (m, r', \rho)) \mid \begin{array}{l} y = f_{fk}(m; r' + r'') \\ \wedge \text{tag} = \text{COM.Com}(ck, (r', m); \rho) \end{array} \right\}.$$

From these ingredients, we construct a NIPoR (cf. Fig. 10). Correctness of NIPoR follows immediately from correctness of the NIZK.

Setup(1^λ)	Prove ^{RO} (crs, f_k , m)	Verify ^{RO} (crs, f_k , y , tag, π)
$ck \leftarrow \$ \text{COM.Setup}(1^\lambda)$	$(ck, crs_{\text{NIZK}}) \leftarrow crs$	$(crs_{\text{NIZK}}, ck) \leftarrow crs$
$crs_{\text{NIZK}} \leftarrow \$ \text{NIZK.Setup}(1^\lambda)$	$r' \leftarrow \$ \mathcal{R}; \rho \leftarrow \$ \{0, 1\}^{\text{rlen}}$	$r'' \leftarrow \text{RO}(fk, tag)$
return $crs := (ck, crs_{\text{NIZK}})$	$tag \leftarrow \text{COM.Com}(ck, (r', m); \rho)$	$\mathbb{x} \leftarrow (fk, y, tag, r'')$
	$r'' \leftarrow \text{RO}(fk, tag)$	$b \leftarrow \$ \text{NIZK.Verify}(crs_{\text{NIZK}}, \mathbb{x}, \pi)$
RO (fk, tag)	$r \leftarrow r' + r''$	return b
if $\text{RO}[fk, tag] = \perp$ then	$y \leftarrow f_k(m; r)$	
$r'' \leftarrow \$ \mathcal{R}$	$\mathbb{x} \leftarrow (ck, fk, y, tag, r'')$	
$\text{RO}[fk, tag] \leftarrow r''$	$\mathbb{w} \leftarrow (m, r', \rho)$	
return $\text{RO}[fk, tag]$	$\pi \leftarrow \$ \text{NIZK.Prove}(crs_{\text{NIZK}}, \mathbb{x}, \mathbb{w})$	
	return (y, tag, π)	

Fig. 10: NIPoR = (Setup, Prove, Verify) for function family $(\mathcal{F}_\lambda)_\lambda$.

Theorem 9. *If COM is hiding and straightline extractable binding, and NIZK is (weakly) simulation-extractable for R^* , then NIPoR in Fig. 10 is (weakly) simulation-extractable. Moreover, it has a stateless tag-only extractor.*

We provide the proof of Theorem 9 in Appendix F. Additional background on commitment schemes is found in Appendix A. To obtain a tag-only extractor, we rely on the straightline extractable COM, which commits to both the value m and the randomness share r' . The notion of stateless tag-only extraction is convenient in security reductions, e.g. it leads to our streamlined CPA-to-CCA transformation in Theorem 10.

Remark 12 (Optimisations). The NIPoR in Fig. 10 is geared towards simplicity, its efficiency and generality can be easily improved. For example, we use a standard-model extractable binding commitment (to prove about it in R^*), but this can be relaxed, e.g. using a (succinct) standard-model commitment augmented by a (succinct) straightline extractable proof of knowledge also works.

5.3 CPA-to-CCA Transformation

Our CPA-to-CCA transformation is a variation of Naor–Yung double encryption: Instead of a second encryption, we use a straightline extractable binding commitment scheme — which, in turn, could be instantiated by a (perfectly correct) encryption scheme, leading to a flavour of double encryption under the hood. Instead of a NIZK, we use a NIPoR.

To transform a TPKE scheme $\text{TPKE} = (\text{KGen}, \text{Enc}, \text{ParDec}, \text{Rec})$, we use a NIPoR for the function family in Fig. 11 (bottom left), and obtain the transformed scheme TPKE' in Fig. 11 and the following theorem.

Theorem 10. *Let $\mathbf{C} \in \{\text{SCor}, \text{AdpCor}\}$, $\mathbf{X} \in \{\text{AdpQ}, \text{SelQ}\}$. Let TPKE be a TPKE, COM be a hiding and straightline extractable binding commitment, and NIPoR be a NIPoR for the function family $\{f_k\}_{fk}$ in Fig. 11 that is SIMEXT for random oracle RO and has a stateless tag-only extractor. Let TPKE' be given in Fig. 11.*

- (1) *TPKE is $\mathbf{C}\text{-}\mathbf{X}\text{-SIM-CPA}$. $\Rightarrow$ TPKE' is $\mathbf{C}\text{-}\mathbf{X}\text{-SIM-CCA}$ in the ROM.*
- (2) *TPKE is $\mathbf{C}\text{-}\mathbf{X}\text{-IND-CPA}$. $\Rightarrow$ TPKE' is $\mathbf{C}\text{-}\mathbf{X}\text{-IND-CCA}$ in the ROM.*

The security proof works by programming the randomness in the NIPoR (through programming TrueRand in NIPoR security) to be the randomness obtained from EncO queries in the TPKE CPA-security. Thus, the NIPoR forces every valid non-challenge ciphertext to coincide with a ciphertext obtained through EncO. Here, it is crucial that EncO outputs not only the ciphertext, but also the encryption randomness. For challenge ciphertexts, the reduction simulates the NIPoR. See Appendix F.2 for a formal proof.

$\text{KGen}'(1^\lambda, \mathbb{A})$	$\text{Enc}'(\text{pk}' = (\text{crs}, \text{pk}), \mu)$
$\text{crs} \leftarrow \text{NIPoR.Setup}(1^\lambda)$	$\text{fk} \leftarrow \text{pk}; m \leftarrow (\mu, r_{\text{COM}})$
$(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \text{TPKE.KGen}(1^\lambda, \mathbb{A})$	$(\text{ct}, \text{tag}, \pi) \leftarrow \text{NIPoR.Prove}(\text{crs}, \text{fk}, m)$
$\text{pk}' \leftarrow (\text{crs}, \text{pk})$	return $\text{ct}' = (\text{ct}, \text{tag}, \pi)$
return $(\text{pk}', (\text{pk}', \text{sk}_j)_{j \in [k]})$	$\text{ParDec}'((\text{pk}', \text{sk}_j), \text{ct}')$
$f_{\text{fk}}(\mu; r)$	
$\text{pk} \leftarrow \text{fk}$	$(\text{crs}, \text{pk}) \leftarrow \text{pk}'; (\text{ct}, \text{tag}, \pi) \leftarrow \text{ct}'$
$\text{ct} \leftarrow \text{TPKE.Enc}(\text{pk}, \mu; r)$	$\text{fk} \leftarrow \text{pk}; y \leftarrow \text{ct}$
return ct	if $\text{ct} = \perp$ or $\text{NIPoR.Verify}(\text{crs}, \text{fk}, y, \text{tag}, \pi) = 0$
	return $\perp$
	return $\text{pd}'_j \leftarrow \text{TPKE.ParDec}(\text{sk}_j, \text{ct})$

Fig. 11: Transformation $\text{TPKE}' = (\text{KGen}', \text{Enc}', \text{ParDec}', \text{Rec}')$ of $\text{TPKE} = (\text{KGen}, \text{Enc}, \text{ParDec}, \text{Rec})$ given COM and NIPoR for $\{f_{\text{fk}}\}_{\text{fk}}$. $\text{Rec}' = \text{Rec}$. We assume $\text{TPKE.Enc}(\text{pk}, \perp; r) = \perp$ for any (pk, r) .

Acknowledgments. We thank the anonymous CRYPTO'25 and TCC'25 reviewers for many helpful comments on improving the content of this work and, in particular, for pointing out a flaw in an earlier version. The research of Chris Brzuska and Ivy K. Y. Woo is supported by Research Council of Finland grant 358950. This work was supported by KASTEL Security Research Labs.

References

- AJL⁺12. Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 483–501. Springer, Berlin, Heidelberg, April 2012. doi:10.1007/978-3-642-29011-4_29. 1
- ASY22. Damiano Abram, Peter Scholl, and Sophia Yakoubov. Distributed (correlation) samplers: How to remove a trusted dealer in one round. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 790–820. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-06944-4_27. 7, 19
- AT09. Seiko Arita and Koji Tsurudome. Construction of threshold public-key encryptions through tag-based encryptions. In Michel Abdalla, David Pointcheval, Pierre-Alain Fouque, and Damien Vergnaud, editors, *ACNS 2009*, volume 5536 of *LNCS*, pages 186–200. Springer, Berlin, Heidelberg, June 2009. doi:10.1007/978-3-642-01957-9_12. 3, 5, 11, 12
- BBH06. Dan Boneh, Xavier Boyen, and Shai Halevi. Chosen ciphertext secure public key threshold encryption without random oracles. In David Pointcheval, editor, *CT-RSA 2006*, volume 3860 of *LNCS*, pages 226–243. Springer, Berlin, Heidelberg, February 2006. doi:10.1007/11605805_15. 3, 5, 6, 11, 12
- BBN⁺25a. Dan Boneh, Benedikt Bünz, Kartik Nayak, Lior Rotem, and Victor Shoup. Context-dependent threshold decryption and its applications. Cryptology ePrint Archive, Report 2025/279, 2025. URL: <https://eprint.iacr.org/2025/279>. 6, 7
- BBN⁺25b. Dan Boneh, Benedikt Bünz, Kartik Nayak, Lior Rotem, and Victor Shoup. Context-dependent threshold decryption and its applications. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part VI*, volume 16250 of *LNCS*, pages 506–538. Springer, Singapore, December 2025. doi:10.1007/978-981-95-5119-4_16. 6
- BCF⁺25. Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyang Qu, and Mingyuan Wang. BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Report 2025/1419, 2025. URL: <https://eprint.iacr.org/2025/1419>. 6
- BCK⁺22. Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas

- Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 517–550. Springer, Cham, August 2022. doi:10.1007/978-3-031-15985-5_18. 7
- BD10. Rikke Bendlin and Ivan Damgård. Threshold decryption and zero-knowledge proofs for lattice-based cryptosystems. In Daniele Micciancio, editor, *TCC 2010*, volume 5978 of *LNCS*, pages 201–218. Springer, Berlin, Heidelberg, February 2010. doi:10.1007/978-3-642-11799-2_13. 3, 11, 52, 53
- BDF⁺18. Chris Brzuska, Antoine Delignat-Lavaud, Cédric Fournet, Konrad Kohbrok, and Markulf Kohlweiss. State separation for code-based game-playing proofs. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part III*, volume 11274 of *LNCS*, pages 222–249. Springer, Cham, December 2018. doi:10.1007/978-3-030-03332-3_9. 35
- BFM15. Christina Brzuska, Pooya Farshim, and Arno Mittelbach. Random-oracle uninstantiability from indistinguishability obfuscation. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 428–455. Springer, Berlin, Heidelberg, March 2015. doi:10.1007/978-3-662-46497-7_17. 20
- BFOQ25. Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyang Qu. BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 3457–3476. USENIX Association, August 2025. URL: <https://www.usenix.org/conference/usenixsecurity25/presentation/bormet>. 6
- BFW15. David Bernhard, Marc Fischlin, and Bogdan Warinschi. Adaptive proofs of knowledge in the random oracle model. In Jonathan Katz, editor, *PKC 2015*, volume 9020 of *LNCS*, pages 629–649. Springer, Berlin, Heidelberg, March / April 2015. doi:10.1007/978-3-662-46447-2_28. 7
- BGG⁺18. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, August 2018. doi:10.1007/978-3-319-96884-1_19. 2, 3, 6, 11, 12, 52, 53
- BJSW25. Olivier Bernard, Marc Joye, Nigel P. Smart, and Michael Walter. Drifting towards better error probabilities in fully homomorphic encryption schemes. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 181–211. Springer, Cham, May 2025. doi:10.1007/978-3-031-91101-9_7. 7
- BLT25. Dan Boneh, Evan Laufer, and Ertem Nusret Tas. Batch decryption without epochs and its application to encrypted mempools. Cryptology ePrint Archive, Report 2025/1254, 2025. URL: <https://eprint.iacr.org/2025/1254>. 6
- BP23. Luís TAN Brandão and Rene Peralta. NIST first call for multi-party threshold schemes. URL: <https://csrc.nist.gov/publications/detail/nistir/8214c/draft>, 2023. 1
- BPR24. Dan Boneh, Aditi Partap, and Lior Rotem. Accountability for misbehavior in threshold decryption via threshold traitor tracing. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 317–351. Springer, Cham, August 2024. doi:10.1007/978-3-031-68394-7_11. 6
- BS23. Katharina Boudgoust and Peter Scholl. Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part I*, volume 14438 of *LNCS*, pages 371–404. Springer, Singapore, December 2023. doi:10.1007/978-981-99-8721-4_12. 3, 6, 11, 12
- CCP⁺24. Jung Hee Cheon, Hyeongmin Choe, Alain Passelègue, Damien Stehlé, and Elias Suvanto. Attacks against the IND-CPA^D security of exact FHE schemes. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2505–2519. ACM Press, October 2024. doi:10.1145/3658644.3690341. 7
- CFP⁺24. Sébastien Canard, Caroline Fontaine, Duong Hieu Phan, David Pointcheval, Marc Renard, and Renaud Sirdey. Relations among new CCA security notions for approximate FHE. Cryptology ePrint Archive, Report 2024/812, 2024. URL: <https://eprint.iacr.org/2024/812>. 7
- CG99. Ran Canetti and Shafi Goldwasser. An efficient threshold public key cryptosystem secure against adaptive chosen ciphertext attack. In Jacques Stern, editor, *EUROCRYPT’99*, volume 1592 of *LNCS*, pages 90–106. Springer, Berlin, Heidelberg, May 1999. doi:10.1007/3-540-48910-X_7. 6
- CGPP24. Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/choudhuri>. 6
- CKN03. Ran Canetti, Hugo Krawczyk, and Jesper Buus Nielsen. Relaxing chosen-ciphertext security. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 565–582. Springer, Berlin, Heidelberg, August 2003. doi:10.1007/978-3-540-45146-4_33. 51

- CLW25. Valerio Cini, Russell W. F. Lai, and Ivy K. Y. Woo. Pilvi: Lattice threshold PKE with small decryption shares and improved security. *ASIACRYPT 2025*, 2025. [3](#), [6](#), [11](#), [12](#), [52](#), [53](#)
- CPP25. Sébastien Canard, Nathan Papon, and Duong Hieu Phan. Public traceability in threshold decryption. *CiC*, 2(2):21, 2025. [doi:10.62056/akjb01mo1](#). [6](#)
- CSBB24. Marina Checri, Renaud Sirdey, Aymen Boudguiga, and Jean-Paul Bultel. On the practical CPA^D security of “exact” and threshold FHE schemes and libraries. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 3–33. Springer, Cham, August 2024. [doi:10.1007/978-3-031-68382-4_1](#). [7](#)
- DF90. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *CRYPTO’89*, volume 435 of *LNCS*, pages 307–315. Springer, New York, August 1990. [doi:10.1007/0-387-34805-0_28](#). [6](#)
- DLN⁺21. Julien Devevey, Benoît Libert, Khoa Nguyen, Thomas Peters, and Moti Yung. Non-interactive CCA2-secure threshold cryptosystems: Achieving adaptive security in the standard model without pairings. In Juan Garay, editor, *PKC 2021, Part I*, volume 12710 of *LNCS*, pages 659–690. Springer, Cham, May 2021. [doi:10.1007/978-3-030-75245-3_24](#). [3](#), [12](#), [13](#)
- DNR04. Cynthia Dwork, Moni Naor, and Omer Reingold. Immunizing encryption schemes from decryption errors. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 342–360. Springer, Berlin, Heidelberg, May 2004. [doi:10.1007/978-3-540-24676-3_21](#). [18](#)
- DP08. Cécile Delerablée and David Pointcheval. Dynamic threshold public-key encryption. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 317–334. Springer, Berlin, Heidelberg, August 2008. [doi:10.1007/978-3-540-85174-5_18](#). [6](#)
- EHK⁺13. Alex Escala, Gottfried Herold, Eike Kiltz, Carla Ràfols, and Jorge Villar. An algebraic framework for Diffie-Hellman assumptions. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 129–147. Springer, Berlin, Heidelberg, August 2013. [doi:10.1007/978-3-642-40084-1_8](#). [52](#)
- FJK23. Rex Fernando, Aayush Jain, and Ilan Komargodski. Maliciously-secure MrNISC in the plain model. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 98–128. Springer, Cham, April 2023. [doi:10.1007/978-3-031-30617-4_4](#). [19](#)
- FO13. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013. [doi:10.1007/s00145-011-9114-1](#). [20](#)
- FP01. Pierre-Alain Fouque and David Pointcheval. Threshold cryptosystems secure against chosen-ciphertext attacks. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 351–368. Springer, Berlin, Heidelberg, December 2001. [doi:10.1007/3-540-45682-1_21](#). [5](#), [18](#)
- GKPW24. Sanjam Garg, Dimitris Kolonelos, Guru Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 352–386. Springer, Cham, August 2024. [doi:10.1007/978-3-031-68394-7_12](#). [6](#)
- GNSJ24. Qian Guo, Denis Nabokov, Elias Suvanto, and Thomas Johansson. Key recovery attacks on approximate homomorphic encryption with non-worst-case noise flooding countermeasures. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/guo-qian>. [7](#)
- GRJK00. Rosario Gennaro, Tal Rabin, Stanislaw Jarecki, and Hugo Krawczyk. Robust and efficient sharing of RSA functions. *Journal of Cryptology*, 13(2):273–300, March 2000. [doi:10.1007/s001459910011](#). [6](#)
- HASW25. Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. Silent threshold encryption with one-shot adaptive security. Cryptology ePrint Archive, Report 2025/1384, 2025. URL: <https://eprint.iacr.org/2025/1384>. [6](#)
- HHK17. Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 341–371. Springer, Cham, November 2017. [doi:10.1007/978-3-319-70500-2_12](#). [20](#)
- HJK⁺16. Dennis Hofheinz, Tibor Jager, Dakshita Khurana, Amit Sahai, Brent Waters, and Mark Zhandry. How to generate and use universal samplers. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 715–744. Springer, Berlin, Heidelberg, December 2016. [doi:10.1007/978-3-662-53890-6_24](#). [7](#)
- IKSS22. Yuval Ishai, Dakshita Khurana, Amit Sahai, and Akshayaram Srinivasan. Round-optimal black-box protocol compilers. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 210–240. Springer, Cham, May / June 2022. [doi:10.1007/978-3-031-06944-4_8](#). [19](#)
- LM21. Baiyu Li and Daniele Micciancio. On the security of homomorphic encryption on approximate numbers. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 648–677. Springer, Cham, October 2021. [doi:10.1007/978-3-030-77870-5_23](#). [7](#)

- LY12. Benoît Libert and Moti Yung. Non-interactive CCA-secure threshold cryptosystems with adaptive security: New framework and constructions. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 75–93. Springer, Berlin, Heidelberg, March 2012. doi:10.1007/978-3-642-28914-9_5. 3, 12, 13, 14
- MOZ22. Alice Murphy, Adam O’Neill, and Mohammad Zaheri. Instantiability of classical random-oracle-model encryption transforms. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part IV*, volume 13794 of *LNCS*, pages 323–352. Springer, Cham, December 2022. doi:10.1007/978-3-031-22972-5_12. 20
- MS25. Daniele Micciancio and Adam Suhl. Simulation-secure threshold PKE from LWE with polynomial modulus. *IACR Communications in Cryptology*, 1(4), 2025. doi:10.62056/a0zogy4e-. 3, 11, 12, 52, 53
- Nie02. Jesper Buus Nielsen. Separating random oracle proofs from complexity theoretic proofs: The non-committing encryption case. In Moti Yung, editor, *CRYPTO 2002*, volume 2442 of *LNCS*, pages 111–126. Springer, Berlin, Heidelberg, August 2002. doi:10.1007/3-540-45708-9_8. 5, 14, 36
- NY90. Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *22nd ACM STOC*, pages 427–437. ACM Press, May 1990. doi:10.1145/100216.100273. 5, 7, 18
- OT25. Hiroki Okada and Tsuyoshi Takagi. Low communication threshold FHE from standard (module-)LWE. *ASIACRYPT 2025*, 2025. 6
- PS24. Alain Passelègue and Damien Stehlé. Low communication threshold fully homomorphic encryption. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 297–329. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0875-1_10. 3, 6, 11, 12
- QWZ⁺12. Bo Qin, Qianhong Wu, Lei Zhang, Oriol Farras, and Josep Domingo-Ferrer. Provably secure threshold public-key encryption with adaptive security and short ciphertexts. *Information Sciences*, 210:67–80, 2012. 6
- Sah99. Amit Sahai. Non-malleable non-interactive zero knowledge and adaptive chosen-ciphertext security. In *40th FOCS*, pages 543–553. IEEE Computer Society Press, October 1999. doi:10.1109/SFFCS.1999.814628. 29
- SG98. Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. In Kaisa Nyberg, editor, *EUROCRYPT’98*, volume 1403 of *LNCS*, pages 1–16. Springer, Berlin, Heidelberg, May / June 1998. doi:10.1007/BFb0054113. 3, 5, 11, 12
- Sho00. Victor Shoup. Practical threshold signatures. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 207–220. Springer, Berlin, Heidelberg, May 2000. doi:10.1007/3-540-45539-6_15. 7
- WW25. Brent Waters and David J. Wu. Silent threshold cryptography from pairings: Expressive policies in the plain model. Cryptology ePrint Archive, Report 2025/1547, 2025. URL: <https://eprint.iacr.org/2025/1547>. 6

A Extended Preliminaries

Below are extended preliminaries on some basic cryptographic primitives.

Definition 10 (Pseudorandom Functions (PRF)). Let $\ell : \mathbb{N} \rightarrow \mathbb{N}$. An ℓ -PRF PRF is a deterministic, polynomial-time computable function which, for every $\lambda \in \mathbb{N}$, takes a key $k \in \{0, 1\}^\lambda$, an input $x \in \{0, 1\}^\lambda$ and returns an output $y \in \{0, 1\}^{\ell(\lambda)}$ such that for all PPT $\mathcal{A}$, $|\Pr_{k \leftarrow \mathcal{S}}[1 = \mathcal{A}^{\text{PRF}(k, \cdot)}] - \Pr_{f \leftarrow \mathcal{F}_\lambda}[1 = \mathcal{A}^{f(\cdot, \cdot)}]|$ is negligible, where $\mathcal{F}_\lambda := \{f : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\ell(\lambda)}\}$.

Definition 11 (Public-key Encryption). A public-key encryption (PKE) is a special case of a TPKE, where KGen supports (the trivial) access structure on $k = 1$ party. In this case, the ParDec and Rec algorithms may simply be replaced by a single algorithm called Dec, which internally runs ParDec and then Rec to return a message μ .

We define commitment schemes with setup and canonical opening (i.e., the opening is implicitly the commitment randomness).

Definition 12 (Commitment Scheme). A commitment scheme (with canonical opening) for message spaces $(\mathcal{M}_\lambda)_{\lambda \in \mathbb{N}}$ is a tuple $\text{COM} = (\text{Setup}, \text{Com})$ of PPT algorithms, such that

$\text{Setup}(1^\lambda) \rightarrow \text{ck}$: Generates a commitment key ck .

$\text{Com}(\text{ck}, \mu) \rightarrow \text{cm}$: Given the commitment key ck and a message $\mu \in \mathcal{M}_\lambda$ (and sometimes explicitly some randomness ρ), outputs a commitment cm .

For simplicity, we assume that $\mathcal{M}_\lambda$ is finite for all λ so that we can ignore message length in the definition of the hiding property of a commitment scheme.

Definition 13 (Hiding). A commitment scheme $\text{COM} = (\text{Setup}, \text{Com})$ is said to be hiding, if for all PPT adversaries $\mathcal{A}$, the advantage

$$\text{Adv}_{\text{COM}, \mathcal{A}}^{\text{Hide}}(\lambda) := |\Pr[1 = \text{Hide}_{\text{COM}, \mathcal{A}}^0(1^\lambda)] - \Pr[1 = \text{Hide}_{\text{COM}, \mathcal{A}}^1(1^\lambda)]|$$

is negligible, where $\text{Hide}_{\text{COM}, \mathcal{A}}^b$ is defined in Fig. 12.

The notion of straightline extractable binding asserts that there is an indistinguishable trapdoored setup TSetup and an extractor Ext, which given the trapdoor extracts a message (or $\perp$) from any commitment, and the adversary cannot open the commitment to any message except the extracted one (or $\perp$).

Definition 14 (Straightline Extractable Binding). A commitment scheme $\text{COM} = (\text{Setup}, \text{Com})$ is said to be straightline extractable binding, if there exist PPT algorithms (TSetup, Ext) such that

$\text{TSetup}(1^\lambda) \rightarrow (\text{ck}, \text{td})$: Outputs a commitment key ck and a trapdoor td ,

$\text{Ext}(\text{td}, \text{cm}) \rightarrow \mu$: Deterministically outputs an extracted message $\mu \in \mathcal{M}_\lambda$ or $\perp$,

and the following hold:

- For all PPT adversaries $\mathcal{A}$, the advantage

$$\text{Adv}_{\text{COM}, \text{TSetup}, \text{Ext}, \mathcal{A}}^{\text{Bind}}(\lambda) := \Pr[\text{Bind}_{\text{COM}, \text{TSetup}, \text{Ext}, \mathcal{A}}(1^\lambda) = 1]$$

is negligible, where $\text{Bind}_{\text{COM}, \text{TSetup}, \text{Ext}, \mathcal{A}}$ is defined in Fig. 12, and

- $\{\text{ck} \mid \text{ck} \leftarrow \mathcal{S} \text{Setup}(1^\lambda)\}$ and $\{\text{ck} \mid (\text{ck}, \text{td}) \leftarrow \mathcal{S} \text{TSetup}(1^\lambda)\}$ are computationally indistinguishable.

Straightline extractable binding commitments can be constructed in the obvious way from perfectly correct PKE schemes, or suitable statistically correct encryption schemes.

The notion of unpredictable commitment follows from binding and hiding.

$\text{Hide}_{\text{COM}, \mathcal{A}}^b(1^\lambda)$	$\text{Bind}_{\text{COM}, \text{TSetup}, \text{Ext}, \mathcal{A}}(1^\lambda)$
$\text{ck} \leftarrow \$ \text{Setup}(1^\lambda)$	$(\text{ck}, \text{td}) \leftarrow \$ \text{TSetup}(1^\lambda)$
$(\mu_0, \mu_1, \text{st}_{\mathcal{A}}) \leftarrow \$ \mathcal{A}(1^\lambda, \text{ck})$	$(\text{cm}, \mu, \rho) \leftarrow \$ \mathcal{A}(1^\lambda, \text{ck}, \text{td})$
if $\mu_0 \notin \mathcal{M}_\lambda \vee \mu_1 \notin \mathcal{M}_\lambda$ then	$\mu' \leftarrow \$ \text{Ext}(\text{td}, \text{cm})$
return b	if $\text{Com}(\text{ck}, \mu; \rho) \neq \text{cm} \vee \mu = \perp$
$\text{cm} \leftarrow \$ \text{Com}(\text{ck}, \mu_b)$	return 0 // Invalid opening
$b' \leftarrow \$ \mathcal{A}(\text{cm}, \text{st}_{\mathcal{A}})$	return $\mu \neq \mu'$
return b'	

Fig. 12: Hiding and straightline extractable binding experiments for COM.

$\boxed{W} \text{SIMEXT}_{\text{NIZK}, \text{Sim}, \mathcal{A}}^b(1^\lambda)$	$\text{ProveO}_1(\mathbb{x}, \mathbb{w})$
if $b = 0$: $\text{crs} \leftarrow \$ \text{NIZK.Setup}(1^\lambda)$	assert $(\mathbb{x}, \mathbb{w}) \in \text{R}$
if $b = 1$: $(\text{crs}, \text{st}_{\text{Sim}}) \leftarrow \$ \text{Sim.Setup}(1^\lambda)$	$(\pi, \text{st}_{\text{Sim}}) \leftarrow \$ \text{Sim.Sim}(\mathbb{x}, \text{st}_{\text{Sim}})$
return $\mathcal{A}^{\text{ProveO}_b, \text{VerifyO}_b}(\text{crs})$	$\boxed{W \leftarrow W \cup \{(\mathbb{x}, \pi)\}}$ $W \leftarrow W \cup \{\mathbb{x}\}$
$\text{ProveO}_0(\mathbb{x}, \mathbb{w})$	return π
assert $(\mathbb{x}, \mathbb{w}) \in \text{R}$	$\text{VerifyO}_1(\mathbb{x}, \pi)$
$\pi \leftarrow \$ \text{Prove}(\text{crs}, \mathbb{x}, \mathbb{w})$	if $\text{Verify}(\text{crs}, \mathbb{x}, \pi) = 0$ then return 0
return π	$\boxed{\text{if } (\mathbb{x}, \pi) \in W \text{ then return } 1}$
$\text{VerifyO}_0(\mathbb{x}, \pi)$	$\boxed{\text{if } \mathbb{x} \in W \text{ then return } 1}$
return $\text{Verify}(\text{crs}, \mathbb{x}, \pi)$	$(\mathbb{w}, \text{st}_{\text{Sim}}) \leftarrow \$ \text{Sim.Ext}(\mathbb{x}, \pi, \text{st}_{\text{Sim}})$
	assert $(\mathbb{x}, \mathbb{w}) \in \text{R}$ // no extraction failure
	return 1

Fig. 13: Experiments for (weak) SIMEXT for NIZK. Code in dashed and solid boxes only executed for SIMEXT and weak SIMEXT respectively.

Definition 15 (Unpredictable). A commitment scheme $\text{COM} = (\text{Setup}, \text{Com})$ is unpredictable, if there is a negligible function $\text{negl}(\lambda)$ such that for all λ and all $\text{ck} \leftarrow \text{Setup}(1^\lambda)$, $m \in \mathcal{M}$ and $c \in \{0, 1\}^*$ we have $\Pr[\text{Com}(\text{ck}, m) = c] \leq \text{negl}(\lambda)$, where the probability is over the randomness of Com .

We will consider simulation-extractable (SIMEXT) non-interactive zero-knowledge (NIZK) proofs. SIMEXT captures that an adversary cannot distinguish real proof/verify oracles from simulation/extraction oracles, where extraction must succeed for all pairs $(\mathbb{x}, \pi)$ of statements and proofs where π was not the output of a proof query. See Sahai [Sah99] for further discussion and for a construction in the CRS model.

Definition 16 (Non-interactive Proof System). A non-interactive proof system for NP-relation R in the CRS model is a tuple $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$ with the syntax:

$\text{Setup}(1^\lambda) \mapsto \text{crs}$: Generates a common reference string.

$\text{Prove}(\text{crs}, \mathbb{x}, \mathbb{w}) \mapsto \pi$: Given $(\mathbb{x}, \mathbb{w}) \in \text{R}$, outputs a proof π .

$\text{Verify}(\text{crs}, \mathbb{x}, \pi) \mapsto b$: Given statement $\mathbb{x}$ and purported proof π , outputs a verdict $b \in \{0, 1\}$.

We require NIZK to be perfectly correct for R , that is, for all $(\mathbb{x}, \mathbb{w}) \in \text{R}$ and all $\text{crs} \in \text{Setup}(1^\lambda)$,

$$\Pr[1 = \text{Verify}(\text{crs}, \mathbb{x}, \pi) \mid \pi \leftarrow \$ \text{Prove}(\text{crs}, \mathbb{x}, \mathbb{w})] = 1.$$

$\text{IndCCA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^b(1^\lambda)$	$\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^b(1^\lambda)$	$\text{EncO}(\mu)$
$(\mathbb{A}_{k,t}, \mathcal{C}, Q_{\text{Enc}}, Q_{\text{Chal}}, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$ <i>// Corrupt set $\mathcal{C}$, Tuple of queries</i> assert $(\mathcal{C} \subset [k]) \wedge (\mathcal{C} \notin \mathbb{A}_{k,t}) \wedge \mathbb{A}_{k,t} \in \mathfrak{A}_\lambda$ $(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \text{KGen}(1^\lambda, \mathbb{A}_{k,t})$ $L_{\text{Enc}} \leftarrow \emptyset; L_{\text{Chal}} \leftarrow \emptyset; P[\cdot] \leftarrow \emptyset$ $\text{st}_{\mathcal{A}} \leftarrow (\text{pk}, (\text{sk}_j)_{j \in \mathcal{C}}, \text{st}_{\mathcal{A}})$ if $\mathbf{X} = \text{SelQ}$ then $\text{SelQueries} \leftarrow \text{SampSelQ}(Q_{\text{Enc}}, Q_{\text{Chal}})$ $b' \leftarrow \mathcal{A}(\text{SelQueries}, \text{st}_{\mathcal{A}})$ if $\mathbf{X} = \text{AdpQ}$ then $b' \leftarrow \mathcal{A}^{\text{EncO}, \text{ChalO}, \text{ParDecO}}(\text{st}_{\mathcal{A}})$ return b'		$\text{ChalO}(\mu_0, \mu_1)$
$\text{SampSelQ}(Q_{\text{Enc}}, Q_{\text{Chal}})$		$\text{ParDecO}(\text{ct}, j)$
for $q = (\mu, T) \in Q_{\text{Enc}} :$ $(\text{ct}_q, \text{rnd}_q) \leftarrow \text{EncO}(\mu)$ for $q = ((\mu_0, \mu_1), T) \in Q_{\text{Chal}} :$ $\text{ct}_q \leftarrow \text{ChalO}(\mu_0, \mu_1)$ for $q = (*, T) \in Q_{\text{Enc}} \cup Q_{\text{Chal}} :$ $\text{pd}_{q,j} \leftarrow \text{ParDecO}(\text{ct}_q, j) \quad \forall j \in T$ return $\text{SelQueries} := ((\text{ct}_q, (\text{pd}_{q,j})_j)_{q \in Q}, (\text{rnd}_q)_{q \in Q_{\text{Enc}}})$		return ct
		assert $\text{ct} \in L_{\text{Enc}} \cup L_{\text{Chal}}$
		if $\text{ct} \in L_{\text{Chal}}$ then $P[\text{ct}] \leftarrow P[\text{ct}] \cup \{j\}$ assert $P[\text{ct}] \cup \mathcal{C} \notin \mathbb{A}_{k,t}$ $\text{pd}_j \leftarrow \text{ParDec}(\text{sk}_j, \text{ct})$ return pd_j

Fig. 14: $\text{IndCCA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^b(1^\lambda)$ and $\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^b(1^\lambda)$ experiments for TPKE with $\mathbf{X} \in \{\text{SelQ}, \text{AdpQ}\}$. Grey code only applies to $\mathbf{X} = \text{SelQ}$. Boxed code only applies to $\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^b(1^\lambda)$.

Definition 17 ((Weak) SIMEXT). A non-interactive proof system $\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$ is (weakly) simulation-extractable (SIMEXT) if there is a PPT simulator $\text{Sim} = (\text{Setup}, \text{Sim}, \text{Ext})$ such that for all PPT adversaries $\mathcal{A}$, the advantage

$$\text{Adv}_{\text{NIZK}, \text{Sim}, \mathcal{A}}^{(\text{W})\text{SIMEXT}}(\lambda) := |\Pr[1 = (\text{W})\text{SIMEXT}_{\text{NIZK}, \text{Sim}, \mathcal{A}}^0(1^\lambda)] - \Pr[1 = (\text{W})\text{SIMEXT}_{\text{NIZK}, \text{Sim}, \mathcal{A}}^1(1^\lambda)]|$$

is negligible, where the experiments $(\text{W})\text{SIMEXT}_{\text{NIZK}, \text{Sim}, \mathcal{A}}^b$ are defined in Fig. 13. For convenience, we require Ext to be correct, that is, whenever $\text{Ext}(\text{td}, \mathbb{X}, \pi)$ outputs $\mathbb{W} \neq \perp$, then $(\mathbb{X}, \mathbb{W}) \in \mathcal{R}$.

B Further Security Definitions

This appendix states all definitional variants formally considered in this work.

Definitions 18 and 19 state IND and SIM security but with selective queries. For comparison, the respective security experiment (Figs. 14 and 15) includes also the corresponding adaptive queries setting presented in Figs. 2 and 3.

Definition 18 (SelQ-IND). A TPKE scheme TPKE is secure under static corruption, selective challenge queries and chosen-plaintext attacks (SelQ-IND-CPA) or chosen-ciphertext attacks (SelQ-IND-CCA), respectively, if for all PPT adversaries $\mathcal{A}$, the advantage

$$\text{Adv}_{\text{TPKE}, \mathcal{A}, \text{SelQ}}^{\text{IndCPA}}(\lambda) := |\Pr[\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \text{SelQ}}^0(1^\lambda) = 1] - \Pr[\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \text{SelQ}}^1(1^\lambda) = 1]|,$$

$\text{SimCCA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^b(1^\lambda)$	$\text{SimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^b(1^\lambda)$	$\text{EncO}(\mu)$
$(\mathbb{A}_{k,t}, \mathcal{C}, Q_{\text{Enc}}, Q_{\text{Chal}}, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$ <i>// Corrupt set $\mathcal{C}$, Tuple of queries</i> assert $(\mathcal{C} \subset [k]) \wedge (\mathcal{C} \not\subseteq \mathbb{A}_{k,t}) \wedge \mathbb{A}_{k,t} \in \mathcal{A}_\lambda$ if $b = 0$ then $(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \text{KGen}(1^\lambda, \mathbb{A}_{k,t})$ if $b = 1$ then $(\text{pk}, (\text{sk}_j)_{j \in \mathcal{C}}, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(1^\lambda, \mathbb{A}_{k,t}, \mathcal{C}, Q_{\text{Enc}}, \text{Filter}_t(Q_{\text{Chal}}))$ $L_{\text{Enc}} \leftarrow \emptyset; L_{\text{Chal}} \leftarrow \emptyset; P[\cdot] \leftarrow \emptyset; M[\cdot] \leftarrow \emptyset$ $\text{st}_{\mathcal{A}} \leftarrow (\text{pk}, (\text{sk}_j)_{j \in \mathcal{C}}, \text{st}_{\mathcal{A}})$ if $\mathbf{X} = \text{SelQ}$ then $\text{SelQueries} \leftarrow \text{SampSelQ}(Q_{\text{Enc}}, Q_{\text{Chal}})$ $b' \leftarrow \mathcal{A}(\text{SelQueries}, \text{st}_{\mathcal{A}})$ if $\mathbf{X} = \text{AdpQ}$ then $b' \leftarrow \mathcal{A}^{\text{EncO}, \text{ChalO}, \text{ParDecO}}(\text{st}_{\mathcal{A}})$ return b'	$\text{ChalO}(\mu)$ if $b = 0$ then $\text{ct} \leftarrow \text{Enc}(\text{pk}, \mu)$ if $b = 1$ then $(\text{ct}, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\text{st}_{\mathcal{S}})$ $L_{\text{Chal}} \leftarrow L_{\text{Chal}} \cup \{\text{ct}\}; M[\text{ct}] \leftarrow \mu$ return ct	if $b = 0$ then $\text{rnd} \leftarrow \{0, 1\}^{\text{rlen}(\lambda)}$ $\text{ct} \leftarrow \text{Enc}(\text{pk}, \mu; \text{rnd})$ if $b = 1$ then $(\text{ct}, \text{rnd}, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\mu, \text{st}_{\mathcal{S}})$ $L_{\text{Enc}} \leftarrow L_{\text{Enc}} \cup \{\text{ct}\}; M[\text{ct}] \leftarrow \mu$ return (ct, rnd)
$\text{SampSelQ}(Q_{\text{Enc}}, Q_{\text{Chal}})$ for $q = (\mu, T) \in Q_{\text{Enc}}$: $(\text{ct}_q, \text{rnd}_q) \leftarrow \text{EncO}(\mu)$ for $q = (\mu, T) \in Q_{\text{Chal}}$: $\text{ct}_q \leftarrow \text{ChalO}(\mu)$ for $q = (\mu, T) \in Q_{\text{Enc}} \cup Q_{\text{Chal}}$: $\text{pd}_{q,j} \leftarrow \text{ParDecO}(\text{ct}_q, j) \quad \forall j \in T$ return $\text{SelQueries} := ((\text{ct}_q, (\text{pd}_{q,j})_j)_{q \in Q}, (\text{rnd}_q)_{q \in Q_{\text{Enc}}})$	$\text{ParDecO}(\text{ct}, j)$ assert $\text{ct} \in L_{\text{Enc}} \cup L_{\text{Chal}}$ $P[\text{ct}] \leftarrow P[\text{ct}] \cup \{j\}$ if $b = 0$ then $\text{pd}_j \leftarrow \text{ParDec}(\text{sk}_j, \text{ct})$ if $b = 1$ then if $P[\text{ct}] \cup \mathcal{C} \in \mathbb{A}_{k,t} \wedge \text{ct} \in L_{\text{Chal}}$ then $\mu \leftarrow M[\text{ct}]$ $(\text{pd}_j, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\text{ct}, \mu, j, \text{st}_{\mathcal{S}})$ else $(\text{pd}_j, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\text{ct}, j, \text{st}_{\mathcal{S}})$ return pd_j	

Fig. 15: $\text{SimCCA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^b(1^\lambda)$ and $\text{SimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^b(1^\lambda)$ experiments for TPKE with $\mathbf{X} \in \{\text{SelQ}, \text{AdpQ}\}$. Grey code only applies to $\mathbf{X} = \text{SelQ}$. $\text{Filter}_t(Q_{\text{Chal}})$ returns $\{(\mu, T) \in Q_{\text{Chal}} \wedge |T| \geq t\} \cup \{(\perp, T) : (\mu, T) \in Q_{\text{Chal}} \wedge |T| < t\}$. Boxed code only applies to $\text{SimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^b(1^\lambda)$.

$$\text{or } \text{Adv}_{\text{TPKE}, \mathcal{A}, \text{SelQ}}^{\text{IndCCA}}(\lambda) := |\Pr[\text{IndCCA}_{\text{TPKE}, \mathcal{A}, \text{SelQ}}^0(1^\lambda) = 1] - \Pr[\text{IndCCA}_{\text{TPKE}, \mathcal{A}, \text{SelQ}}^1(1^\lambda) = 1]|,$$

is negligible, respectively, where the security experiments are defined in Fig. 14.

Definition 19 (SelQ-SIM). A TPKE scheme TPKE is simulation-secure under static corruption, selective challenge queries and chosen-plaintext attacks (SelQ-SIM-CPA) or chosen-ciphertext attacks (SelQ-SIM-CCA), respectively, if there is a PPT simulator $\mathcal{S}$, such that for all PPT adversaries $\mathcal{A}$ the advantage

$$\begin{aligned} \text{Adv}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \text{SelQ}}^{\text{SimCPA}}(\lambda) &:= |\Pr[\text{SimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \text{SelQ}}^0(1^\lambda) = 1] - \Pr[\text{SimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \text{SelQ}}^1(1^\lambda) = 1]|, \\ \text{or } \text{Adv}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \text{SelQ}}^{\text{SimCCA}}(\lambda) &:= |\Pr[\text{SimCCA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \text{SelQ}}^0(1^\lambda) = 1] - \Pr[\text{SimCCA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \text{SelQ}}^1(1^\lambda) = 1]|, \end{aligned}$$

is negligible, respectively, where the security experiments are defined in Fig. 15.

Next we state the security definitions under adaptive corruption. Definition 20 is IND-CPA and IND-CCA with adaptive corruption, which is used in Theorem 1. Definition 21 is SIM-CPA with adaptive corruption, which is used in Theorem 2.

$\text{AdpCorIndCCA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^b(1^\lambda)$	$\text{AdpCorIndCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^b(1^\lambda)$
$(\mathbb{A}_{k,t}, Q_{\text{Enc}}, Q_{\text{Chal}}, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$ // Tuple of queries assert $\mathbb{A}_{k,t} \in \mathfrak{A}_\lambda$ $(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \text{KGen}(1^\lambda, \mathbb{A}_{k,t})$ $L_{\text{Enc}} \leftarrow \emptyset; L_{\text{Chal}} \leftarrow \emptyset; P[\cdot] \leftarrow \emptyset; \mathcal{C} \leftarrow \emptyset$ $\text{st}_{\mathcal{A}} \leftarrow (\text{pk}, \text{st}_{\mathcal{A}})$ if $\mathbf{X} = \text{SelQ}$ then $\text{SelQueries} \leftarrow \text{SampSelQ}(Q_{\text{Enc}}, Q_{\text{Chal}})$ $b' \leftarrow \mathcal{A}^{\text{CorO}}(\text{SelQueries}, \text{st}_{\mathcal{A}})$ if $\mathbf{X} = \text{AdpQ}$ then $b' \leftarrow \mathcal{A}^{\text{EncO}, \text{ChalO}, \text{ParDecO}, \text{CorO}}(\text{st}_{\mathcal{A}})$ return b' $\text{CorO}(j)$ assert $P[\text{ct}] \cup \mathcal{C} \cup \{j\} \notin \mathbb{A}_{k,t} \quad \forall \text{ct} \in L_{\text{Chal}}$ $\mathcal{C} \leftarrow \mathcal{C} \cup \{j\}$ return sk_j	

(a) $\text{AdpCorIndCCA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^b(1^\lambda)$ and $\text{AdpCorIndCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^b(1^\lambda)$ experiments for TPKE with $\mathbf{X} \in \{\text{SelQ}, \text{AdpQ}\}$. Grey code only applies to $\mathbf{X} = \text{SelQ}$. The algorithm SampSelQ and oracles EncO , ChalO and ParDecO are identical to those in Fig. 14 and omitted.

$\text{AdpCorSimCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{S}, \mathbf{X}}^b(1^\lambda)$
$(\mathbb{A}_{k,t}, Q_{\text{Enc}}, Q_{\text{Chal}}, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$ // Tuple of queries assert $\mathbb{A}_{k,t} \in \mathfrak{A}_\lambda$ if $b = 0$ then $(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \text{KGen}(1^\lambda, \mathbb{A}_{k,t})$ if $b = 1$ then $(\text{pk}, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(1^\lambda, \mathbb{A}_{k,t}, Q_{\text{Enc}}, \text{Filter}_t(Q_{\text{Chal}}))$ $L_{\text{Enc}} \leftarrow \emptyset; L_{\text{Chal}} \leftarrow \emptyset; P[\cdot] \leftarrow \emptyset; M[\cdot] \leftarrow \emptyset; \mathcal{C} \leftarrow \emptyset$ $\text{st}_{\mathcal{A}} \leftarrow (\text{pk}, \text{st}_{\mathcal{A}})$ if $\mathbf{X} = \text{SelQ}$ then $\text{SelQueries} \leftarrow \text{SampSelQ}(Q_{\text{Enc}}, Q_{\text{Chal}})$ $b' \leftarrow \mathcal{A}^{\text{CorO}}(\text{SelQueries}, \text{st}_{\mathcal{A}})$ if $\mathbf{X} = \text{AdpQ}$ then $b' \leftarrow \mathcal{A}^{\text{EncO}, \text{ChalO}, \text{ParDecO}, \text{CorO}}(\text{st}_{\mathcal{A}})$ return b' $\text{CorO}(j)$ $\mathcal{C} \leftarrow \mathcal{C} \cup \{j\}$ if $b = 1$ then $\mathcal{M} \leftarrow \left\{ (\text{ct}, \mu) : \begin{array}{l} (\text{ct} \in L_{\text{Enc}} \cup L_{\text{Chal}}) \wedge (M[\text{ct}] = \mu) \\ \wedge (P[\text{ct}] \cup \mathcal{C} \in \mathbb{A}_{k,t}) \end{array} \right\}$ $(\text{sk}_j, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\mathcal{M}, \text{st}_{\mathcal{S}})$ return sk_j

(b) $\text{AdpCorSimCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{S}, \mathbf{X}}^b(1^\lambda)$ experiments for TPKE, with $\mathbf{X} \in \{\text{SelQ}, \text{AdpQ}\}$. Grey code only applies to $\mathbf{X} = \text{SelQ}$. The algorithm SampSelQ and oracles EncO , ChalO and ParDecO are identical to those in Fig. 15 and omitted.

Fig. 16: TPKE security experiments for adaptive corruption.

Definition 20 (IND under Adaptive Corruption). Let $\mathbf{X} \in \{\text{AdpQ}, \text{SelQ}\}$. A TPKE scheme TPKE is secure under adaptive corruption, $\mathbf{X}$ challenge queries and chosen-plaintext attacks (AdpCor-X-IND-CPA) or chosen-ciphertext attacks (AdpCor-X-IND-CCA), respectively, if for all PPT adversaries $\mathcal{A}$, the advantage

$$\begin{aligned} \text{Adv}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^{\text{AdpCorIndCPA}}(\lambda) &:= |\Pr[\text{AdpCorIndCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^0(1^\lambda) = 1] - \Pr[\text{AdpCorIndCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^1(1^\lambda) = 1]|, \\ \text{or } \text{Adv}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^{\text{AdpCorIndCCA}}(\lambda) &:= |\Pr[\text{AdpCorIndCCA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^0(1^\lambda) = 1] - \Pr[\text{AdpCorIndCCA}_{\text{TPKE}, \mathcal{A}, \mathbf{X}}^1(1^\lambda) = 1]|, \end{aligned}$$

is negligible, respectively, where the security experiments are defined in Fig. 16a.

Definition 21 (SIM-CPA under Adaptive Corruption). Let $\mathbf{X} \in \{\text{AdpQ}, \text{SelQ}\}$. A TPKE scheme TPKE is simulation-secure under adaptive corruption and $\mathbf{X}$ challenge queries (AdpCor-X-SIM-CPA) if there is a PPT simulator $\mathcal{S}$, such that for any PPT adversary $\mathcal{A}$, the advantage

$$\text{Adv}_{\text{TPKE}, \mathcal{A}, \mathbf{S}, \mathbf{X}}^{\text{AdpCorSimCPA}}(\lambda) := |\Pr[\text{AdpCorSimCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{S}, \mathbf{X}}^0(1^\lambda) = 1] - \Pr[\text{AdpCorSimCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{S}, \mathbf{X}}^1(1^\lambda) = 1]|$$

is negligible, where $\text{AdpCorSimCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{S}, \mathbf{X}}^b$ is defined in Fig. 16b.

HkgSimCPA _{TPKE, A, S, X} ^b (1 ^λ)	1StSimCPA _{TPKE, A, S, X} ^b (1 ^λ)
$(\mathbb{A}_{k,t}, \mathcal{C}, Q_{\text{Enc}}, Q_{\text{Chal}}, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$ <i>// Corrupt set C, Tuple of queries</i> assert $(\mathcal{C} \subset [k]) \wedge (\mathcal{C} \notin \mathbb{A}_{k,t}) \wedge \mathbb{A}_{k,t} \in \mathfrak{A}_\lambda$ $(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \text{KGen}(1^\lambda, \mathbb{A}_{k,t})$ $L_{\text{Enc}} \leftarrow \emptyset; L_{\text{Chal}} \leftarrow \emptyset; P[\cdot] \leftarrow \emptyset; M[\cdot] \leftarrow \emptyset$ $\text{st}_{\mathcal{A}} \leftarrow (\text{pk}, (\text{sk}_j)_{j \in \mathcal{C}}, \text{st}_{\mathcal{A}})$ $\text{st}_{\mathcal{S}} \leftarrow \mathcal{S}(\text{pk}, (\text{sk}_j)_{j \in \mathcal{C}}, Q_{\text{Enc}}, \text{Filter}_t(Q_{\text{Chal}}))$ <i>// Simulator S is called only after completed setup.</i>	if $b = 0$ then $\text{rnd} \leftarrow \{0, 1\}^{\text{rlen}(\lambda)}$ $\text{ct} \leftarrow \text{Enc}(\text{pk}, \mu; \text{rnd})$ if $b = 1$ then $(\text{ct}, \text{rnd}, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\mu, \text{st}_{\mathcal{S}})$ $(\text{ct}, \text{rnd}) \leftarrow \mathcal{S}(\mu, \text{init-st}_{\mathcal{S}})$ <i>// Use initial state only</i> $L_{\mathcal{S}}[\text{ct}] \leftarrow (\mu, \text{rnd})$ <i>// Explicit storage</i>
$\text{init-st}_{\mathcal{S}} \leftarrow \text{st}_{\mathcal{S}}$ <i>// Store initial state</i> if $\mathbf{X} = \text{SelQ}$ then for $q = (\mu, T) \in Q_{\text{Enc}}$: $(\text{ct}_q, \text{rnd}_q) \leftarrow \text{EncO}(\mu)$ for $q = (\mu, T) \in Q_{\text{Chal}}$: $\text{ct}_q \leftarrow \text{ChalO}(\mu)$ for $q = (\mu, T) \in Q_{\text{Enc}} \cup Q_{\text{Chal}}$: $\text{pd}_{q,j} \leftarrow \text{ParDecO}(\text{ct}_q, j) \quad \forall j \in T$ $b' \leftarrow \mathcal{A}((\text{ct}_q, (\text{pd}_{q,j})_j)_{q \in Q}, (\text{rnd}_q)_{q \in Q_{\text{Enc}}}, \text{st}_{\mathcal{A}})$ if $\mathbf{X} = \text{AdpQ}$ then $b' \leftarrow \mathcal{A}^{\text{EncO}, \text{ChalO}, \text{ParDecO}}(\text{st}_{\mathcal{A}})$ return b'	$L_{\text{Enc}} \leftarrow L_{\text{Enc}} \cup \{\text{ct}\}; M[\text{ct}] \leftarrow \mu$ return (ct, rnd)
$\text{ChalO}(\mu)$ if $b = 0$ then $\text{ct} \leftarrow \text{Enc}(\text{pk}, \mu)$ if $b = 1$ then $(\text{ct}, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\text{st}_{\mathcal{S}})$ $L_{\text{Chal}} \leftarrow L_{\text{Chal}} \cup \{\text{ct}\}; M[\text{ct}] \leftarrow \mu$ return ct	$\text{ParDecO}(\text{ct}, j)$ assert $\text{ct} \in L_{\text{Enc}} \cup L_{\text{Chal}}$ $P[\text{ct}] \leftarrow P[\text{ct}] \cup \{j\}$ if $b = 0$ then $\text{pd}_j \leftarrow \text{ParDec}(\text{sk}_j, \text{ct})$ if $b = 1$ then if $P[\text{ct}] \cup \mathcal{C} \in \mathbb{A}_{k,t} \wedge \text{ct} \in L_{\text{Chal}}$ then $\mu \leftarrow M[\text{ct}]$ $(\text{pd}_j, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\text{ct}, \mu, j, \text{st}_{\mathcal{S}})$ elseif $\text{ct} \in L_{\text{Enc}}$ then $(\mu, \text{rnd}) \leftarrow L_{\mathcal{S}}[\text{ct}]$ $\text{pd}_j \leftarrow \mathcal{S}(\text{ct}, \mu, \text{rnd}, \text{init-st}_{\mathcal{S}})$ <i>// Stateless EncO simulation only</i> <i>// uses randomness & message</i> else $(\text{pd}_j, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\text{ct}, j, \text{st}_{\mathcal{S}})$ return pd_j

Fig. 17: Experiments for Definitions 22 and 23. Grey code only applies to $\mathbf{X} = \text{SelQ}$. $\text{Filter}_t(Q_{\text{Chal}})$ returns $\{(\mu, T) : (\mu, T) \in Q_{\text{Chal}} \wedge |T| \geq t\} \cup \{(\perp, T) : (\mu, T) \in Q_{\text{Chal}} \wedge |T| < t\}$. Boxed code is only for $\text{1StSimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^b(1^\lambda)$. Dash-boxed code is only for $\text{HkgSimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^b(1^\lambda)$.

We strengthen our standard SIM-CPA definition by disallowing the simulator from emulating KGen , which is used in Theorem 6.

Definition 22 (HonKgenSimCPA). Let $\mathbf{X} \in \{\text{AdpQ}, \text{SelQ}\}$. A TPKE scheme TPKE is simulation-secure with honest KGen , under selective corruption and $\mathbf{X}$ challenge queries ($\mathbf{X}$ -Hkg-SIM-CPA) if there is a PPT simulator $\mathcal{S}$, such that for any PPT adversary $\mathcal{A}$, the advantage

$$\text{Adv}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^{\text{SimCPA}}(\lambda) := |\Pr[\text{HkgSimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^0(1^\lambda) = 1] - \Pr[\text{HkgSimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^1(1^\lambda) = 1]|$$

is negligible, where $\text{HkgSimCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{S}, \mathbf{X}}^b$ is defined in Fig. 17 (dash-boxed code).

Recall from Theorem 5 that sometimes, a simulator does not require state to emulate *honest* encryptions and the associated partial decryption shares pd_j , but rather, the ciphertext ct , the message μ and the

$\text{KGen}^*(1^\lambda, \mathbb{A}_{k,t})$	$\text{ParDec}^*(\text{sk}_j^*, \text{ct})$
$(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \text{KGen}(1^\lambda, \mathbb{A}_{k,t})$	$(k_{\text{PRF},j}, \text{sk}_j) \leftarrow \text{sk}_j^*$
for $j \in [k]$:	$\text{rnd} \leftarrow \text{PRF}(k_{\text{PRF},j}, \text{ct})$
$k_{\text{PRF},j} \leftarrow \{0, 1\}^\lambda$	$\text{pd}_j \leftarrow \text{ParDec}(\text{sk}_j, \text{ct}; \text{rnd})$
$\text{sk}_j^* \leftarrow (\text{sk}_j, k_{\text{PRF},j})$	return pd_j
$\text{pk}^* \leftarrow \text{pk}$	
return $(\text{pk}^*, (\text{sk}_j^*)_{j \in [k]})$	

Fig. 18: Transforming TPKE into Derand[TPKE]; $\text{Enc}^* := \text{Enc}$ and $\text{Rec}^* := \text{Rec}$

encryption randomness rnd suffice. In this case, we say that the simulation is *partially stateless*, see formal definition below. Note that for partially stateless simulation, we additionally require that KGen is honestly simulated (although one could treat those two as orthogonal features of the simulation as well).

Definition 23 (StatelessSimCPA). Let $\mathbf{X} \in \{\text{AdpQ}, \text{SelQ}\}$. A TPKE scheme TPKE is simulation-secure with partially stateless simulation, under selective corruption and $\mathbf{X}$ challenge queries ($\mathbf{X}$ -1St-SIM-CPA) if there is a PPT simulator $\mathcal{S}$, such that for any PPT adversary $\mathcal{A}$, the advantage

$$\text{Adv}_{\text{TPKE}, \mathcal{A}, \mathbf{S}, \mathbf{X}}^{\text{1StSimCPA}}(\lambda) := |\Pr[\text{1StSimCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{S}, \mathbf{X}}^0(1^\lambda) = 1] - \Pr[\text{1StSimCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{S}, \mathbf{X}}^1(1^\lambda) = 1]|$$

is negligible, where $\text{1StSimCPA}_{\text{TPKE}, \mathcal{A}, \mathbf{S}, \mathbf{X}}^b$ is defined in Fig. 17 (boxed code).

C Derandomising Partial Decryption Oracle

ParDec can be derandomised for many of our security notions by embedding an additional PRF key into sk_j and deriving pseudo-randomness from ct , see Fig. 18 for details. In particular, the derandomisation also preserves stateless simulation (Definition 23).

Lemma 1 (Derandomisation preserves security). Let TPKE be a TPKE scheme, let $\mathbf{X} \in \{\text{SelQ}, \text{AdpQ}\}$, $\mathbf{Y} \in \{\text{SIM}, \text{IND}\}$, $\mathbf{Z} \in \{\text{CPA}, \text{CCA}\}$ and let PRF be an ℓ -PRF with sufficiently large ℓ . Let Derand[TPKE] be as in Fig. 18.

- (1) TPKE is $\mathbf{X}$ -Y-Z. $\Rightarrow$ Derand[TPKE] is $\mathbf{X}$ -Y-Z.
- (2) TPKE is 1St-X-SIM-Z. $\Rightarrow$ Derand[TPKE] is 1St-X-SIM-Z

Proof. For $\mathbf{X}$ -IND-CPA and $\mathbf{X}$ -IND-CCA, we first apply PRF security to replace the PRF for uncorrupted keys with a truly random function. Next, we reduce to security of TPKE by forwarding all queries, but we store all ParDecO queries and their answers, so that if a ParDecO query is repeated, the reduction responds with the stored response.

For $\mathbf{X}$ -SIM-CPA and $\mathbf{X}$ -SIM-CCA, the proof depends on whether the simulator for TPKE is stateful or not. If the simulator for TPKE is stateful, the simulator for Derand[TPKE] stores all query-answers to ParDecO and replays previous answers if a query repeats. The game-hops are then analogous to the IND setting. If the simulator $\mathcal{S}$ for TPKE is stateless for EncO as well as for ParDecO queries related to ciphertexts ct coming from EncO (as required by Definition 23), then we construct a simulator Derand[$\mathcal{S}$] analogously to Fig. 18, namely, for each $j \in [k]$, we include PRF keys $k_{\text{PRF},j}$ into $\text{init-st}_{\text{Derand}[\mathcal{S}]}$ and for answering ParDecO queries related to ciphertexts ct coming from EncO, Derand[$\mathcal{S}$] now generates pseudo-randomness from its input via $k_{\text{PRF},j}$ so that repeating queries lead to repeating answers. For answering ParDecO queries related to ciphertexts ct coming from ChalO, Derand[$\mathcal{S}$] uses the same (stateful) simulation strategy as before. The game-hops are, once more, analogous to the IND setting. $\square$

D Proofs for Section 3

D.1 Proof of Proposition 1

We describe the proof for X-IND-CPA under static corruption, those for CCA and adaptive corruption are analogous. Assume towards contradiction that there exists a successful PPT adversary $\mathcal{A}$ against (multi-challenge) X-IND-CPA. Let $s = \text{poly}(\lambda)$ be an upper bound on $\mathcal{A}$'s queries to ChalO , and assume w.l.o.g. that $\mathcal{A}$ never triggers an **assert** in ChalO . We construct a reduction $\mathcal{R}$ against single-challenge X-IND-CPA as follows. $\mathcal{R}$ draws a random $i \leftarrow_{\$} [\text{poly}(\lambda)]$ and initialises a counter ctr to 0. Next, $\mathcal{R}$ just runs $\mathcal{A}$ and forwards its queries, except that for $\mathcal{A}$'s queries to ChalO , $\mathcal{R}$ answers as follows:

```

ChalO( $\mu_0, \mu_1$ )
ctr  $\leftarrow$  ctr + 1
if ctr <  $i$  then
  (ct, rnd)  $\leftarrow$  EncO( $\mu_0$ ); return ct
if ctr =  $i$  then
  ct  $\leftarrow$  ChalO( $\mu_0, \mu_1$ ); return ct
if ctr >  $i$  then
  (ct, rnd)  $\leftarrow$  EncO( $\mu_1$ ); return ct

```

The probability analysis follows from a standard hybrid argument (cf. [BDF⁺18, Appendix B]). If $i = 1$ and $\mathcal{R}$ plays against the single-challenge version of $\text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^b(1^\lambda)$ with $b = 0$, $\mathcal{R}$ perfectly emulates $\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{X}}^0(1^\lambda)$ towards $\mathcal{A}$, thus

$$\Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^0(1^\lambda) \mid i = 1] = \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{X}}^0(1^\lambda)]. \quad (1)$$

If $i = \text{poly}(\lambda)$ and $\mathcal{R}$ plays against the single-challenge version of $\text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^b(1^\lambda)$ with $b = 1$, $\mathcal{R}$ perfectly emulates $\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{X}}^0(1^\lambda)$ towards $\mathcal{A}$, thus

$$\Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^1(1^\lambda) \mid i = \text{poly}(\lambda)] = \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{X}}^1(1^\lambda)]. \quad (2)$$

Finally, for any $i' \in [\text{poly}(\lambda) - 1]$, we have that if $i = i'$ and $\mathcal{R}$ plays against the single-challenge version of $\text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^b(1^\lambda)$ with $b = 1$, $\mathcal{A}$'s view is the same as when $i = i' + 1$ and $\mathcal{R}$ plays against the single-challenge version of $\text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^b(1^\lambda)$ with $b = 0$. Thus, for all $i' \in [\text{poly}(\lambda) - 1]$

$$\Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^1(1^\lambda) \mid i = i'] = \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^0(1^\lambda) \mid i = i' + 1]. \quad (3)$$

By a standard telescopic sum argument, we have

$$\begin{aligned}
& \left| \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{X}}^0(1^\lambda)] - \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{A}, \mathcal{X}}^1(1^\lambda)] \right| \\
& \stackrel{(1), (2)}{=} \left| \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^0(1^\lambda) \mid i = 1] - \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^1(1^\lambda) \mid i = \text{poly}(\lambda)] \right| \\
& \stackrel{(3)}{=} \left| \sum_{i' \in [\text{poly}(\lambda)]} \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^0(1^\lambda) \mid i = i'] - \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^1(1^\lambda) \mid i = i'] \right| \\
& = \text{poly}(\lambda) \cdot \left| \sum_{i' \in [\text{poly}(\lambda)]} \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^0(1^\lambda) \mid i = i'] \cdot \Pr[i = i'] \right. \\
& \quad \left. - \sum_{i' \in [\text{poly}(\lambda)]} \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^1(1^\lambda) \mid i = i'] \cdot \Pr[i = i'] \right| \\
& = \text{poly}(\lambda) \cdot \left| \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^0(1^\lambda)] - \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \mathcal{X}}^1(1^\lambda)] \right|.
\end{aligned}$$

We conclude, if $\mathcal{A}$ has non-negligible probability, so has $\mathcal{R}$ and we reached a contradiction. $\square$

D.2 Proof of Proposition 2

We describe the proofs for the CPA setting (Item (1)), those for CCA (Item (2)) are analogous.

Let $X \in \{\text{SelQ}, \text{AdpQ}\}$, let $\mathcal{A}$ be a PPT adversary against $\text{IndCPA}_{\text{TPKE}, \mathcal{A}, X}$, and assume w.l.o.g. that $\mathcal{A}$ never makes a query such that **assert** $P[\text{ct}] \cup \mathcal{C} \notin \mathbb{A}_{k,t}$ in ParDecO is triggered. Using $\mathcal{A}$, we define two adversaries $\mathcal{A}_0$ and $\mathcal{A}_1$, against $\text{SimCPA}_{\text{TPKE}, \mathcal{A}_b, S, X}$. $\mathcal{A}_0$ behaves as $\mathcal{A}$, except that when $\mathcal{A}$ queries $\text{EncO}(\mu_0, \mu_1)$, $\mathcal{A}_0$ queries $\text{ChalO}(\mu_0)$. Analogously, $\mathcal{A}_1$ queries $\text{ChalO}(\mu_1)$.

For the real SIM-CPA experiment and both $b \in \{0, 1\}$, we have

$$\Pr[1 = \text{SimCPA}_{\text{TPKE}, \mathcal{A}_b, S, X}^0(1^\lambda)] = \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{A}, X}^b(1^\lambda)].$$

Moreover, since the IND-CPA adversary $\mathcal{A}$ never causes $P[\text{ct}] \cup \mathcal{C} \in \mathbb{A}_{k,t}$ for $\text{ct} \in L_{\text{Chal}}$, the output from the ideal experiment $\text{SimCPA}_{\text{TPKE}, \mathcal{A}_b, S, X}^1(1^\lambda)$ does not depend on $b \in \{0, 1\}$. Hence, by triangle inequality,

$$\text{Adv}_{\text{TPKE}, \mathcal{A}, X}^{\text{IndCPA}}(\lambda) \leq \text{Adv}_{\text{TPKE}, \mathcal{A}_0, S, X}^{\text{SimCPA}}(\lambda) + \text{Adv}_{\text{TPKE}, \mathcal{A}_1, S, X}^{\text{SimCPA}}(\lambda),$$

which concludes the proof.

To see that the same reduction works for $X = \text{MaxCor}$, note that if $\mathcal{A}$ corrupts a maximal set, so do $\mathcal{A}_0$ and $\mathcal{A}_1$, and an identical analysis applies. $\square$

D.3 Proof of Theorem 1

We describe the proof for IND-CPA, and that for IND-CCA is analogous.

We will show that under adaptive corruption, if TPKE is single-challenge **AdpQ**-IND-CPA secure without Type II queries, then it is also single-challenge **AdpQ**-IND-CPA secure (with Type II queries). Then the proof is completed by invoking Proposition 1, which tells that TPKE is also multi-challenge **AdpQ**-IND-CPA secure (with Type II queries).

The only difference between the games with and without Type II queries is the oracle ParDecO , where the reduction must simulate the Type II queries $\text{ParDecO}(\text{ct}^*, j)$ on the (single) challenge ciphertext ct^* . To do so, the reduction does the following:

- Upon receiving a Type II query $\text{ParDec}(\text{sk}_j, \text{ct}^*)$ on an index j from the adversary $\mathcal{A}$, the reduction corrupts party j and learns the key share sk_j , then it runs $\text{ParDec}(\text{sk}_j, \text{ct}^*)$ honestly to obtain pd_j , and passes pd_j to $\mathcal{A}$.
- Upon receiving a corruption query $\text{CorO}(j)$ on an index j , the reduction corrupts party j and passes its key share sk_j to $\mathcal{A}$.

(All other queries from $\mathcal{A}$ can be passed through as is.) Recall that throughout the experiment, $\mathcal{A}$ may only query $\text{ParDecO}(\text{ct}^*, j)$ on some set T of indices j and corrupt some set $\mathcal{C}$, such that $T \cup \mathcal{C} \notin \mathbb{A}_{k,t}$, i.e. all partial decryption $(\text{pd}_j)_{j \in T}$ of ct^* and the set $\mathcal{C}$ of corrupt indices jointly do not allow for decryption. Meanwhile, the reduction may corrupt a set $\mathcal{C}'$ as long as $\mathcal{C}' \notin \mathbb{A}_{k,t}$. The above implies $\mathcal{C}' = T \cup \mathcal{C}$, so that all corruption queries $j \in \mathcal{C}'$ made by the reduction are allowed. Then, by correctness of TPKE, the reduction statistically simulates ParDecO to $\mathcal{A}$ (with a negligible reduction loss ϵ corresponding to the correctness error). $\square$

D.4 Proof of Theorem 2

First, we observe that for every TPKE scheme TPKE, there is a polynomial $\text{poly}(\lambda)$ such that

$$\Pr_{(\text{pk}, \text{sk}_1) \leftarrow \text{sKGen}(1^\lambda, \mathbb{A}_{1,1})} [|\text{sk}_1| > \text{poly}(\lambda)] = 0,$$

since KGen is a polynomial-time algorithm and its output length is bounded by its runtime. Let μ_0 and μ_1 be two arbitrary distinct messages in the message space. We construct a PPT adversary $\mathcal{A}$ distinguishing the X -SIM-CPA with adaptive corruption game for any TPKE for both $X \in \{\text{SelQ}, \text{AdpQ}\}$. In the easiest separating example, $\mathcal{A}$ chooses $t = k = 1$, makes no ParDecO queries, but rather just challenge queries and then corrupts the single party. This is a direct adaptation of Nielsen's separation [Nie02]. In order to show that the result also holds in an “actual” threshold setting, we construct a slightly more complicated $\mathcal{A}$ which chooses $k = 3$ and $t = 2$. Then after receiving pk , the adversary's main routine proceeds as follows:

$$\mathcal{A}(1^\lambda, \text{pk})$$

```

 $s \leftarrow \text{poly}(\lambda) + \lambda$ 
for  $j \in [s]$  :
     $b_j \leftarrow_{\$} \{0, 1\}$ ;  $\text{ct}_j \leftarrow \text{ChalO}(\mu_{b_j})$ 
     $\text{pd}_j \leftarrow_{\$} \text{ParDecO}(\text{ct}_j, 1)$ 
 $\text{sk}_2 \leftarrow \text{CorO}(2)$ 
if  $|\text{sk}_2| > \text{poly}(\lambda)$  then return 1
for  $j \in [s]$  :
     $\text{pd} \leftarrow_{\$} \text{ParDec}(\text{sk}_2, \text{ct}_j)$ 
     $\mu \leftarrow_{\$} \text{Rec}((\text{pd}, \text{pd}_j), \text{ct}_j)$ 
    if  $\mu \neq \mu_{b_j}$  then return 1
return 0

```

If $\mathcal{A}$ interacts with $\text{AdpCorSimCPA}_{\text{TPKE}, \mathcal{A}, S, \mathbf{x}}^0(1^\lambda)$, then $\mathcal{A}$ returns 0 with overwhelming probability by correctness of TPKE. Else, $\mathcal{A}$ interacts with $\text{AdpCorSimCPA}_{\text{TPKE}, \mathcal{A}, S, \mathbf{x}}^1(1^\lambda)$, then $\mathcal{A}$ returns 0 at most with probability $2^{-\lambda}$, since there are less than $2^{\text{poly}(\lambda)}$ many options for sk (which induce a value for each $\text{Rec}(\text{pd}, \text{ct}_j)$), but there are $2^{\text{poly}(\lambda) + \lambda}$ many options for $b_1, \dots, b_s$. Note that this analysis even applies if Rec or ParDec are randomised since their randomness is independent of sk_2 , and for each randomness string, the analysis applies—and since it applies to all randomness strings, it applies, in particular, when the randomness string is drawn uniformly at random. $\square$

$\text{KGen}^*(1^\lambda, \mathbb{A}_{k,t})$ <pre> $(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow_{\\$} \text{KGen}(1^\lambda, \mathbb{A}_{k,t})$ if $k < \lambda$ or $t > \frac{k}{2} + 1$ then return $((\text{pk}, \perp), (\text{sk}_j, \perp)_{j \in [k]})$ $k' \leftarrow \frac{k}{2}$; $t' \leftarrow \frac{k}{2}$ $(\text{pk}', (\text{sk}'_j)_{j \in [k']}) \leftarrow_{\\$} \text{KGen}(1^\lambda, \mathbb{A}_{k',t'})$ $\text{crs} \leftarrow_{\\$} \text{NIZK.Setup}(1^\lambda)$ $\text{pk}^* \leftarrow (\text{pk}, \text{pk}', \text{crs})$ $R \leftarrow_{\\$} \{S \subseteq [k] : S = \frac{k}{2}\}$ $\text{ctr} \leftarrow 0$ for $j \in R$: $\text{ctr} \leftarrow \text{ctr} + 1$; $\text{sk}_j^* \leftarrow (\text{sk}_j, \text{sk}'_{\text{ctr}})$ for $j \in [k] \setminus R$: $\text{sk}_j^* \leftarrow (\text{sk}_j, \perp)$ return $(\text{pk}^*, (\text{sk}_j^*)_{j \in [k]})$ </pre>	$\text{Enc}^*(\text{pk}^*, \mu)$ <pre> $(\text{pk}, \text{pk}', \text{crs}) \leftarrow \text{pk}^*$ $\text{rnd} \leftarrow_{\\$} \{0, 1\}^{\text{rlen}}$ $\text{rnd}' \leftarrow_{\\$} \{0, 1\}^{\text{rlen}}$ $\text{ct} \leftarrow_{\\$} \text{Enc}(\text{pk}, \mu; \text{rnd})$ if $\text{pk}' = \perp$ then $\text{ct}' \leftarrow \perp$ else $\text{ct}' \leftarrow_{\\$} \text{Enc}(\text{pk}', \mu; \text{rnd}')$ $\mathbb{x} \leftarrow ((\text{pk}, \text{pk}'), (\text{ct}, \text{ct}'))$ $\mathbb{w} \leftarrow (\mu, \text{rnd} \text{rnd}')$ $\pi \leftarrow_{\\$} \text{NIZK.Prove}(\mathbb{x}, \mathbb{w}, \text{crs})$ return (ct, ct') </pre>	$\text{ParDec}^*(\text{sk}^*, \text{ct}^*)$ <pre> $(\text{sk}, \text{sk}') \leftarrow \text{sk}^*$ $(\text{ct}, \text{ct}', \pi) \leftarrow \text{ct}^*$ $\mathbb{x} \leftarrow ((\text{pk}, \text{pk}'), (\text{ct}, \text{ct}'))$ $\text{assert NIZK.Verify}(\mathbb{x}, \pi, \text{crs}) = 1$ $\text{pd} \leftarrow_{\\$} \text{ParDec}(\text{sk}, \text{ct})$ if $\text{sk}' = \perp$ then $\text{pd}' \leftarrow \perp$ else $\text{pd}' \leftarrow_{\\$} \text{ParDec}(\text{sk}', \text{ct}')$ return (pd, pd') </pre> $\text{Rec}^*((\text{pd}_j^*)_{j \in T}, \text{ct}^*)$ <pre> $(\text{ct}, \text{ct}') \leftarrow \text{ct}^*$ for $j \in T$: $(\text{pd}_j, _) \leftarrow \text{pd}_j^*$ $\mu \leftarrow_{\\$} \text{Rec}((\text{pd}_j)_{j \in T}, \text{ct})$ return μ </pre>
--	--	---

Fig. 19: Without boxed code: $\text{TPKE}^* = (\text{KGen}^*, \text{Enc}^*, \text{ParDec}^*, \text{Rec}^*)$, which runs $\text{TPKE} = (\text{KGen}, \text{Enc}, \text{ParDec}, \text{Rec})$. With boxed code: $\text{TPKE}' = \text{SMT}[\text{TPKE}^*, \text{NIZK}]$.

D.5 Proofs of Theorems 3 and 4

We prove Theorems 3 and 4, jointly, subsuming both theorems in the following lemma statement. We include both $\text{TPKE}^* = (\text{KGen}^*, \text{Enc}^*, \text{ParDec}^*, \text{Rec}^*)$ and $\text{TPKE}' = \text{SMT}[\text{TPKE}^*, \text{NIZK}]$ in Fig. 19 to make this appendix self-contained.

Lemma 2. *If TPKE is SCor-AdpQ-SIM-CCA , then TPKE^**

- (1) *is $\text{MaxCor-AdpQ-SIM-CPA}$ and $\text{MaxCor-AdpQ-SemiMal-SIM-CPA}$;*
- (2) *is SCor-SelQ-SIM-CPA and $\text{SCor-SelQ-SemiMal-SIM-CPA}$;*
- (3) *but is not SCor-AdpQ-IND-CPA .*

If additionally, NIZK is SIMEXT , then $\text{TPKE}' = \text{SMT}[\text{TPKE}^, \text{NIZK}]$*

- (1) *is $\text{MaxCor-AdpQ-SIM-CCA}$;*
- (2) *is SCor-SelQ-SIM-CCA ;*
- (3) *but is not SCor-AdpQ-IND-CPA .*

Proof of Lemma 2. We first prove Items (3) of Lemma 2.

Claim 1. TPKE^* and $\text{TPKE}' = \text{SMT}[\text{TPKE}^*, \text{NIZK}]$ are not SCor-AdpQ-IND-CPA .

Proof. For both schemes, we consider a PPT adversary $\mathcal{A}$ against SCor-AdpQ-IND-CPA , which chooses $k \leftarrow 2\lambda$ and $t \leftarrow \lambda + 1$, and returns the threshold access structure $\mathbb{A}_{k,t}$. Next, on input pk^* , $\mathcal{A}$ returns an empty set of corrupt parties $\mathcal{C} \leftarrow \emptyset$. Let μ_0 and μ_1 be two arbitrary distinct messages in the message space $\mathcal{M}_\lambda$. The main routine of $\mathcal{A}$ is as follows¹⁹, where the boxed code only applies to $\text{TPKE}' = \text{SMT}[\text{TPKE}^*, \text{NIZK}]$:

```

 $\mathcal{A}((1^\lambda, \text{pk}^*), \emptyset)$ 
 $R \leftarrow \emptyset$ 
for  $j \in [k]$  : // Determine  $R$ 
     $\text{ct} \leftarrow \text{ChalO}(\mu_0, \mu_0)$ 
     $(\text{pd}_j, \text{pd}'_j) \leftarrow \text{ParDecO}(\text{ct}, j)$ 
    if  $\text{pd}'_j \neq \perp$  then  $R \leftarrow R \cup \{j\}$ 
 $\text{ct}^* \leftarrow \text{ChalO}(\mu_0, \mu_1)$  // Get a challenge
for  $j \in R$  : // Get partial decryptions
     $(\text{pd}_j, \text{pd}'_j) \leftarrow \text{ParDecO}(\text{ct}^*, j)$ 
     $(\text{ct}, \text{ct}', \boxed{\pi}) \leftarrow \text{ct}^*$ 
     $\mu \leftarrow \text{Rec}((\text{pd}_j)_{j \in R}, \text{ct})$  // Decrypt
    if  $\mu = \mu_0$  then return 0
return 1

```

To see that $\mathcal{A}$ returns the correct bit with probability $1 - \text{negl}(\lambda)$, observe the following: First, $k = 2\lambda$ satisfies $k \geq \lambda$ and $t = \lambda + 1$ satisfies $t \leq \frac{k}{2} + 1 = \lambda + 1$, therefore pk^* indeed consists of two public keys (pk, pk') (and a crs in the case of $\text{TPKE}' = \text{SMT}[\text{TPKE}^*, \text{NIZK}]$), and in the for-loop over $j \in [k]$, ChalO indeed returns ciphertexts ct^* 's which also consist of pairs of ciphertexts (ct, ct') (and a proof π in the case of $\text{TPKE}' = \text{SMT}[\text{TPKE}^*, \text{NIZK}]$). Then, by design of ParDec , each $\text{ParDecO}(\text{ct}, j)$ query from $\mathcal{A}$ returns $\text{pd}'_j \neq \perp$ if and only if $j \in R$, thus $\mathcal{A}$ determines R correctly. Finally, for the ciphertext ct^* which encrypts μ_b for the challenge bit b , since R is a set of size $\frac{k}{2} = \lambda$ lower than the threshold $t = \lambda + 1$, all $\text{ParDecO}(\text{ct}^*, j)$ queries for ct^* are accepted. By correctness of TPKE , $\text{Rec}((\text{pd}'_j)_{j \in R}, \text{ct})$ returns the correct value, unless a correctness error occurs whose probability is negligible. $\square$

Next, we prove Item (1) for TPKE^* , and we will see that Item (1) for TPKE' follows as a direct corollary, cf. Claim 3.

¹⁹ An alternative main routine where, in the first for-loop running over $i \in [k]$, $\mathcal{A}$ queries $\text{EncO}(\mu_0)$ instead of $\text{ChalO}(\mu_0, \mu_1)$ would work in the same way.

Claim 2. *If TPKE is SCor-AdpQ-SIM-CCA, then TPKE* is MaxCor-AdpQ-SIM-CPA and MaxCor-AdpQ-SemiMal-SIM-CPA.*

Proof. First, observe that MaxCor-AdpQ-SemiMal-SIM-CPA security implies MaxCor-AdpQ-SIM-CPA security and thus, it suffices to prove the former. Assume towards contradiction that $\mathcal{A}$ is a PPT adversary breaking MaxCor-AdpQ-SemiMal-SIM-CPA security of TPKE* by corrupting a maximal set $\mathcal{C}$ at the beginning of the game. To define the simulator $\mathcal{S}^*$ for TPKE*, recall that CCA-security implies semi-malicious CPA-security for the same flavour of security (cf. Remark 10). Let $\mathcal{S}$ and $\mathcal{S}'$ be these two simulators for the semi-malicious CPA-security of the first and second component, respectively. Now, $\mathcal{S}^*$ runs the simulators $\mathcal{S}$ and $\mathcal{S}'$ for simulating the first and second components respectively, drawing a random $\frac{k}{2}$ -out-of- k subset of parties at initialisation to simulate that secret keys corresponding to the second component are embedded in a random $\frac{k}{2}$ -size subset. In summary, $\mathcal{S}^*$ behaves as specified in Fig. 20.

<u>Initialisation</u>	<u>EncO (cf. Fig. 7b)</u>	<u>ParDecO(ct, j)</u>
$\mathcal{S}^*(1^\lambda, \mathbb{A}_{k,t}, \mathcal{C})$	$\mathcal{S}^*(\mu, \text{rnd} \parallel \text{rnd}', \text{st}_{\mathcal{S}^*})$	$\mathcal{S}^*(\text{ct}^*, \mu, j, \text{st}_{\mathcal{S}^*})$
$(\text{pk}, (\text{sk}_j)_{j \in \mathcal{C}}, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(1^\lambda, \mathbb{A}_{k,t}, \mathcal{C})$	$(\text{st}_{\mathcal{S}}, \text{st}_{\mathcal{S}'}, H, ..) \leftarrow \text{st}_{\mathcal{S}^*}$	// $\mu = \perp$ denotes empty μ .
if $k < \lambda$ or $t > \frac{k}{2} + 1$ then	$\text{st}_{\mathcal{S}} \leftarrow \mathcal{S}(\mu, \text{rnd}, \text{st}_{\mathcal{S}})$	$(\text{ct}, \text{ct}') \leftarrow \text{ct}^*$
$\text{pk}' \leftarrow \perp$; $(\text{sk}'_j)_{j \in [k]} \leftarrow (\perp)_{j \in [k]}$	$\text{st}'_{\mathcal{S}} \leftarrow \mathcal{S}'(\mu, \text{rnd}', \text{st}'_{\mathcal{S}})$	$(\text{st}_{\mathcal{S}}, \text{st}_{\mathcal{S}'}, H, T, U) \leftarrow \text{st}_{\mathcal{S}^*}$
$\text{st}_{\mathcal{S}'} \leftarrow \perp$	$H[\text{Enc}(\text{pk}', \mu; \text{rnd}')] \leftarrow 1$	$(\text{pd}, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\text{ct}, \mu, j, \text{st}_{\mathcal{S}})$
else $R \leftarrow \{S \subseteq [k] : S = \frac{k}{2}\}$	// Mark ct' as honest.	if $T[j] = \perp$: $\text{pd}'_j \leftarrow \perp$
$c \leftarrow R \cap \mathcal{C} $; assert $c < \frac{k}{2}$	$\text{st}_{\mathcal{S}^*} \leftarrow (\text{st}_{\mathcal{S}}, \text{st}_{\mathcal{S}'}, H, ..)$	// j associated with $\text{sk}' = \perp$.
$(\text{pk}', (\text{sk}'_j)_{j \in [c]}, \text{st}_{\mathcal{S}'}) \leftarrow \mathcal{S}'(1^\lambda, \mathbb{A}_{\frac{k}{2}, \frac{k}{2}}, [c])$	return $\text{st}_{\mathcal{S}^*}$	elseif $H[\text{ct}'] = 1$: $\mu \leftarrow \perp$
$\text{ctr} \leftarrow 0$; $H, T, U \leftarrow \text{EmptyTable}$	<u>ChalO</u>	// Empty μ for honest ct' .
for $j \in \mathcal{C} \cap R$:	$\mathcal{S}^*(\text{st}_{\mathcal{S}^*})$	$(\text{pd}', \text{st}_{\mathcal{S}'}) \leftarrow \mathcal{S}'(\text{ct}, \mu, T[j], \text{st}_{\mathcal{S}'})$
$\text{ctr} \leftarrow \text{ctr} + 1$; $T[j] \leftarrow \text{ctr}$	$(\text{st}_{\mathcal{S}}, \text{st}_{\mathcal{S}'}, ..) \leftarrow \text{st}_{\mathcal{S}^*}$	else $U[\text{ct}'] \leftarrow U[\text{ct}'] \cup \{j\}$
$\text{sk}_j^* \leftarrow (\text{sk}_j, \text{sk}'_{\text{ctr}})$	$(\text{ct}, \text{st}_{\mathcal{S}}) \leftarrow \mathcal{S}(\text{st}_{\mathcal{S}})$	if $ U[\text{ct}'] < \frac{k}{2}$: $\mu \leftarrow \perp$
for $j \in \mathcal{C} \setminus R$:	$(\text{ct}', \text{st}'_{\mathcal{S}}) \leftarrow \mathcal{S}'(\text{st}'_{\mathcal{S}})$	// Empty μ if below threshold.
$\text{sk}_j^* \leftarrow (\text{sk}_j, \perp)$	$\text{st}_{\mathcal{S}^*} \leftarrow (\text{st}_{\mathcal{S}}, \text{st}_{\mathcal{S}'}, ..)$	$(\text{pd}', \text{st}_{\mathcal{S}'}) \leftarrow \mathcal{S}'(\text{ct}, \mu, T[j], \text{st}_{\mathcal{S}'})$
$\text{st}_{\mathcal{S}^*} \leftarrow (\text{st}_{\mathcal{S}}, \text{st}_{\mathcal{S}'}, H, T, U)$	$\text{ct}^* \leftarrow (\text{ct}, \text{ct}')$	$\text{pd}_j^* \leftarrow (\text{pd}, \text{pd}')$
return $(\text{st}_{\mathcal{S}^*}, \text{pk}^*, (\text{sk}_j^*)_{j \in \mathcal{C}})$	return $(\text{st}_{\mathcal{S}^*}, \text{ct}^*)$	$\text{st}_{\mathcal{S}^*} \leftarrow (\text{st}_{\mathcal{S}}, \text{st}_{\mathcal{S}'}, H, T, U)$
		return $(\text{pd}_j^*, \text{st}_{\mathcal{S}^*})$

Fig. 20: Simulator for Claim 2.

To prove indistinguishability, we first make two game-hops: In the first hop, we replace all computations pertaining to the second public key pk' by the simulator $\mathcal{S}'$ guaranteed by the SCor-AdpQ-SemiMal-SIM-CPA security of TPKE. Here, the hybrid game also picks a random subset $R \subseteq [k]$ of size $\frac{k}{2}$ and associates the $\frac{k}{2}$ keys $(\text{pk}'_{\text{ctr}})_{\text{ctr} \in [\frac{k}{2}]}$ it gets from the simulator $\mathcal{S}'$ to $j \in R$, analogously to the behaviour of KGen of TPKE* and the initialisation simulation in $\mathcal{S}^*$ (cf. Fig. 20). The hybrid game later also performs mapping between j and ctr using a table T similarly to $\mathcal{S}^*$ for ParDecO queries; it also uses tables H and U to track when $\mathcal{S}'$ should receive μ , once more analogously to $\mathcal{S}^*$.

In the second game-hop, we replace all computations pertaining to the first public key pk by the simulator $\mathcal{S}$ also guaranteed by the SCor-AdpQ-SemiMal-SIM-CPA security of TPKE. Since we are in the maximal corruption setting, any non-trivial query to ParDecO (i.e., query for non-corrupted party) passes the threshold t and thus, the simulator $\mathcal{S}^*$ for TPKE* always gets the challenge message μ (as we consider CPA-security) and can input it to the simulators $\mathcal{S}$ and $\mathcal{S}'$ whenever they expect to receive it.

We summarise the reductions for the two game-hops next. The first game-hop is trivial if $\mathcal{A}$ chooses $k < \lambda$ or $t > \frac{k}{2} + 1$, since all second components are empty in this case. In case $\mathcal{A}$ chooses $k \geq \lambda$ and $t \leq \frac{k}{2} + 1$, the reduction $\mathcal{R}$ picks a random subset $R \subseteq [k]$ of size $\frac{k}{2}$ and associates the $\frac{k}{2}$ keys $(\text{pk}'_{\text{ctr}})_{\text{ctr} \in [\frac{k}{2}]}$ it gets to $j \in R$ analogously to the behaviour of KGen of TPKE^* . Additionally, the reduction emulates the behaviour of TPKE^* for the first components as described in Fig. 19. I.e., the reduction first runs $(\text{pk}, (\text{sk}_j)_{j \in [k]}) \leftarrow \text{KGen}(1^\lambda, \mathbb{A}_{k,t})$. To answer oracle calls, the reduction emulates behaviour pertaining to the first component honestly and forwards calls pertaining to the second oracle to its game, replacing j by $T[j]$ similarly to the simulator $\mathcal{S}^*$ (cf. Fig. 20). Moreover, the reduction maintains the lists $L_{\text{Enc}} \cup L_{\text{Chal}}$ to check whether **assert** $\text{ct}^* \in L_{\text{Enc}} \cup L_{\text{Chal}}$ in ParDecO queries. Note that if $\text{ct}^* \in L_{\text{Enc}} \cup L_{\text{Chal}}$ and $\text{ct}^* = (\text{ct}, \text{ct}')$, then the reduction's ParDecO query for ct' is also valid. Moreover, notice that when the reduction plays against the ideal game, μ is given to the simulator $\mathcal{S}'$ exactly when the hybrid game would do so using tables H and U , and thus, the reduction's emulation is perfect. $\mathcal{R}$ returns whatever $\mathcal{A}$ returns and has the same advantage.

Analogously, for the second game-hop, the reduction $\mathcal{R}$ forwards $\mathbb{A}_{k,t}$ and $\mathcal{C}$ from $\mathcal{A}$'s to its $\text{SCor-AdpQ-SemiMal-SIM-CPA}$ challenger. For all oracle queries of $\mathcal{A}$, it forwards the first components (pertaining to pk) to its own $\text{SCor-AdpQ-SemiMal-SIM-CPA}$ game, and forwards the second components (pertaining to pk') to the corresponding simulator $\mathcal{S}'$ which it runs locally using tables H, T, U to map j to ctr and remove μ as input to $\mathcal{S}'$ when necessary and maintaining lists $L_{\text{Enc}} \cup L_{\text{Chal}}$ to reject invalid ParDecO queries. As before, if the reduction deems $\text{ct}^* = (\text{ct}, \text{ct}')$ a valid query to ParDecO , then its own query for ct is valid, too, and the simulators receive μ consistently. Thus, the emulation of both games is perfect. The reduction returns whatever the adversary returns and has the same advantage. $\square$

Claim 3. *If TPKE is SCor-AdpQ-SIM-CCA and NIZK is SIMEXT , then $\text{TPKE}' = \text{SMT}[\text{TPKE}^*, \text{NIZK}]$ is $\text{MaxCor-AdpQ-SIM-CCA}$.*

Proof. Claim 2 implies that TPKE^* is $\text{SCor-AdpQ-SemiMal-SIM-CPA}$ and then, Theorem 7 shows that the transformed scheme $\text{TPKE}' = \text{SMT}[\text{TPKE}^*, \text{NIZK}]$ is SCor-AdpQ-SIM-CCA . $\square$

Next, we prove Item (2) for TPKE^* , and we will see that Item (2) for TPKE' follows as a direct corollary, cf. Claim 5.

Claim 4. *If TPKE is SCor-AdpQ-SIM-CCA , then TPKE^* is SCor-SelQ-SIM-CPA and $\text{SCor-SelQ-SemiMal-SIM-CPA}$.*

Proof. Observe that $\text{SCor-SelQ-SemiMal-SIM-CPA}$ implies SCor-SelQ-SIM-CPA and thus, it suffices to prove the former. Assume that $\mathcal{A}$ is a PPT adversary against $\text{SCor-SelQ-SemiMal-SIM-CPA}$ of TPKE^* . We execute the two analogous game-hops as in the proof of Claim 2, and also build the simulator analogously. The core difference is that, here we are no longer in the maximal corruption setting and we have to ensure that, when being queried on partial decryptions, the simulator $\mathcal{S}^*$ of TPKE^* gets the message μ as input whenever μ is expected by $\mathcal{S}, \mathcal{S}'$.

For any challenge ciphertext ct^* , to answer its partial decryption queries, the complicated case is when the threshold t for the first components is not met, but the threshold $t' = k/2$ for the second components is met. This is only possible if $t' = k/2 < t$ which implies that $t = k/2 + 1$, because the second component is only defined if $t \leq k/2 + 1$ (cf. lines 2–3 in KeyGen^* in Fig. 19). Only in this case the reduction does not learn the message, and thus cannot (trivially) simulate the responses for $\mathcal{A}$.

Concretely, let $\mathcal{I} = \mathcal{C} \cup P[\text{ct}]$. Our goal is to upper-bound the probability of the bad event that

- $|\mathcal{I}| < \frac{k}{2} + 1$, i.e., non-admissible w.r.t. pk , and
- $|\mathcal{I} \cap R| \geq t' = \frac{k}{2}$, i.e., admissible w.r.t. pk'

happen at the same time. The only possibility for the above is $R = \mathcal{I}$. In other words, $\mathcal{A}$ must succeed in guessing all of $R = \mathcal{I}$, a set of size $k/2$. Recall that $\mathcal{A}$ chooses its queries *selectively*, so that R is uniformly random and *independent* of $\mathcal{A}$'s queries. Clearly, the probability of success is $\binom{k}{k/2} \leq 2^{-\frac{k}{2}}$, negligible. By an additional union bound over the number of challenge oracle queries, we conclude that the probability of the bad event is still negligible. $\square$

Claim 5. *If TPKE is SCor-AdpQ-SIM-CCA and NIZK is SIMEXT, then $\text{TPKE}' = \text{SMT}[\text{TPKE}^*, \text{NIZK}]$ is SCor-SelQ-SIM-CCA.*

Proof. Claim 4 implies that TPKE^* is SCor-SelQ-SemiMal-SIM-CPA and then, Theorem 7 shows that the transformed scheme $\text{TPKE}' = \text{SMT}[\text{TPKE}^*, \text{NIZK}]$ is SCor-SelQ-SIM-CCA. $\square$

D.6 Proof of Proposition 4

We describe the proof for SelQ-IND-CPA, that for the CCA setting is analogous. By Proposition 1, it suffices to consider the single-challenge case. Assume towards a contradiction that there exists a PPT adversary $\mathcal{A}$ against SelQ-IND-CPA which only makes a single query $((\mu_0, \mu_1), T)$ to ChalO, and w.l.o.g. we assume that for this particular challenge query $|\mathcal{C} \cup T| \leq t - 1$. We construct a reduction $\mathcal{R}$ which breaks SelQ-IND-CPA under maximal corruption as follows. $\mathcal{R}$ runs $\mathcal{A}$ and returns the same access structure $\mathbb{A}_{k,t}$ and same state $\text{st}_{\mathcal{A}}$ as $\mathcal{A}$. Next, on input pk , $\mathcal{R}$ proceeds as follows:

```

 $\mathcal{R}(\text{st}_{\mathcal{A}}, \text{pk})$ 


---


 $(\mathcal{C}, Q_{\text{Enc}}, Q_{\text{Chal}}, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(\text{pk}, \text{st}_{\mathcal{A}})$ 
 $\{((\mu_0, \mu_1), T)\} \leftarrow Q_{\text{Chal}}$ 
 $\mathcal{C}' \leftarrow T \cup \mathcal{C}$ 
for  $i \in [k]$  :
  if  $|\mathcal{C}'| < t - 1$  then  $\mathcal{C}' \leftarrow \mathcal{C}' \cup \{i\}$ 
 $\text{st}_{\mathcal{R}} \leftarrow (\mathcal{C}', Q_{\text{Enc}}, Q_{\text{Chal}}, \text{st}_{\mathcal{A}})$ 
return  $(\mathcal{C}', Q_{\text{Enc}}, Q_{\text{Chal}}, \text{st}_{\mathcal{R}})$ 

```

Upon receiving the answers to all the queries, $\mathcal{R}$ replaces $(\text{sk}_i)_{i \in \mathcal{C}'}$ by $(\text{sk}_i)_{i \in \mathcal{C}}$, but else forwards all the answers. By assumption, $\mathcal{A}$ chooses $\mathcal{C}$ and T for the challenge query such that $|\mathcal{C} \cup T| \leq t - 1$, i.e. the set $\mathcal{C} \cup T$ does not exceed the maximum possible number of corruptions. Moreover, $|\mathcal{C}'| = t - 1$, since $\mathcal{R}$ adds indices to $\mathcal{C}'$ until it reaches size $t - 1$ and thus, $\mathcal{R}$ indeed corrupts a maximal set as required. Finally, since $T \subseteq \mathcal{C}'$, $\mathcal{R}$'s ChalO query does not trigger an assert, thus its emulation is perfect, and we have $\text{Adv}_{\text{TPKE}, \mathcal{R}, \text{SelQ}}^{\text{1Ch-IndCPA}}(\lambda) = \text{Adv}_{\text{TPKE}, \mathcal{A}, \text{SelQ}}^{\text{IndCPA}}(\lambda)$. We have reached a contradiction. $\square$

D.7 Proof of Proposition 5

The proof is analogous to that of Proposition 4, but since we are in the adaptive queries setting, instead of reading T from Q_{Chal} , this time, the reduction will guess the missing parties.

In more detail, we again use Proposition 1, since it allows us to consider the single-challenge case only. We then assume towards contradiction that $\mathcal{A}$ is an adversary against AdpQ-IND-CPA of TPKE under $t - c$ corruption, which indeed only makes a single query to ChalO. Let ct be the ciphertext returned from ChalO and let $T \subseteq [k]$ be the set defined by $\mathcal{A}$ making a ParDecO(ct, i) query for $i \in T$. Then, once again, we assume w.l.o.g. that $|\mathcal{C} \cup T| \leq t - 1$.

Now, similar to the proof of Proposition 4, $\mathcal{R}$ returns the same $\mathbb{A}_{k,t}$ as $\mathcal{A}$ and then runs $(\mathcal{C}, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(\text{pk}, \text{st}_{\mathcal{A}})$. By assumption, $t - 1 \geq t - |\mathcal{C}| \geq c$. Now, $\mathcal{R}$ guesses a random subset $\mathcal{I} \subseteq [k] \setminus \mathcal{C}$ such that $\mathcal{C}' = \mathcal{C} \cup \mathcal{I}$ is of size $t - 1$ and thus maximal. $\mathcal{R}$ can now forward all of $\mathcal{A}$'s queries to its own experiment except when $\mathcal{A}$ makes a ParDecO query for (i, ct) , where ct is the challenge ciphertext and $i \notin \mathcal{C}'$. Note that $\mathcal{R}$ cannot forward such a query since $\mathcal{C}'$ is already maximal and ParDecO would abort. Thus, in this case, $\mathcal{R}$ returns a random bit. Else $\mathcal{R}$ runs $\mathcal{A}$ to the end and returns whatever $\mathcal{A}$ returns.

First, if $T \subseteq \mathcal{C}'$, then $\mathcal{R}$ perfectly simulates $\text{IndCPA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^b(1^\lambda)$ and thus, we have for $b \in \{0, 1\}$ that

$$\Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{R}, \text{AdpQ}}^b(1^\lambda) \mid T \subseteq \mathcal{C}'] = \Pr[1 = \text{IndCPA}_{\text{TPKE}, \mathcal{A}, \text{AdpQ}}^b(1^\lambda)].$$

Next, the event $T \not\subseteq C'$ is equivalent to the event $(T \setminus C) \not\subseteq \mathcal{I}$, which happens with probability at least

$$p \geq \frac{1}{\binom{k}{c}} \geq k^{-c}, \quad (4)$$

which is polynomial in $k \in \text{poly}(\lambda)$. Thus, if $\mathcal{A}$ has advantage $\epsilon_{\mathcal{A}}$ against AdpQ-IND-CPA under $t - c$ corruption, then the reduction $\mathcal{R}$ has advantage at least $(1 - k^{-c}) \cdot 0 + k^{-c} \cdot \epsilon_{\mathcal{A}}$ against AdpQ-IND-CPA under maximal corruption. $\square$

Remark 13. One can further refine Eq. (4) to $p \geq \frac{1}{\binom{\ell}{c}}$, where $\ell = k - |\mathcal{C}| \geq k - (t - c)$. In particular, if t is close to or equal to k , then c does not need to be constant for p to be inverse polynomial.

D.8 Proof of Theorem 5

Claim 6. TPKE^* in Fig. 5 is single-challenge AdpQ-1St-SIM-CPA (Definition 23) and single-challenge AdpQ-Hkg-SIM-CPA (Definition 22).

Proof. Throughout this proof, we will use multiple times the fact that for all PPT adversaries, the probability of finding a message μ such that TPKE encryptions of μ with independent randomness collide with non-negligible probability is negligible, which is implied by (single-challenge) AdpQ-1St-SIM-CPA security since else, choosing $\mu_0 = \mu$ and a distinct μ_1 as challenge, then encrypting μ once again and comparing the result to the challenge ciphertext would be a successful distinguishing attack against AdpQ-1St-SIM-CPA security. By perfect correctness, also encryptions of distinct messages cannot collide.

We prove single-challenge AdpQ-Hkg-SIM-CPA security (simulator is not allowed to simulate key generation) of TPKE^* and AdpQ-1St-SIM-CPA security then also follows by observing that our simulator $\mathcal{S}^*$ is actually low-state. To construct $\mathcal{S}^*$ for TPKE^* , we build on the *low-state* simulator $\mathcal{S}$ (cf. Fig. 17) for TPKE . Since we assume that partial decryption computations in TPKE are derandomised, we can assume that partial decryption queries by $\mathcal{S}$ are derandomised as well. In particular, the stateless simulation branch for ParDec of $\mathcal{S}$ is a deterministic algorithm of polynomial size. Recall that we defined TPKE^* *dependent* on the circuit size of the ParDec branch of $\mathcal{S}$ in the sense that we choose the size of the (committed) circuit C_j in ParDec^* as a *fixed size* which is sufficiently large to encompass the original partial decryption circuit of TPKE and C_j in Fig. 21. Given this setup, we construct $\mathcal{S}^*$ from $\mathcal{S}$ as follows:

- In the setup phase of the AdpQ-Hkg-SIM-CPA game for TPKE^* , $\mathcal{S}^*$ receives $\text{pk}^* = (\text{pk}, \text{ck}, \text{crs})$, $\mathcal{C}$ and $(\text{sk}_j^*)_{j \in \mathcal{C}}$ where $\text{sk}_j^* = (\text{sk}_j, \tau_j^*, \rho_j^*, k_{\text{PRF},j}^*)$ from the game. It then hands pk^* , $\mathcal{C}$, $(\text{sk}_j^*)_{j \in \mathcal{C}}$ to $\mathcal{S}$ to get the initial state $\text{st}_{\mathcal{S},0}$ which $\mathcal{S}^*$ stores. Additionally, $\mathcal{S}^*$ stores all of its own inputs as well as several freshly sampled values: It stores a fresh PRF key $k_{\text{PRF,Enc}}^* \leftarrow \{0, 1\}^\lambda$. For every $j \in [k]$, the simulator $\mathcal{S}^*$ samples τ_j^* , ρ_j^* and $k_{\text{PRF},j}^*$ randomly. Alternatively, for $j \in \mathcal{C}$, the simulator $\mathcal{S}^*$ could also use τ_j^* and ρ_j^* provided by the game. The distributions are identical with no trapdoor information depending on them. In other words, $\mathcal{S}^*$ can “simulate” several parts of the secret key. For $j \in [k] \setminus \mathcal{C}$, $\mathcal{S}^*$ sets the (partial) secret key to $\text{sk}_j^* \leftarrow (\perp, \tau_j^*, \rho_j^*, k_{\text{PRF},j}^*)$. The initial state of $\mathcal{S}^*$ is then

$$\text{init-st}_{\mathcal{S}^*} = (\text{pk}^*, k_{\text{PRF,Enc}}^*, (\text{sk}_j^*)_{j \in [k]}).$$

We note that

- $k_{\text{PRF,Enc}}^* \leftarrow \{0, 1\}^\lambda$ serves as an encryption key (see EncO implementation below).
- ρ_j^* serves as fixed commitment randomness for cm_j .
- τ_j^* serves as a (committed) one-time trapdoor for the b_2 -branch in the NIZK relation R_{NIZK}^* .
- $k_{\text{PRF},k}^*$ serves to derandomise τ_j (so that the same one-time trapdoor τ_j^* can be used for repeated queries to ParDec with same challenge ciphertext as input).
- The response to EncO^* is given in Fig. 21. Observe $\mathcal{S}^*$ uses the random padding pad in ct^* to encrypt the message and randomness for later use. This is done to satisfy statelessness in ParDec^* for simulation.
- To handle challenge queries $\mathcal{S}^*$ acts as follows:

- To respond to the (single) ChalO^* query, $\mathcal{S}^*$ simply runs $\mathcal{S}$ with its initial state since the stateful part of $\mathcal{S}$ has not been invoked before. $\mathcal{S}$ outputs a tuple $(\text{ct}, \text{st}_{\mathcal{S}})$, and $\mathcal{S}^*$ returns $\text{ct}^* = (\text{ct}, \text{pad})$ for a truly random padding pad , and the updated state.
- To respond to $\text{ParDecO}^*(j, \text{ct}^*)$ for $\text{ct}^* \in L_{\text{Chal}} \setminus L_{\text{Enc}}$ (which is at most a single possible ciphertext) $\mathcal{S}^*$ simply runs $\mathcal{S}$ (with the current state $\text{st}_{\mathcal{S}}$) which outputs $(\text{pd}_j, \text{st}_{\mathcal{S}})$. Then $\mathcal{S}^*$ uses the witness τ_j^* for the OR-branch in R^* to generate π_j , re-generates cm_j and sets $\text{pd}_j^* = (\text{pd}_j, \tau_j^*, \text{cm}_j, \pi_j)$. It outputs pd_j^* and the updated state.

<u>$\text{Enc}(\mu)$</u>	<u>$\text{ParDec for } \text{ct}^* \in L_{\text{Enc}}$</u>
<u>$\mathcal{S}^*(\mu, \text{init-st}_{\mathcal{S}^*})$</u>	<u>$\mathcal{S}^*(\text{pk}^*, \text{ct}^*, j, \text{init-st}_{\mathcal{S}^*})$</u>
$(\text{init-st}_{\mathcal{S}}, k_{\text{PRF,Enc}}^*, _) \leftarrow \text{init-st}_{\mathcal{S}^*}$	$(\text{init-st}_{\mathcal{S},0}, k_{\text{PRF,Enc}}^*, _) \leftarrow \text{init-st}_{\mathcal{S}^*}$
$(\text{ct}, \text{rnd}) \leftarrow_{\$} \mathcal{S}(\mu, \text{init-st}_{\mathcal{S}})$	$\text{pd}_j \leftarrow C[k_{\text{PRF,Enc}}^*, \text{init-st}_{\mathcal{S}}](\text{ct}^*, j)$
// Encrypt (μ, rnd) for stateless use	$(\text{ck}, \tau_j^*, \rho_j^*, k_{\text{PRF,j}}^*) \leftarrow \text{init-st}_{\mathcal{S}^*}$
$\text{pad} = (\mu, \text{rnd}) \oplus \text{PRF}(k_{\text{PRF,Enc}}^*, \text{ct})$	$C_j(\cdot) \leftarrow C[k_{\text{PRF,Enc}}^*, \text{init-st}_{\mathcal{S}}](\cdot)$
$\text{ct}^* = (\text{ct}, \text{pad}); \text{rnd}^* = (\text{rnd}, \text{pad})$	$\text{cm}_j \leftarrow \text{Com}(\text{ck}, (C_j, \tau_j^*); \text{rnd}_j)$
return $(\text{ct}^*, \text{rnd}^*)$	$\text{pd}_j \leftarrow C_j(\text{ct})$
<u>$C[k_{\text{PRF,Enc}}^*, \text{init-st}_{\mathcal{S}}](\text{ct}^*, j)$</u>	$\tau_j = \text{PRF}(k_{\text{PRF,j}}^*, \text{ct}^*)$ // No trapdoor
$(\text{ct}, \text{pad}) \leftarrow \text{ct}^*$	$\mathbb{x} \leftarrow (\text{ck}, \text{ct}, \text{pd}_j, \tau_j^*, \text{cm}_j)$
$(\mu, \text{rnd}) \leftarrow \text{pad} \oplus \text{PRF}(k_{\text{PRF,Enc}}^*, \text{ct})$	$\mathbb{w} \leftarrow (C_j(\cdot), \tau_j^*, \rho_j^*)$
$\text{pd}_j \leftarrow \mathcal{S}(\text{ct}, \mu, \text{rnd}, \text{init-st}_{\mathcal{S}})$	$\pi_j \leftarrow_{\$} \text{NIZK.Prove}^{\text{RO}}(\mathbb{x}, \mathbb{w})$
return pd_j	$\text{pd}_j^* \leftarrow (\text{pd}_j, \tau_j^*, \text{cm}_j, \pi_j)$
	return pd_j^*

Fig. 21: Subroutines of $\mathcal{S}^*$ for $\text{Enc}(\mu)$ and ParDec for $\text{ct}^* \in L_{\text{Enc}}$ in Theorem 5. For conciseness of notation, we use $(x, _) \leftarrow \text{st}_{\mathcal{S}^*}$ and $(y, _) \leftarrow \text{st}_{\mathcal{S}^*}$ to denote that $\text{st}_{\mathcal{S}^*}$ retrieves x or y from its state, respectively, even if x and y cannot both be the first entry in $\text{st}_{\mathcal{S}^*}$.

The proof for single-challenge adaptive SIM-CPA security of TPKE^* proceeds via a sequence of game-hops starting from the real game for single-challenge adaptive SIM-CPA until reaching the ideal game with the simulator $\mathcal{S}^*$ defined above.

Game 0: This is the real game with TPKE^* .

Game 1: In this game, we simulate all NIZK proofs. By the zero-knowledge of NIZK (implied by SIMEXT), this change is indistinguishable.

Game 2: We make a hybrid argument over all ParDecO^* queries w.r.t. the challenge ciphertext ct^* , where $j \in [k] \setminus \mathcal{C}$. Namely, for these queries, we replace $\tau_j = \text{PRF}(k_{\text{PRF,j}}^*, \text{ct}^*)$ by $\tau_j = \tau_j^*$ in pd_j^* . This reduces to the hiding property of the commitment scheme and the security of the PRF for each $j \in [k] \setminus \mathcal{C}$ for which the adversary makes a ParDecO^* query. This game-hop uses that ct^* does not collide with any of the other ciphertexts with overwhelming probability, which follows from the same property in the TPKE component in ct^* . Moreover, we use that ParDec^* is derandomised (except for π), and thus repeated queries with the same ct^* result in the same τ_j . Under these constraints, a hybrid can first draw τ_j uniformly at random for the challenge query by the PRF property. The second hybrid then equates τ_j and τ_j^* , which uses the hiding property of the commitment.

Note that now for every ParDecO^* query with the (single) challenge ct^* , we have the witness $\tau_j^* = \tau_j$ for the b_2 -branch available.

Game 3: In EncO^* queries, we replace uniformly random pad by $\text{pad} = (\mu, \text{rnd}) \oplus \text{PRF}(k_{\text{PRF,Enc}}^*, \text{ct})$. Since with overwhelming probability, ct does not repeat, this change reduces to PRF security.

Game 4: We run $\mathcal{S}^*$ as defined, except that we still simulate all NIZK proofs. By single-challenge SIM-CPA for TPKE with simulator $\mathcal{S}$, this change in oracles is indistinguishable. Note that the current game (and $\mathcal{S}^*$) maps challenges to challenges and non-challenges to non-challenges, so the oracle behaviours align as required.

The only difference between Game 4 and the simulation for TPKE* is that NIZK proofs are still simulated and the commitments to C_j do not contain the code for ParDec^* of $\mathcal{S}$. We change this below.

Game 5: We replace the commitments to C_j by commitments to the simulator code C_j as in Fig. 21. This reduces to the hiding property of the commitment scheme.

Game 6: We use the witnesses to generate the proofs in ParDecO^* . Due to the changes in Games 2 and 5, we always have a witness available. Crucially, there is only a single challenge ciphertext, and derandomisation $\tau_j = \text{PRF}(k_{\text{PRF},j}^*, \text{ct}^*)$ ensures that we can reuse τ_j^* for every repeated challenge query to ParDecO^* once (per $j \in [k]$). By the zero-knowledge of NIZK (implied by SIMEXT), this change is indistinguishable. $\square$

Claim 7. TPKE* in Fig. 5 is not (multi-challenge) X-SIM-CPA.

Proof. We first give a conceptual overview. The adversary has two phases.

In the first phase, the adversary $\mathcal{A}$ makes an $\text{EncO}(\mu_0)$ query for an arbitrary message μ_0 . For the resulting ciphertext ct_0^* , $\mathcal{A}$ obtains ParDec decryptions from every party. This first phase is useful since once the adversary obtains a pd_j^* from a party, the commitment cm_j in pd_j^* becomes fixed. The real game never changes the output of cm_j , so the simulator never outputs anything else either except with negligible probability (as this leads to a trivial distinguishing attack). By the (straightline extractable) binding property of the commitment scheme, the circuit C_j and trapdoor τ_j^* in cm_j are now fixed.

In the second phase, the adversary samples two messages μ_1 and μ_2 from $\mathcal{M}$ and queries ChalO for both of them, obtaining ct_1^* and ct_2^* , respectively. By the soundness of the proof system, this means that for all $j \in [k]$ and for $i \in \{1, 2\}$, all $\text{ParDec}(\text{ct}_i^*, j)$ queries either need to return $C_j(\text{ct}_i^*)$ or we must have $\tau_j^* = \tau_j$. The latter can only be done once, since τ_j^* don't repeat in the real scheme (except with negligible probability) and repeating a τ_j^* would reveal that this is a simulation. Hence, for at least one of $i \in \{1, 2\}$, we need to have $\text{pd}_{j,i} = C_j(\text{ct}_i^*)$. Now, when asking for partial decryptions of the first $t - 1$ many parties, their contributions $\text{pd}_{j,i}$ are chosen independently of the messages μ_1 and μ_2 since the simulator does not know them. In turn, for the t -th party, $\text{pd}_{t,i}$ can depend on μ_1 and μ_2 , but by our previous argument, only *one* of $\text{ParDec}(\text{ct}_i^*, t)$ queries can return something different from $\text{pd}_{j,i} = C_j(\text{ct}_i^*)$. Thus, for either ct_1^* or ct_2^* , all $\text{pd}_{t,i}$ are chosen independently from μ_i , and Rec^* returns the correct message μ_i on them with probability $\frac{1}{|\mathcal{M}|} \geq \frac{1}{2}$. The probability that on the default $\text{pd}_{t,i}$, Rec^* returns the wrong message on *both* ct_0^* and ct_1^* is thus at least $\frac{1}{4}$ and thus, the simulation fails with noticeable probability, namely $\frac{1}{4} - \text{negl}(\lambda)$. We now describe the adversary formally and provide a reduction to (knowledge) soundness (which is implied by SIMEXT).

Adversary. The adversary chooses $\mathbb{A}_{k,t}$ and $t < k$ which TPKE* admits. (Any choice works, say $k \leftarrow \lambda$ and $t \leftarrow \text{[k]}$.) The adversary does not corrupt any party. The main stage of the adversary is as follows:

```

 $\mathcal{A}((1^\lambda, \text{pk}^*), \emptyset)$ 
 $\mu_0 \leftarrow \$ \mathcal{M}; \text{ct}_0^* \leftarrow \text{EncO}(\mu_0)$ 
for  $j \in [k]$  : // Phase I: Fix  $\text{cm}_j, \text{cm}'_j$ 
   $\text{pd}_{j,0}^* \leftarrow \text{ParDecO}(\text{ct}_0^*, j)$ 
   $(\text{pd}_{j,0}, \tau_{j,0}, \text{cm}_{j,0}, \pi_{j,0}) \leftarrow \text{pd}_{j,0}^*$ 
for  $i \in \{1, 2\}$  : // Phase II: Get inconsistent  $\text{pd}_{t,i}$ 
   $\mu_i \leftarrow \$ \mathcal{M}; \text{ct}_i^* \leftarrow \text{ChalO}(\mu_i)$ 
  for  $1 \leq j \leq t$  :
     $\text{pd}_{j,i}^* \leftarrow \text{ParDecO}(\text{ct}_i^*, j)$ 
     $(\text{pd}_{j,i}, \tau_{j,i}, \text{cm}_{j,i}, \pi_{j,i}) \leftarrow \text{pd}_{j,i}^*$ 
     $\mathbb{X}_{j,i} \leftarrow (\text{ck}, \text{ct}, \text{pd}_{j,i}, \tau_{j,i}, \text{cm}_{j,i})$ 
    if  $\text{NIZK.Verify}(\text{crs}, \mathbb{X}_{j,i}, \pi_{j,i}) = 0$  or  $\text{cm}_{j,i} \neq \text{cm}_{j,0}$  then return 1
  if  $\mu_i \neq \text{Rec}^*(\text{pk}^*, (\text{pd}_{j,i})_{j \in [t]}, \text{ct}_i^*)$  then return 1
if  $\tau_{t,0} = \tau_{t,1}$  then return 1
return 0

```

In the real game, $\mathcal{A}$ returns 0 except if $s_{t,0} = s_{t,1}$ occurs, which has negligible probability (by security of PRF). In the ideal game, we argue that $\mathcal{A}$ returns 1 with probability at least $\frac{1}{4} - \text{negl}(\lambda)$. For this, observe that by the previous proof sketch, it is clear that $\mathcal{A}$ succeeds with probability $\frac{1}{4}$ *unless* it breaks the binding property of the commitment or the (knowledge) soundness of the proof system. Because if this bad event does not occur, then for one of the two messages, the response $\text{pd}_{j,i}$ coincides with the output of the committed circuit C_j , which is independent of the chosen message m_i (since ct is independent of m_i due to simulation). Clearly, whenever pd_j is inconsistent with the committed C_j , this constitutes either a break of the (straightline extractable) binding property of COM or the (knowledge) soundness property (implied by SIMEXT) of NIZK, and the probability that this occurs is negligible. $\square$

D.9 Proof of Theorem 6

For $\mathbf{C} \in \{\text{SCor}, \text{AdpCor}\}$ and $\mathbf{Y} \in \{\text{IND}, \text{SIM}\}$, that TPKE^* is $\mathbf{C}\text{-AdpQ-Y-CPA}$ against adversaries that make no EncO queries is proven via two game-hops: First, we reduce to IND-CPA security of PKE and replace the ciphertexts ct_{PKE} generated by ParDec by encryptions of zeroes. Since the adversary does not get access to the randomness used to generate pk_{PKE} , this reduction is straightforward. In a second game-hop, we can directly reduce to $\mathbf{C}\text{-AdpQ-Y-CPA}$ -security of TPKE.

To see that TPKE^* is not SCor-SelQ-IND-CPA , consider an adversary which chooses $k = t = 1$, corrupts no one, sets $Q_{\text{Enc}} \leftarrow \{(\mu_0, \{1\})\}$ and $Q_{\text{Chal}} \leftarrow \{(\mu_0, \mu_1), \emptyset\}$ for two distinct messages $\mu_0 \neq \mu_1$. On input the answers $(\text{ct}_{\text{Enc}}, \text{rnd}_{\text{Enc}}, \text{pd}_{\text{Enc}})$ from EncO and ParDecO and ct_{Chal} from ChalO , $\mathcal{A}$ proceeds as follows:

```

 $\mathcal{A}((\text{ct}_{\text{Enc}}, \text{rnd}_{\text{Enc}}, \text{pd}_{\text{Enc}}), \text{ct}_{\text{Chal}}, \text{pk})$ 
 $(\text{rnd}_0, \text{rnd}_1) \leftarrow \text{rnd}_{\text{Enc}}$  // Split randomness for  $\text{KGen}_{\text{PKE}}$ 
 $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{KGen}_{\text{PKE}}(1^\lambda; \text{rnd}_1)$  // Re-compute  $\text{KGen}_{\text{PKE}}$ 
 $(\_, \text{ct}_{\text{PKE}}) \leftarrow \text{pd}_{\text{Enc}}$  // Get PKE ciphertext from  $\text{pd}_{\text{Enc}}$ 
 $\text{sk} \leftarrow \text{Dec}_{\text{PKE}}(\text{sk}, \text{ct}_{\text{PKE}})$ 
 $(\text{ct}'_{\text{Chal}}, \_) \leftarrow \text{ct}_{\text{Chal}}$ 
 $\text{pd} \leftarrow \$ \text{ParDec}(\text{sk}, \text{ct}'_{\text{Chal}})$ 
 $\mu \leftarrow \$ \text{Rec}(\text{pd}, \text{ct}_{\text{Chal}})$ 
if  $\mu = \mu_0$  then return 0
return 1

```

By correctness of the PKE and the TPKE TPKE , $\Pr[b = \text{IndCPA}_{\text{TPKE}, \mathcal{A}, \text{Seq}}^b(1^\lambda)] = 1 - \text{negl}(\lambda)$ for both $b \in \{0, 1\}$ and thus, $\mathcal{A}$ is indeed a successful distinguisher. $\square$

E Proofs for Section 4

Proof of Theorem 7. We describe the case for static corruption, that for adaptive corruption is analogous and so is the case of maximal static corruptions (in either case, the corruption queries are directly forwarded to the SIM-CPA simulator or IND-CPA challenger).

We first describe the proof for **X-SemiMal-SIM-CPA** to **X-SIM-CCA**, then explain how to adapt it to the IND case. We denote by EncO' , ChalO' , $\text{ParDecO}'$ the oracles of the TPKE' security game. For simplicity we consider PPT adversaries $\mathcal{A}'$ against TPKE' that do not make EncO' queries, which is without loss of generality (see Remark 1).

Let $\mathcal{S}$ be the simulator for **X-SemiMal-SIM-CPA** security of TPKE , and $\text{Sim} = (\text{Setup}, \text{Sim}, \text{Ext})$ the simulator for **SIMEXT** of **NIZK**. The simulator $\mathcal{S}'$ for **X-SIM-CCA** security of TPKE' runs KGen to get $(\text{pk}, (\text{sk}_j)_{j \in [k]})$, runs $(\text{crs}, \text{st}_{\text{Sim}}) \leftarrow \text{Sim.Setup}(1^\lambda)$ to get a trapdoored crs , and sets $\text{pk}' \leftarrow (\text{pk}, \text{crs})$. It returns $(\text{sk}_j)_{j \in \mathcal{C}}$ for the corrupted set $\mathcal{C}$. Next, $\mathcal{S}'$ runs

$$\text{st}_{\mathcal{S}} \leftarrow \mathcal{S}((\text{sk}_j)_{j \in \mathcal{C}}, \text{pk}, Q_{\text{Enc}}, \text{Filter}_t(Q_{\text{Chal}}); \text{st}_{\mathcal{S}}).$$

To answer ChalO' queries, $\mathcal{S}'$ simply runs $\mathcal{S}$ to obtain a ciphertext ct , and attaches a simulated proof $\pi \leftarrow \text{Sim.Sim}(\text{x} = (\text{pk}, \text{ct}), \text{st}_{\text{Sim}})$ to ct .

Upon receiving a $\text{ParDecO}'$ query $\text{ct}' = (\text{ct}, \pi)$, if it is generated by ChalO' , then $\mathcal{S}'$ runs $\mathcal{S}$ on ct and returns the result. Naturally, the interesting part is when ct' is not generated by ChalO' , but by the adversary. In this case, $\mathcal{S}$ checks that π is a valid proof that there exists μ, rnd such that $\text{ct} = \text{Enc}(\text{pk}, \mu; \text{rnd})$, extracts $(\mu, \text{rnd}) \leftarrow \text{Sim.Ext}((\text{pk}, \text{ct}), \pi, \text{crs})$, and returns

$$\text{pd}_j \leftarrow \mathcal{S}(\text{ct}, \mu, \text{rnd}, \text{st}_{\mathcal{S}}) \quad (5)$$

where the above corresponds to the simulator for EncO of TPKE . Looking ahead, Eq. (5) is the code which ParDecO of the TPKE game would run if (ct, rnd) is the result of an EncO' query.

The proof that the simulation is indistinguishable proceeds via 3 game-hops. We first reduce to **SIMEXT**, replace crs by a simulated crs , proofs by simulated proofs and verification by extraction (cf. Verify_1 in Fig. 9) which is indistinguishable since **NIZK** is **SIMEXT**. The next syntactical game-hop additionally uses the extracted values (μ, rnd) in $\text{ParDecO}'(\text{ct}', j)$ to execute the code of an EncO query for TPKE whenever ct' is not in L_{Chal} . This game-hop is syntactical, since L_{Enc} has no influence on the behaviour of the game for $b = 0$. Finally, we can reduce to **X-SemiMal-SIM-CPA** with the natural reduction.

To adapt the above proof for the **X-SemiMal-IND-CPA** to **X-IND-CCA** reduction, we proceed via the same game hops: we first replace the crs , simulate proofs and replace verification by extraction, then add the step to $\text{ParDecO}'(\text{ct}', j)$ where we run EncO code on the extracted (μ, rnd) . Afterwards, we can directly reduce to **X-SemiMal-IND-CPA** to swap encryptions of μ_0 to encryptions of μ_1 . $\square$

F Proofs for Section 5

F.1 Proof of Theorem 9

Proof. We provide the NIPoR simulator Sim in Fig. 22. It is evident that Sim.Ext is a tag-only extractor. For convenience, we assume w.l.o.g., that the adversary $\mathcal{A}$ queries $\text{RO}(\text{fk}, \text{tag})$ before querying $\text{VerifyO}(f_{\text{fk}}, y, \text{tag}, \pi)$. Next, we gradually transform the real game $\text{SIMEXT}_{\text{NIPoR}, \mathcal{A}, \text{Sim}}^0$ into the ideal game $\text{SIMEXT}_{\text{NIPoR}, \mathcal{A}, \text{Sim}}^1$ via several game-hops, depicted in Fig. 23.

Game 0 is the real game $\text{SIMEXT}_{\text{NIPoR}, \mathcal{A}, \text{Sim}}^0$ with the NIPoR construction (cf. Fig. 10) inlined. The adversary gets as input crs and access to oracles ProveO , VerifyO and RO , where $\text{crs} \leftarrow \text{NIPoR.Setup}^{\text{RO}}(1^\lambda)$ and the oracles honestly implement NIPoR.Prove , NIPoR.Verify , and a random oracle.

Game 1: We modify the crs and the ProveO and VerifyO oracles:

Sim.Setup(1^λ)	Sim.RO(fk, tag)	Sim.Sim(fk, y)	Sim.Ext($f_{fk}, y, \text{tag}, \pi$)
$(\text{ck}, \text{td}_{\text{Com}}) \leftarrow \text{COM.TSetup}(1^\lambda)$ $(\text{crs}_{\text{NIZK}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_{\text{NIZK}}.\text{Setup}(1^\lambda)$ $\text{crs} := (\text{ck}, \text{crs}_{\text{NIZK}})$ $\text{st}_{\text{Sim}} := (\text{td}_{\text{Com}}, \text{td}_{\text{Sim}})$ return $(\text{crs}, \text{st}_{\text{Sim}})$	if $\text{RO}[\text{fk}, \text{tag}] = \perp$ then $(r', m) \leftarrow \text{COM.Ext}(\text{td}, \text{tag})$ if $r' \neq \perp$ $r \leftarrow \text{TrueRand}(\text{fk}, \text{tag}, m)$ $\text{RO}[\text{fk}, \text{tag}] \leftarrow r - r'$ else $\text{RO}[\text{fk}, \text{tag}] \leftarrow \mathcal{R}$ return $\text{RO}[\text{fk}, \text{tag}]$	$\text{tag} \leftarrow \text{COM.Com}(\text{ck}, 0)$ assert $\text{RO}[\text{fk}, \text{tag}] = \perp$ $r'' \leftarrow \mathcal{R}$ $\text{RO}[\text{fk}, \text{tag}] \leftarrow r''$ $\pi \leftarrow \text{Sim}_{\text{NIZK}}.\text{Sim}(\text{fk}, y, \text{tag}, r'')$ return (tag, π)	$r'' \leftarrow \text{RO}[\text{fk}, \text{tag}]$ $\mathbb{x}^* \leftarrow (\text{fk}, y, \text{tag}, r'')$ $(r', m) \leftarrow \text{COM.Ext}(\text{td}_{\text{Com}}, \text{tag})$ assert $r' \neq \perp$ $r \leftarrow \text{TrueRand}(\text{fk}, \text{tag}, m)$ assert $y = f(\text{fk}, m; r)$ return m

Fig. 22: Simulator for NIPoR in Fig. 10. For conciseness of code, we keep st_{Sim} of Sim_{NIZK} implicit.

- Replace $\text{crs} \leftarrow \text{NIZK.Setup}(1^\lambda)$ by $(\text{crs}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_{\text{NIZK}}.\text{Setup}(1^\lambda)$ and remember td_{Sim} for future use.
- In **ProveO**, replace NIZK.Prove with $\text{Sim}_{\text{NIZK}}.\text{Sim}$.
- In both oracles, we introduce a set $\mathcal{L}^*$ which contains pairs $(\mathbb{x}, \pi)$ (respectively, just $\mathbb{x}$ for weak simulation extractability). These are now tracked in **ProveO** similar to **ProveO**₁ in the SIMEXT game for NIPoR.
- In **VerifyO**, we make no change.²⁰

This is the first column in Fig. 23.

Games 0 and 1 are indistinguishable by a reduction to (weak) simulation extractability of NIZK. Thanks to the introduction of $\mathcal{L}^*$, the handling of simulated proofs in **ProveO** is according to SIMEXT for NIZKs. Thus, the reduction is straightforward. For the respective adversary $\mathcal{B}_{\text{SIM-EXT}}$, we have

$$\Pr[\text{G}_0] \leq \Pr[\text{G}_1] + \text{Adv}_{\text{NIZK}, \mathcal{B}_{\text{SIM-EXT}}}^{\text{SIMEXT}}(\lambda)$$

Note that the checks w.r.t. $\mathcal{L}^*$ are not yet fully compatible with the checks w.r.t. $\mathcal{L}$ which the ideal NIPoR game $\text{SIMEXT}_{\text{NIPoR}, \mathcal{A}, \text{Sim}}^0$ would execute.

Game 2: We rename $\mathcal{L}^*$ to $\mathcal{L}$, and instead of $(\mathbb{x}, \pi)$ we insert and check for $((\text{ck}, \text{fk}, y, \text{tag}), \pi)$, e.g., $(\mathbb{x}, \pi) \in \mathcal{L}^*$ becomes $((\text{ck}, \text{fk}, y, \text{tag}), \pi) \in \mathcal{L}$. Recall that $\mathbb{x} = (\text{ck}, \text{fk}, y, \text{tag}, r'')$, where $r'' = \text{RO}(\text{fk}, \text{tag})$ is (re)computed in **NIPoR.Verify**. Since r'' is deterministically derived from tag , we have that $((\text{ck}, \text{fk}, y, \text{tag}, r''), \pi) \in \mathcal{L}^* \iff ((\text{ck}, \text{fk}, y, \text{tag}), \pi) \in \mathcal{L}$ (respectively $(\text{ck}, \text{fk}, y, \text{tag}, r'') \in \mathcal{L}^* \iff (\text{ck}, \text{fk}, y, \text{tag}) \in \mathcal{L}$ in the weak SIMEXT case), and this does not change the game behaviour. As these changes are conceptual, they are perfectly indistinguishable. Thus, $\Pr[\text{G}_2] = \Pr[\text{G}_1]$.

Game 3: In **ProveO**, after computing tag , we abort if $\text{RO}[\text{fk}, \text{tag}] \neq \perp$.

Games 2 and 3 are statistically indistinguishable by a reduction to unpredictability of COM. Formally, we can move the computation of the (polynomially many) $\text{tag} = \text{COM.Com}(\text{ck}, (r', m); \rho)$ for random r', ρ before running the adversary without affecting the abort event. Now, it is clear that the probability an abort is caused because $\mathcal{A}$ guessed the i -th tag_i is negligible by unpredictability of COM. By a union bound over all (at most) Q_{Prove} queries to **ProveO**, the abort event occurs with negligible probability, namely

$$\Pr[\text{G}_2] \leq \Pr[\text{G}_3] + Q_{\text{Prove}} \cdot \text{Adv}_{\text{NIZK}, \mathcal{B}_{\text{Unpred}}}^{\text{Unpred}}(\lambda)$$

where $\mathcal{B}_{\text{Unpred}}$ is the respective hybrid adversary against unpredictability.

Game 4: We commit to 0 instead of r' inside **ProveO**, that is, we run $\text{tag} \leftarrow \text{COM.Com}(\text{ck}, 0)$. This is the second column in Fig. 23. Games 3 and 4 are indistinguishable by a hybrid argument which reduces to the hiding property of COM. For the respective adversary $\mathcal{B}_{\text{Hide}}$, we find

$$\Pr[\text{G}_3] \leq \Pr[\text{G}_4] + Q_{\text{Prove}} \cdot \text{Adv}_{\text{NIZK}, \mathcal{B}_{\text{Hide}}}^{\text{Hide}}(\lambda)$$

Observe that at this point, we successfully changed **ProveO** to be identical to **ProveO**₁ with simulator code inlined.

²⁰ Since we don't use extraction in **VerifyO** (yet), standard ZK would suffice for this change.

<u>Game₁(1^λ)</u>	<u>Game₄(1^λ)</u>	<u>Game₆(1^λ)</u>	<u>Game₇(1^λ)</u>
<u>RO(fk, tag)</u> if RO[fk, tag] = ⊥ then RO[fk, tag] ← _{\$} $\mathcal{R}$ return RO[fk, tag]	<u>RO(fk, tag)</u> if RO[fk, tag] = ⊥ then RO[fk, tag] ← _{\$} $\mathcal{R}$ return RO[fk, tag]	<u>RO(fk, tag)</u> if RO[fk, tag] = ⊥ then (r', m) ← COM.Ext(td _{Com} , tag) if r' ≠ ⊥ r ← TrueRand(x, tag, m) RO[fk, tag] ← r - r' else RO[fk, tag] ← _{\$} $\mathcal{R}$ return RO[fk, tag]	<u>RO(fk, tag)</u> if RO[fk, tag] = ⊥ then r' ← COM.Ext(td _{Com} , tag) if r' ≠ ⊥ r ← TrueRand(fk, tag, m) RO[fk, tag] ← r - r' else RO[fk, tag] ← _{\$} $\mathcal{R}$ return RO[fk, tag]
<u>ProveO(f_{ik}, m)</u> r' ← _{\$} $\mathcal{R}$; ρ ← _{\$} {0, 1} ^{rlen} tag ← COM.Com(ck, r'; ρ) r'' ← RO(fk, tag) r ← r' + r'' y ← f _{ik} (m; r) x ← (ck, fk, y, tag, r'') w ← (m, r', ρ) π ← _{\$} Sim _{NIZK} .Sim(td _{Sim} , x) L* ← L* ∪ {(x, π)} return (y, tag, π)	<u>ProveO(f_{ik}, m)</u> r' ← _{\$} $\mathcal{R}$; tag ← _{\$} COM.Com(ck, 0) assert RO[x, tag] = ⊥ r'' ← RO(fk, tag) r ← r' + r'' y ← f _{ik} (m; r) x ← (fk, y, tag, r'') π ← _{\$} Sim _{NIZK} .Sim(x) L ← L ∪ {(fk, y, tag, π)} return (y, tag, π)	<u>ProveO(f_{ik}, m)</u> tag ← _{\$} COM.Com(ck, 0) assert RO[fk, tag] = ⊥ r'' ← RO(fk, tag) r ← TrueRand(fk, tag, m) y ← f _{ik} (m; r) x ← (fk, y, tag, r'') π ← _{\$} Sim _{NIZK} .Sim(x) L ← L ∪ {(fk, y, tag, π)} return (y, tag, π)	<u>ProveO(f_{ik}, m)</u> tag ← _{\$} COM.Com(ck, 0) assert RO[fk, tag] = ⊥ r'' ← RO(fk, tag) r ← TrueRand(fk, tag) y ← f _{ik} (m; r) x ← (fk, y, tag, r'') π ← _{\$} Sim _{NIZK} .Sim(x) L ← L ∪ {(fk, y, tag, π)} return (y, tag, π)
<u>VerifyO(f_{ik}, y, tag, π)</u> r'' ← RO(fk, tag) x ← (fk, y, tag, r'') if NIZK.Verify(crs _{NIZK} , x, π) = 0 return 0 if (x, π) ∈ L* return 1 return 1	<u>VerifyO(f_{ik}, y, tag, π)</u> r'' ← RO[fk, tag] x ← (fk, y, tag, r'') if NIZK.Verify(crs _{NIZK} , x, π) = 0 return 0 if (fk, y, tag, π) ∈ L return 1 return 1	<u>VerifyO(f_{ik}, y, tag, π)</u> r'' ← RO[fk, tag] x ← (fk, y, tag, r'') if NIZK.Verify(crs _{NIZK} , x, π) = 0 return 0 if (fk, y, tag, π) ∈ L return 1 return 1	<u>VerifyO(f_{ik}, y, tag, π)</u> r'' ← RO[fk, tag] x ← (fk, y, tag, r'') if NIZK.Verify(crs _{NIZK} , x, π) = 0 return 0 if (fk, y, tag, π) ∈ L* return 1 (r', m) ← COM.Ext(td _{Com} , tag) assert y = f _{ik} (m; r) return 1

Fig. 23: NIPoR proof game hops: Oracles for $\mathcal{A}^{\text{RO}, \text{ProveO}, \text{VerifyO}}$. For conciseness of code, we keep st_{Sim} of Sim_{NIZK} implicit and omit the setup and crs.

Game 5: We replace the generation of the commitment key $\text{ck} \leftarrow \$ \text{COM.Setup}(1^\lambda)$ with a commitment key generated in extractable mode, i.e. we run $(\text{ck}, \text{td}_{\text{Com}}) \leftarrow \text{TSetup}(1^\lambda)$ and remember the trapdoor. By straightline extractable binding, this change is indistinguishable. For the respective adversary against commitment key indistinguishability $\mathcal{B}_{\text{ck}}$, we get

$$\Pr[\text{G}_4] \leq \Pr[\text{G}_5] + \text{Adv}_{\text{Setup}, \text{TSetup}, \mathcal{B}_{\text{ck}}}^{\text{CRS}}(\lambda)$$

Game 6: We change ProveO and RO as follows:

- On a query $\text{RO}(\text{fk}, \text{tag})$, if $\text{RO}[\text{fk}, \text{tag}] = \perp$, we first extract from the tag with $(r', m) \leftarrow \text{COM.Ext}(\text{td}_{\text{Com}}, \text{tag})$.
 - If $r' \neq \perp$, we query $r \leftarrow \text{TrueRand}(\text{fk}, \text{tag}, m)$ and program $\text{RO}[\text{fk}, \text{tag}] = r - r'$. Note that this asserts $r' + r'' = r$, where $r'' = \text{RO}(\text{fk}, \text{tag})$.
 - If $r' = \perp$, we sample $\text{RO}[\text{fk}, \text{tag}] \leftarrow \$ \mathcal{R}$ uniformly.
- Within ProveO, set $r = \text{TrueRand}(\text{fk}, \text{tag}, m)$.

Additionally, we set $r \leftarrow \text{TrueRand}(\text{fk}, \text{tag}, m)$ in ProveO. This game is the third column in Fig. 23.

Games 5 and 6 are computationally indistinguishable. This follows from three game hops. First, instead of programming $\text{RO}[\text{fk}, \text{tag}] = r - r'$, we sample $\text{RO}[\text{fk}, \text{tag}] \leftarrow \$ \mathcal{R}$ in both cases $r' \neq \perp$ and $r' = \perp$. Clearly, the observable behaviour of $\text{RO}(\text{fk}, \text{tag})$ is independent of the internal case distinction.

In the next hop, we implement $\text{RO}(\text{fk}, \text{tag})$ in Game 6 fully. Observe that $\text{TrueRand}(\text{fk}, \text{tag}, m)$ is always run on the same tag as $\text{RO}(\text{fk}, \text{tag})$, and therefore, whenever $\text{RO}[\text{fk}, \text{tag}] = \perp$, we have not previously queried $\text{TrueRand}(\text{fk}, \text{tag}, m)$, and thus $r = \text{TrueRand}(\text{fk}, \text{tag}, m)$ is uniformly random over $\mathcal{R}$. Thus, in the case $r' \neq \perp$, the distribution of $\text{RO}[\text{fk}, \text{tag}]$ again remains unchanged.

Finally, since we asserted that tag was never queried before to $\text{RO}(\text{fk}, \text{tag})$ (else, the game aborts due to the change in Game 3), we can replace $r \leftarrow r' + r''$ in ProveO (which is freshly random since $r' \leftarrow \$ \mathcal{R}$) by $r \leftarrow \text{TrueRand}(\text{fk}, \text{tag}, m)$ (which is also freshly random). Overall, we get that $\Pr[\text{G}_5] = \Pr[\text{G}_6]$.

At this point, we implement RO as in the SIMEXT experiment with the simulator Sim.RO inlined. The only difference to the SIMEXT experiment is in VerifyO.

Game 7: This game is the right-most column in Fig. 23. In this game, we add the extraction $(r', m) \leftarrow \text{COM.Ext}(\text{td}_{\text{Com}}, \text{tag})$ and the assertion $y = f_{\text{fk}}(m; r)$ to VerifyO. This fully implements Sim.Ext for the NIPoR. Note that since $r = r' + r''$ and $r = \text{TrueRand}(\text{fk}, \text{tag}, m)$ by our programming, Game 7 also implements the SIMEXT game, up to syntactical changes.

To argue indistinguishability, we bound the event E that the assertion $y = f_{\text{fk}}(m; r)$ fails. By a straightforward reduction to SIMEXT of NIZK, we can extract a witness $((r'^*, m^*), \rho^*) = \mathbb{w}^* \leftarrow \text{Sim}_{\text{NIZK}}.\text{Ext}(\text{td}_{\text{Sim}}, \mathbb{x}, \pi)$ of the failing query (and abort if that fails). Namely,

- Receive crs_{NIZK} from the SIMEXT experiment and use its oracles to simulate proofs.
- When the assertion $y = f_{\text{fk}}(m; r)$ fails, send $(\mathbb{x}, \pi)$ as the proof forgery.

Next, by a direct reduction to straightline extractable binding of COM, we bound E (i.e., failure of assertion $y = f_{\text{fk}}(m; r)$) after that change. The reduction works as follows:

- Receive $\text{ck}, \text{td}_{\text{Com}}$ from the binding experiment and embed it into game G_6 .
- When the assertion $y = f_{\text{fk}}(m; r)$ fails: Let $((r'^*, m^*), \rho^*) = \mathbb{w}^* \leftarrow \text{Sim}_{\text{NIZK}}.\text{Ext}(\text{td}_{\text{Sim}}, \mathbb{x}, \pi)$ be the extracted witness from Sim_{NIZK} for the failed query (as above). Send $(\text{tag}, (r'^*, m^*), \rho^*)$ to the extractable binding challenger.

Clearly, if the assertion $y = f_{\text{fk}}(m; r)$ fails, then $\text{tag} = \text{COM.Com}(\text{ck}, (r'^*, m^*); \rho^*)$ (because if extraction for $\text{Sim}_{\text{NIZK}}.\text{Ext}(\text{td}_{\text{Sim}}, \mathbb{x}, \pi)$ fails, the game aborts before E can occur). But that means $(r'^*, m^*) \neq (r', m) \leftarrow \text{COM.Ext}(\text{td}_{\text{Com}}, \text{tag})$. Since the output of $\text{COM.Ext}(\text{td}_{\text{Com}}, \text{tag})$ is deterministic, the reduction succeeds in breaking extractable binding, because the challenger will also extract (r', m) . Hence, we find

$$\begin{aligned} \Pr[E] &\leq \text{Adv}_{\text{NIZK}, \mathcal{B}'_{\text{SIM-EXT}}}^{\text{SIMEXT}}(\lambda) + \text{Adv}_{\text{COM}, \mathcal{B}_{\text{bind}}}^{\text{Bind}}(\lambda) \\ \Pr[\text{G}_6] &\leq \Pr[\text{G}_7] + \Pr[E] \end{aligned}$$

Taking all steps together, we arrive at

$$\begin{aligned} \Pr[G_0] \leq & 2 \cdot \text{Adv}_{\text{NIZK}, \mathcal{B}_{\text{SIM-EXT}}}^{\text{SIMEXT}}(\lambda) + Q_{\text{Prove}} \cdot \text{Adv}_{\text{NIZK}, \mathcal{B}_{\text{Unpred}}}^{\text{Unpred}}(\lambda) + Q_{\text{Prove}} \cdot \text{Adv}_{\text{NIZK}, \mathcal{B}_{\text{Hide}}}^{\text{Hide}}(\lambda) \\ & + \text{Adv}_{\text{Setup}, \text{TSetup}, \mathcal{B}_{\text{ck}}}^{\text{CRS}}(\lambda) + \text{Adv}_{\text{COM}, \mathcal{B}_{\text{bind}}}^{\text{Bind}}(\lambda). \end{aligned}$$

As a minor remark, we note that our reduction only needs normal (single-challenge, non-black-box) simulation extractability for NIZK, instead of straightline extractability. $\square$

F.2 Proof of Theorem 10

Overview of the main proof steps. First, we apply the SIMEXT notion of NIPoR to introduce the TrueRand random oracle, which will determine the randomness that is used in every valid ciphertext. All proofs in ChalO', the challenge encryption oracle for CCA security of TPKE', are now simulated. Recall that $\text{fk} = (\text{crs}, \text{pk})$ and the function $f_{\text{fk}}(\mu; r) = \text{TPKE}.\text{Enc}(\text{pk}, \mu, r)$ is the TPKE encryption. Observe that, by stateless tag-only extraction, the reduction will learn m from tag alone (or more precisely, (fk, tag)). Crucially, we can now reduce to the CPA security game of TPKE: Whenever a query $\text{RO}(\text{fk}, \text{tag})$ is made by $\mathcal{A}$, we query the CPA game for $(\text{ct}, r) \leftarrow \text{EncO}(m)$, where (r', m) is the message extracted from tag. Then we program $\text{TrueRand}(\text{fk}, \text{tag}, m) = r$, which by definition of $f_{\text{fk}}(\mu; r) = \text{TPKE}.\text{Enc}(\text{pk}, \mu, r)$ yields ct. This ensures that any valid ciphertext, i.e., ciphertext accompanied by an accepting proof, that is produced by the adversary is a ciphertext the reduction previously received from EncO. After tying up loose ends, we have turned the CCA attack into a CPA attack.

Proof. We argue via several game hops for SIM-CCA and IND-CCA simultaneously. We write ChalO', ParDecO', to indicate the CCA game oracles (in line with our notation TPKE' for the transformation), and EncO, ChalO, ParDecO for the CPA game oracles.

Game 0 (Honest). This is the honest CCA-security game.

Game 1 (Apply SIMEXT). We introduce a fresh random oracle TrueRand, as in the SIMEXT experiment for NIPoR with simulator Sim. We replace NIPoR.Prove within ChalO' by Sim.Sim, and we replace NIPoR.Verify by Sim.Ext (in ParDecO'). We use Sim.RO to implement the random oracle RO for the adversary from now on. The game aborts if extraction fails. By (weak) SIMEXT, this change is indistinguishable.

Game 2 (Extract tag in RO-queries). When $\mathcal{A}$ makes a query $\text{RO}(\text{fk}, \text{tag})$, we run $(m, _) \leftarrow \text{Sim.Ext}^{\text{TrueRand}}(\text{fk}, _, \text{tag}, _)$. Since we assume stateless tag-only extraction, using $\perp$ in place of y, π does not affect Sim.Ext. This change is only conceptual, since we do not yet use m .

Game 3 (Reduction to CPA) We can finally reduce to SIM-CPA/IND-CPA security via a natural reduction as follows:

- The reduction embeds the SIM-CPA/IND-CPA challenger's public key pk within $\text{pk}' = (\text{crs}, \text{pk})$. It responds to corruption queries by forwarding them and their response.
- Upon random oracle query $\text{RO}(\text{fk}, \text{tag})$ with $\text{fk} = (\text{crs}, \text{pk})$, if $\text{RO}(\text{fk}, \text{tag})$ is not yet defined, the reduction extracts m from tag (as introduced in Game 2) and distinguishes the following cases:
 - If $m \neq \perp$, the reduction queries $\text{EncO}(m)$ to its SIM-CPA/IND-CPA challenger and receives (ct, r) as a response. Then, the reduction programs $\text{TrueRand}(\text{fk}, \text{tag}, m) \leftarrow r$.
 - If $m = \perp$, $\text{TrueRand}(\text{fk}, \text{tag}, m)$ is programmed randomly. Since $f_{\text{fk}}(\perp, r) = \perp$ for any r , the attached proof π (and Game 1) ensure that $\text{ct}' = (\perp, \pi)$. Thus, ParDec' always rejects, i.e., returns $\perp$, in this case.
- Upon a ChalO'(m₀, m₁) query in the IND-CCA game: The reduction queries ChalO(m₀, m₁) in the IND-CPA game, receiving ct*. Then it completes the ciphertext via $(\text{tag}, \pi) \leftarrow \text{Sim.Sim}(\text{fk}, \text{ct}^*)$, where $\text{fk} = (\text{crs}, \text{pk})$. Except that ct* comes from the CPA game, this computation is identical to Games 1 and 2. Analogously, the reduction to SIM-CCA proceeds in this way (and we can define a natural simulator induced by the reduction).
- Upon a ParDec'(i, ct' = (ct, tag, π)) query in the CCA game: The reduction runs Sim.Ext to obtain m and $r \leftarrow \text{TrueRand}(\text{tag})$ (or return $\perp$ if extraction fails, $\text{ct} = \perp$, or the query is not admissible, i.e. $(\text{ct}, \text{tag}, \pi)$ is a challenge output by ChalO' in the IND-CCA case). Then it queries ParDec(i, ct) to the CPA game. If ct was not obtained from EncO by the CPA game, the reduction aborts.

It is not hard to see that this simulation is perfect. Indeed, for RO , EncO' and ChalO' queries, this is evident. For ParDec' queries, the only change w.r.t. Game 4 is the abort if ct was not obtained from the CPA game. We analyse this case. Let $\text{fk} = (\text{crs}, \text{pk})$ in the following. As a simplifying assumption, we posit that, w.l.o.g., the adversary queried $\text{RO}(\text{fk}, \text{tag})$ before submitting (ct, tag) to ParDec' .

- Suppose ct is fresh (i.e., not obtained from EncO or ChalO by the CPA game). Then evidently, also the tuple (ct, tag) is fresh. Thus, by the (weak) SIMEXT game, Sim.Ext succeeds²¹ in extracting a witness m and for $r \leftarrow \text{TrueRand}(\text{fk}, \text{tag}, m)$ we have $\text{ct} = \text{TPKE.Enc}(\text{pk}, m; r)$. However, for ct to be fresh, it cannot be of the form $\text{Enc}(\text{pk}, m; r)$ for $r = \text{TrueRand}(\text{fk}, \text{tag}, m)$ (by our programming of TrueRand). A contradiction.
- Suppose now that ct is not fresh, but $\text{ct} = \text{ct}^*$ was obtained from ChalO . (If it was obtained from EncO , no discrepancy occurs.)
 - If $(\text{ct}, \text{tag}, \pi) = (\text{ct}^*, \text{tag}^*, \pi^*)$ for a challenge generated by the reduction, then any admissible ParDec' query in the CCA game maps to an admissible ParDec query for ct in the CPA game. Thus, no discrepancy occurs.
 - Suppose tag differs from challenges generated by the reduction. Then by weak SIMEXT, we can extract the witness m from tag , which leads to the same contradiction as when ct is fresh.
 - Suppose π differs from challenges previously generated by the reduction. Then by (full) SIMEXT, we can extract m from π , obtaining the same contradiction.

This completes the reduction of Game 3 to CPA security, and thus the proof. $\square$

Remark 14 (Relaxed CCA and weak SIMEXT). If NIPoR is only weak SIMEXT, it is easily seen that we obtain relaxed CCA-security [CKN03] w.r.t. the public equivalence relation

$$\underbrace{(\text{ct}, \text{tag}, \pi)}_{\mathbb{x}} \sim \underbrace{(\text{ct}', \text{tag}', \pi)}_{\mathbb{x}'} \iff \mathbb{x} = \mathbb{x}' \wedge \text{Verify}(\mathbb{x}, \pi) = 1 = \text{Verify}(\mathbb{x}', \pi').$$

Indeed, we required (full) SIMEXT only in the last step of the reduction, where we consider the case that ct is a CPA challenge, and $(\text{ct}, \text{tag}, \pi)$ is a ParDec' query which only differs in π from a generated CCA challenge. By the above equivalence relation, this query would not be admissible, and thus weak SIMEXT would suffice.

G Threshold ElGamal

G.1 Preliminaries

Notations. Throughout this section we adopt the following notations. $\mathbb{G}$ is a cyclic group of order q , where q is a prime exponential in the security parameter λ . Small letters (e.g. x) denote elements in $\mathbb{Z}_q$, bold small and capital letters (e.g. $\mathbf{x}$ and $\mathbf{X}$) denote vectors and matrices over $\mathbb{Z}_q$ respectively. We use the implicit notation $[\cdot]$ for group elements, group operation is written additively. In particular, $[1] \in \mathbb{G}$ denotes an arbitrary generator and for any $a, x, y \in \mathbb{Z}_q$, $a[x] + [y] = [ax + y]$. We write GGen as the algorithm $(\mathbb{G}, [1], q) \leftarrow \text{GGen}(1^\lambda)$ which, on input the security parameter, samples randomly a cyclic group $\mathbb{G}$ with generator $[1]$ and prime order q . To ease notation, sets may be interpreted as ordered sets without further specifying, which should not cause confusion.

Computational Assumption. We recall the DDH assumption and its random self-reducibility.

Definition 24. *The DDH assumption states that, for any PPT adversary $\mathcal{A}$,*

$$\begin{aligned} \text{Adv}_{\text{GGen}, \mathcal{A}}^{\text{DDH}}(\lambda) &:= \left| \Pr \left[b = 1 \left| \begin{array}{l} (\mathbb{G}, [1], q) \leftarrow \text{GGen}(1^\lambda) \\ x, y \leftarrow \mathbb{Z}_q \\ b \leftarrow \mathcal{A}(\mathbb{G}, q, [1], [x], [y], [xy]) \end{array} \right. \right] - \Pr \left[b = 1 \left| \begin{array}{l} (\mathbb{G}, [1], q) \leftarrow \text{GGen}(1^\lambda) \\ x, y, z \leftarrow \mathbb{Z}_q \\ b \leftarrow \mathcal{A}(\mathbb{G}, q, [1], [x], [y], [z]) \end{array} \right. \right] \right| \\ &\leq \text{negl}(\lambda). \end{aligned}$$

²¹ In Game 1, we introduced aborts if extraction fails.

Lemma 3 ([EHK⁺13]). For any integer $m > 1$ and any PPT $\mathcal{B}$, there is a PPT $\mathcal{A}$ such that

$$\left| \Pr \left[b = 1 \mid \begin{array}{l} (\mathbb{G}, [1], q) \leftarrow \$ \text{GGen}(1^\lambda) \\ x \leftarrow \$ \mathbb{Z}_q; \mathbf{y} \leftarrow \$ \mathbb{Z}_q^m \\ b \leftarrow \$ \mathcal{B}(\mathbb{G}, q, [1], [x], [\mathbf{y}], [x\mathbf{y}]) \end{array} \right] - \Pr \left[b = 1 \mid \begin{array}{l} (\mathbb{G}, [1], q) \leftarrow \$ \text{GGen}(1^\lambda) \\ x \leftarrow \$ \mathbb{Z}_q; \mathbf{y}, \mathbf{z} \leftarrow \$ \mathbb{Z}_q^m \\ b \leftarrow \$ \mathcal{B}(\mathbb{G}, q, [1], [x], [\mathbf{y}], [\mathbf{z}]) \end{array} \right] \right| \leq \text{Adv}_{\text{GGen}, \mathcal{A}}^{\text{DDH}}(\lambda) + \frac{1}{q-1}.$$

Secret Sharing. We recall the basics of secret sharing, phrased in the language of share-generating matrices. We focus on the special case of Shamir's secret sharing over $\mathbb{Z}_q$. Let $t, k \in \mathbb{N}, t \leq k$. For a t -out-of- k Shamir's secret sharing, the share-generating matrix is the Vandermonde matrix

$$\mathbf{V} := \begin{pmatrix} 1 & 1 & \dots & 1 \\ \alpha_1 & \alpha_2 & \dots & \alpha_k \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_1^{t-1} & \alpha_2^{t-1} & \dots & \alpha_k^{t-1} \end{pmatrix} \in \mathbb{Z}_q^{t \times k}$$

where $(\alpha_i)_{i \in [k]}$ are distinct public fixed non-zero elements in $\mathbb{Z}_q$, denoting the public indices of the k parties.²² It holds that any $t \times t$ submatrix of $\mathbf{V}$ is invertible over $\mathbb{Z}_q$. For a secret $s \in \mathbb{Z}_q$, the secret shares $(s_i)_{i \in [k]}$ for the k parties are

$$(s_1, \dots, s_k) = \underbrace{(s, \mathbf{r}^T)}_{:= \mathbf{s}^T} \mathbf{V} = (\mathbf{s}^T \mathbf{v}_1, \dots, \mathbf{s}^T \mathbf{v}_k)$$

where $\mathbf{r} \in \mathbb{Z}_q^{t-1}$ are the secret sharing randomness uniformly random over $\mathbb{Z}_q$, and $\mathbf{v}_i$ is the i -th column of $\mathbf{V}$.

Denote by $\mathbf{v}_0 = (1, 0, \dots, 0)^T \in \mathbb{Z}_q^t$ the first unit-vector. For any (ordered) set $T \subseteq [k]$, denote by $\mathbf{V}_T$ the submatrix of $\mathbf{V}$ indexed by T . To recover the secret s given the secret shares $(s_i)_{i \in T}$ of a set T of threshold t indices, right-multiply $\mathbf{V}_T^{-1} \mathbf{v}_0$ (equivalently the Lagrange coefficients) to obtain

$$(s_i)_{i \in T} \mathbf{V}_T^{-1} \mathbf{v}_0 = \mathbf{s}^T \mathbf{V}_T \mathbf{V}_T^{-1} \mathbf{v}_0 = s.$$

For an (ordered) set $\mathcal{C} \subset [k]$ where $|\mathcal{C}| < t$, with $\mathbf{V}_{\mathcal{C}}$ the corresponding submatrix of $\mathbf{V}$, we let T^* be an arbitrary fixed (only dependent on $\mathcal{C}$) t -subset of $\{0, 1, \dots, k\}$ containing 0, i.e. $\mathcal{C} \cup \{0\} \subseteq T^* \subseteq \{0, 1, \dots, k\}$.

(E.g. if $|\mathcal{C}| = t - 1$, then $T^* = \mathcal{C} \cup \{0\}$.) We denote by $\mathbf{T}_{\mathcal{C}} = \mathbf{V}_{T^*}^{-1} = \begin{pmatrix} \mathbf{T}_{\mathcal{C}}^\perp \\ \mathbf{T}_{\mathcal{C}}^\vee \end{pmatrix} \in \mathbb{Z}_q^{t \times t}$, where $\mathbf{T}_{\mathcal{C}}^\perp \in \mathbb{Z}_q^{(t-|\mathcal{C}|) \times t}$, $\mathbf{T}_{\mathcal{C}}^\vee \in \mathbb{Z}_q^{|\mathcal{C}| \times t}$. Using these notations, we have the following lemma which is adapted from [CLW25].

Lemma 4 ([CLW25]). Let q be prime. Fix any (ordered) set $\mathcal{C} \subset [k]$ where $|\mathcal{C}| < t$. Let $T \subset [k]$ be a (non-empty) set such that $T, \mathcal{C}$ are disjoint and $|T \cup \mathcal{C}| < t$. Then, the rows of $(\mathbf{0}_{(t-|\mathcal{C}|-1) \times 1}, \mathbf{I}_{t-|\mathcal{C}|-1}) \cdot \mathbf{T}_{\mathcal{C}}^\perp \mathbf{V}_T$ generate $\mathbb{Z}_q^{|T|}$.

ElGamal TPKE. For any number of parties k , any threshold $t \leq k$, let $\text{SS}_{k,t} = (\text{Share}, \text{Rec})$ be a t -out-of- k Shamir's secret sharing scheme. The threshold ElGamal TPKE scheme is given in Fig. 24. Correctness follows immediately from the correctness of the underlying secret-sharing scheme $\text{SS}_{k,t}$.

G.2 Proof of Theorem 8

The majority of the following proof is based on combining different techniques which have appeared in the TPKE literature, in particular works considering SIM security with maximal corruptions [BD10, BGG⁺18, MS25] and also the recent work [CLW25] which considered arbitrary (static) corruption.

²² In the language of polynomials, α_i 's are the public evaluation points, $\mathbf{r}$ is the coefficient of a random degree- $(t-1)$ polynomial.

$\text{KGen}(1^\lambda)$	$\text{Enc}(\text{pk}, \mu = [m])$
$(\mathbb{G}, q, [1]) \leftarrow \text{GGen}(1^\lambda)$	$r \leftarrow \mathbb{Z}_q$
$s \leftarrow \mathbb{Z}_q$	$[c_0] := [r]$
$(s_j)_{j \in [k]} \leftarrow \text{SS}_{k,t}.\text{Share}(s)$	$[c_1] := [s] \cdot r + [m]$
$\text{pk} := (\mathbb{G}, q, [1], [s])$	$\text{ct} := ([c_0], [c_1])$
$\text{sk}_j := s_j \quad \forall j \in [k]$	return ct
return $(\text{pk}, (\text{sk}_j)_{j \in [k]})$	$\text{Rec}((\text{sk}_j)_{j \in T}, \text{ct})$
$\text{ParDec}(\text{sk}_j, \text{ct})$	return $[c_1] - \text{SS}_{k,t}.\text{Rec}((\text{pd}_j)_{j \in T}, T)$
return $\text{pd}_j := s_j \cdot [c_0]$	

Fig. 24: ElGamal TPKE.

Proof overview. In Hyb_1 we express the ElGamal TPKE with the above notations with the Vandermonde matrix explicitly, which allows to examine the underlying correlations. In Hyb_2 , we rewrite the computation of ParDecO , where for any query on some ciphertext, if already $|T| + \mathcal{C} \geq t$, then it computes the answer pd_j based on the first $t - 1$ queries and corrupt keys, the ciphertext, together with $\mu = [m]$. This relies on that such pd_j 's are fully determined by the secret-sharing relation, the ciphertext and the first $t - 1$ partial decryptions, and allows to simulate them without the corresponding secret key shares. This step has appeared in a number of prior works on SIM security (which considered only maximal corruptions²³) and is not necessary for IND security.

In Hyb_3 we rewrite the linear relations induced by secret sharing, in a way such that the public key pk and secret key shares sk_j 's are generated with independent randomness. This step is borrowed from [CLW25]. In Hyb_4 we further rewrite partial decryptions based on the change in Hyb_3 . These allow to neatly invoke DDH on the ciphertexts (encrypted w.r.t. pk) in Hyb_5 .

Finally in Hyb_6 , we invoke DDH again to swap the first (up to) $t - 1 - |\mathcal{C}|$ partial decryptions of any challenge ciphertext to uniform. This is also borrowed from [CLW25] and relies on Lemma 4, where the concerned term $(\mathbf{0}_{(t-|\mathcal{C}|-1) \times 1}, \mathbf{I}_{t-|\mathcal{C}|-1}) \cdot \mathbf{T}_{\mathcal{C}}^\perp \mathbf{V}_T$ will appear as part of the simulated pd_j .

Hyb_7 is an additional step, showing that the stronger 1StSimCPA security is also satisfied (as claimed in Section 3.4), and is not necessary for SemiMal-SIM-CPA .

Proof (of Theorem 8). Fix any PPT adversary $\mathcal{A}$, and let $\mathcal{C} \subset [k]$ be the set of corrupt indices which it declares at the beginning of the experiment. We denote by $\mathcal{H} := [k] \setminus \mathcal{C}$ the set of honest indices, and we adopt the notations introduced in Appendix G.1. In particular: $\mathbf{V} = (\mathbf{v}_1, \dots, \mathbf{v}_k) \in \mathbb{Z}_q^{t \times k}$ is the share-generating matrix of $\text{SS}_{k,t}$, and $\mathbf{v}_0 = (1, 0, \dots, 0)^T$. The set T^* is an arbitrary fixed (only dependent on $\mathcal{C}$) t -subset of $\{0, 1, \dots, k\}$ containing 0, i.e. $\mathcal{C} \cup \{0\} \subseteq T^* \subseteq \{0, 1, \dots, k\}$, and $\mathbf{T}_{\mathcal{C}} = \mathbf{V}_{T^*}^{-1} = \begin{pmatrix} \mathbf{T}_{\mathcal{C}}^\perp \\ \mathbf{T}_{\mathcal{C}}^\vee \end{pmatrix} \in \mathbb{Z}_q^{t \times t}$.

We define a sequence of hybrid experiments $\text{Hyb}_i, i \in [7]$, whose formal descriptions are given in Figs. 25 to 28. Below we argue that, for all $i \in [7]$ it holds

$$|\Pr[\text{Hyb}_i(1^\lambda) = 1] - \Pr[\text{Hyb}_{i+1}(1^\lambda) = 1]| \leq \text{negl}(\lambda).$$

The proof is then completed by observing that Hyb_1 is functionally equivalent to the $\text{SCor-AdpQ-SemiMal-SIM-CPA}$ experiment for $b = 0$, and Hyb_7 is functionally equivalent to the $\text{SCor-AdpQ-SemiMal-SIM-CPA}$ experiment for $b = 1$.

²³ When restricted to maximal corruption, the **if** $P[\text{ct}] \cup \mathcal{C} \notin \mathbb{A}_{k,t}$ branch in ParDecO is voided. In this case, all subsequent hybrids except (an analogue of) Hyb_5 are also voided, leading to a significantly simpler proof (and which resembles those of [BD10, BGG⁺18, MS25]).

Hyb₁. This is functionally equivalent to the **SCor-AdpQ-SemiMal-SIM-CPA** experiment for $b = 0$. Instead of writing the key shares $s_j, j \in [k]$, as secret shares of the secret key s , we express them explicitly as inner products with the respective columns $\mathbf{v}_j$ of the share-generating matrix $\mathbf{V}$. We also introduce an additional table S_{ParDec} for bookkeeping the partial decryptions.

Hyb₂. This is functionally equivalent to **Hyb₁**. The only change is to rewrite **ParDecO** answers for (ct, j) such that $P[\text{ct}] \cup \mathcal{C} \in \mathbb{A}_{k,t}$, i.e. the oracle has previously already answered queries on ct for at least $t - 1 - |\mathcal{C}|$ other distinct non-corrupt indices $\neq j$, as a function of ct and its previous answers. To see the equivalence, first recall that in **Hyb₁**, for any $(\text{ct} = ([c_0], [c_1]), j)$ queried to **ParDecO**,

$$\text{pd}_j = [c_0] \cdot \mathbf{s}^T \mathbf{v}_j.$$

Consider any $\text{ct} = ([c_0], [c_1])$ encrypting some $[m]$. Denote below by T , $|T| = t - 1 - |\mathcal{C}|$, the (ordered) set of the first $t - 1 - |\mathcal{C}|$ indices which **ParDecO** has answered for ct , and $(\text{pd}_i)_{i \in T}$ the oracle answers for these indices. In both **Hyb₁** and **Hyb₂**,

$$[c_1] - [m] = [\mathbf{s}^T \mathbf{v}_0] \cdot r = [c_0] \cdot \mathbf{s}^T \mathbf{v}_0, \quad \text{pd}_i = [c_0] \cdot \mathbf{s}^T \cdot \mathbf{v}_i \quad \forall i \in \mathcal{C} \cup T.$$

Collecting the above terms, we have

$$\begin{aligned} ([c_1] - [m], (\text{pd}_i)_{i \in \mathcal{C} \cup T}) &= [c_0] \cdot \mathbf{s}^T \cdot \mathbf{V}_{\{0\} \cup \mathcal{C} \cup T} \\ ([c_1] - [m], (\text{pd}_i)_{i \in \mathcal{C} \cup T}) \mathbf{V}_{\{0\} \cup \mathcal{C} \cup T}^{-1} &= [c_0] \cdot \mathbf{s}^T \\ ([c_1] - [m], (\text{pd}_i)_{i \in \mathcal{C} \cup T}) \mathbf{V}_{\{0\} \cup \mathcal{C} \cup T}^{-1} \cdot \mathbf{v}_j &= [c_0] \cdot \mathbf{s}^T \mathbf{v}_j = \text{pd}_j. \end{aligned}$$

That is, given $[c_1] - [m]$ and $(\text{pd}_i)_{i \in \mathcal{C} \cup T}$, the partial decryption of any remaining party is fully determined, specifically via the computation on the LHS. The LHS is precisely how pd_j for $j \notin \mathcal{C} \cup T$ is computed in **Hyb₂**. Therefore, pd_j in **Hyb₁** and **Hyb₂** are identical.

Hyb₃. This is functionally equivalent to **Hyb₂**. The only difference between **Hyb₂** and **Hyb₃** is that we change pk and all secret key shares sk_j

$$\begin{aligned} \text{from} \quad & (\text{pk}, (\text{sk}_j)_{j \in \mathcal{H}}, (\text{sk}_j)_{j \in \mathcal{C}}) = (\mathbf{s}^T \mathbf{v}_0, \mathbf{s}^T \mathbf{V}_{\mathcal{H}}, \mathbf{s}^T \mathbf{V}_{\mathcal{C}}) = \mathbf{s}^T (\mathbf{v}_0, \mathbf{V}_{\mathcal{H}}, \mathbf{V}_{\mathcal{C}}) \\ \text{to} \quad & (\text{pk}, (\text{sk}_j)_{j \in \mathcal{H}}, (\text{sk}_j)_{j \in \mathcal{C}}) = (p, (p, \mathbf{w}^T, \mathbf{l}^T) \mathbf{T}_{\mathcal{C}} \mathbf{V}_{\mathcal{H}}, \mathbf{l}^T), \end{aligned}$$

where $\mathbf{T}_{\mathcal{C}}$ is that defined at the beginning of this proof, and $p, \mathbf{w}, \mathbf{l}$ are uniformly random. To see that the two are identically distributed, we observe

$$\begin{aligned} \mathbf{s}^T (\mathbf{v}_0, \mathbf{V}_{\mathcal{H}}, \mathbf{V}_{\mathcal{C}}) &= \mathbf{s}^T \underbrace{(\mathbf{v}_0, \mathbf{V}_{T^* \setminus \{0\} \cup \mathcal{C}}, \mathbf{V}_{\mathcal{C}})}_{=\mathbf{V}_{T^*}} \left(\begin{pmatrix} 1 \\ 0 \end{pmatrix}, \mathbf{T}_{\mathcal{C}} \mathbf{V}_{\mathcal{H}}, \begin{pmatrix} 0 \\ \mathbf{I}_{\mathcal{C}} \end{pmatrix} \right) \\ &\equiv \underbrace{(p, \mathbf{w}^T, \mathbf{l}^T)}_{\text{uniform}} \left(\begin{pmatrix} 1 \\ 0 \end{pmatrix}, \mathbf{T}_{\mathcal{C}} \mathbf{V}_{\mathcal{H}}, \begin{pmatrix} 0 \\ \mathbf{I}_{\mathcal{C}} \end{pmatrix} \right) = (p, (p, \mathbf{w}^T, \mathbf{l}^T) \mathbf{T}_{\mathcal{C}} \mathbf{V}_{\mathcal{H}}, \mathbf{l}^T), \end{aligned}$$

where the first equality is due to $\mathbf{V}_{T^*} \mathbf{T}_{\mathcal{C}} \mathbf{V}_{\mathcal{H}} = \mathbf{V}_{\mathcal{H}}$ by definition of T^* and $\mathbf{T}_{\mathcal{C}}$, and the $\equiv$ holds since $\mathbf{V}_{T^*}$ is a bijective map over $\mathbb{Z}_q$. This shows that **Hyb₂** $\equiv$ **Hyb₃**. Note that in **Hyb₃**, the simulated public key $\text{pk} = [p]$ used in encryption is independent of the corrupt secret key shares $(\text{sk}_j)_{j \in \mathcal{C}} = \mathbf{l}^T$.

Hyb₄. Functionally equivalent to **Hyb₃**. The only change in **Hyb₄** is to express $[r \cdot p]$ as $[c_1] - [m]$ during the computation of partial decryptions. By design of **ChalO** and **EncO**, clearly $[r \cdot p] = [c_0 \cdot p] = [c_1] - [m]$. Note that in **Hyb₄**, any partial decryption is obtained by simply post-processing $\text{ct} = ([c_0], [c_1]), [\mathbf{w}_{c_0}], \mathbf{l}$ and $[m]$.

Hyb₅. For each run of **ChalO**, we swap the ciphertext from $([r], [pr] + [m])$ to $([r], [z])$, where $z \leftarrow \mathbb{Z}_q$ is a uniformly random $\mathbb{Z}_q$ element. We claim $\text{Hyb}_4 \approx_c \text{Hyb}_5$ under the DDH assumption.

Let $Q \in \text{poly}(\lambda)$ denote the total number of queries to **ChalO**. The above change follows from the DDH assumption and Lemma 3, stating that

$$(\mathbb{G}, q, [1], [p], [\mathbf{r}], [pr]) \approx_c (\mathbb{G}, q, [1], [p], [\mathbf{r}], [\mathbf{z}]),$$

where $\mathbf{r} \in \mathbb{Z}_q^Q$ denotes the concatenation of all Q **ChalO** randomness, and $\mathbf{z} \leftarrow \mathbb{Z}_q^Q$ is uniformly random. To simulate Hyb_4 (resp. Hyb_5), we exploit that p (in plain) is never required in the games, but only $[p]$:

- Simulate the public key using $[p]$.
- Simulate each of the Q **ChalO** ciphertexts using an entry $[r]$ of $[\mathbf{r}]$, together with the corresponding entry $[pr]$ of $[pr]$ (resp. $[z]$ of $[\mathbf{z}]$).
- Generate any **EncO** ciphertext using the adversarially chosen randomness r , and compute $[pr]$ using $[p]$ and r .

By Lemma 3, the advantage of distinguishing Hyb_4 and Hyb_5 is upper-bounded by

$$\text{Adv}_{\text{Gen}, \mathcal{A}}^{\text{DDH}}(\lambda) + \frac{1}{q-1}. \quad (6)$$

We note that the above quantity is independent of Q . Finally, for any of the Q simulated **ChalO** ciphertexts, $[z] + [m] \equiv [z]$ since $[z]$ is uniformly random. We thus conclude that $\text{Hyb}_4 \approx_c \text{Hyb}_5$, with the distinguishing advantage upper-bounded by Eq. (6).

Hyb₆. For each **ParDecO** answer for (ct, j) , where $j \notin \mathcal{C}$, $\text{ct} \in L_{\text{Chal}}$ and $P[\text{ct}] \cup \mathcal{C} \notin \mathbb{A}_{k,t}$, i.e. j is among the first $t-1-|\mathcal{C}|$ non-corrupt indices queried on ct , we swap the answer to a uniformly random group element. We claim $\text{Hyb}_5 \approx_c \text{Hyb}_6$ under the DDH assumption.

To begin, consider any challenge ct from **ChalO**. Denote by T the set of the first (up to) $t-1-|\mathcal{C}|$ non-corrupt indices being queried. In Hyb_5 , the **ParDecO** answers for these indices are (horizontally concatenated)

$$\begin{aligned} (\text{pd}_i)_{i \in T} &= ([z] - [m], \mathbf{w}^T[r], \mathbf{I}^T[r]) \cdot \mathbf{T}_{\mathcal{C}} \mathbf{V}_T \\ &= ([z] - [m], \mathbf{w}^T[r], \mathbf{I}^T[r]) \begin{pmatrix} \mathbf{T}_{\mathcal{C}}^\perp \\ \mathbf{T}_{\mathcal{C}}^\vee \end{pmatrix} \mathbf{V}_T \\ &= ([z] - [m], \mathbf{w}^T[r]) \cdot \begin{pmatrix} 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \cdot \mathbf{T}_{\mathcal{C}}^\perp \mathbf{V}_T + \mathbf{I}^T[r] \cdot \mathbf{T}_{\mathcal{C}}^\vee \bar{\mathbf{V}}_T = [\mathbf{w}^T r] \cdot (\mathbf{0} \ \mathbf{I}) \cdot \mathbf{T}_{\mathcal{C}}^\perp \mathbf{V}_T + * \end{aligned}$$

where in the last line we denote by $*$ all other additive terms except the stated $\mathbf{w}^T \cdot [r] \cdot (\mathbf{0} \ \mathbf{I}) \cdot \mathbf{T}_{\mathcal{C}}^\perp \mathbf{V}_T$, which we will not care in the rest. We swap the above to

$$(\text{pd}_i)_{i \in T} \approx_c [\mathbf{y}^T] \cdot (\mathbf{0} \ \mathbf{I}) \cdot \mathbf{T}_{\mathcal{C}}^\perp \mathbf{V}_T + *$$

where $\mathbf{y} \leftarrow \mathbb{Z}_q^{t-1-|\mathcal{C}|}$ is uniformly random, and $*$ is the same as that in the previous expression.

Let $Q \in \text{poly}(\lambda)$ denote the total number of queries to **ChalO**. The above change follows from the DDH assumption, Lemma 3, and a routine hybrid argument (over each entry of $[\mathbf{w}]$ below), stating that

$$(\mathbb{G}, q, [1], [\mathbf{w}], [\mathbf{r}^T], [\mathbf{w} \cdot \mathbf{r}^T]) \approx_c (\mathbb{G}, q, [1], [\mathbf{w}], [\mathbf{r}^T], [\mathbf{Y}]),$$

where $\mathbf{r} \in \mathbb{Z}_q^Q$ denotes the concatenation of all Q **ChalO** randomness, and $\mathbf{Y} \leftarrow \mathbb{Z}_q^{(t-1-|\mathcal{C}|) \times Q}$ is uniformly random. To simulate Hyb_5 (resp. Hyb_6), the reduction works as follows:

- Simulate each of the Q **ChalO** ciphertexts using an entry $[r]$ of $[\mathbf{r}]$, and simulate its partial decryptions using $[\mathbf{w}r]$ (resp. a column $[\mathbf{y}]$ of $[\mathbf{Y}]$) and $[r]$. Note that neither $\mathbf{w}$ nor r (in plain) is required.
- Generate any **EncO** ciphertext using the adversarially chosen randomness r , and compute its partial decryptions using $[\mathbf{w}]$ and r . Again, note that $\mathbf{w}$ (in plain) is never required.

$$(t - 1 - |\mathcal{C}|) \cdot (\text{Adv}_{\text{Gen}, \mathcal{A}}^{\text{DDH}}(\lambda) + \frac{1}{q-1}) \leq (t - 1) \cdot (\text{Adv}_{\text{Gen}, \mathcal{A}}^{\text{DDH}}(\lambda) + \frac{1}{q-1}). \quad (7)$$

Finally, by Lemma 4, the rows of $(\mathbf{0} \text{ I}) \cdot \mathbf{T}_{\mathcal{C}}^{\perp} \mathbf{V}_T$ generate $\mathbb{Z}_q^{|T|}$. Since $\mathbf{y}$ from the above is uniformly random (fresh and independent for each ChalO query), we have that $\mathbf{y}^{\top} \cdot (\mathbf{0} \text{ I}) \cdot \mathbf{T}_{\mathcal{C}}^{\perp} \mathbf{V}_T$ is uniformly random over $\mathbb{Z}_q^{1 \times |T|}$. Therefore,

We thus conclude that $\text{Hyb}_5 \approx_c \text{Hyb}_6$ as wanted, with the distinguishing advantage bounded by Eq. (7).

Finally we write out the full Hyb_7 with syntactical adjustments in Fig. 28. When interpreting the code highlighted in grey as run by a PPT simulator $\mathcal{S}$, Hyb_7 is identical to the experiment for $\text{SCor-AdpQ-SemiMal-SIM-CPA}$ for $b = 1$. $\square$

(a) Hyb₁. (b) Hyb₂. Other algorithms are unchanged.

56

Hyb₃ (1^λ) $(\mathbb{A}_{k,t}, \mathcal{C}, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$ assert $(\mathcal{C} \subset [k]) \wedge (\mathcal{C} \notin \mathbb{A}_{k,t})$ $(\mathbb{G}, q, [1]) \leftarrow \mathcal{G}\text{Gen}(1^\lambda)$ $p \leftarrow \mathbb{Z}_q; \mathbf{w} \leftarrow \mathbb{Z}_q^{t-1- \mathcal{C} }; \mathbf{l} \leftarrow \mathbb{Z}_q^{ \mathcal{C} }$ $\text{pk} := (\mathbb{G}, q, [1], [p])$ $(\text{sk}_j)_{j \in \mathcal{C}} := (l_j)_{j \in \mathcal{C}} = \mathbf{l}^T$ $L_{\text{Enc}} \leftarrow \emptyset; L_{\text{Chal}} \leftarrow \emptyset; P[\cdot] \leftarrow \emptyset; M[\cdot] \leftarrow \emptyset$ $S_{\text{ParDec}}[\cdot] \leftarrow \emptyset$ return $b' \leftarrow \mathcal{A}^{\text{EncO}, \text{ChalO}, \text{ParDecO}}(\text{st}_{\mathcal{A}}, (\text{sk}_j)_{j \in \mathcal{C}})$ ParDecO ($\text{ct} = ([c_0], [c_1]), j$) assert $(\text{ct} \in L_{\text{Enc}} \cup L_{\text{Chal}}) \wedge j \in [k]$ $P[\text{ct}] \leftarrow P[\text{ct}] \cup \{j\}$ if $j \in \mathcal{C}$ then return $\text{pd}_j := [c_0] \cdot l_j$ if $P[\text{ct}] \cup \mathcal{C} \notin \mathbb{A}_{k,t}$ then $\text{pd}_j := [c_0] \cdot (p, \mathbf{w}^T, \mathbf{l}^T) \mathbf{T}_{\mathcal{C}} \mathbf{v}_j$ $S_{\text{ParDec}}[\text{ct}, j] \leftarrow \text{pd}_j$ else $\text{pd}_i := [c_0] \cdot l_i \quad \forall i \in \mathcal{C}$ parse $\text{pd}_i \leftarrow S_{\text{ParDec}}[\text{ct}, i] \quad \forall i \in T$ parse $[m] \leftarrow M[\text{ct}]$ $\text{pd}_j := ([c_1] - [m], (\text{pd}_j)_{j \in \mathcal{C} \cup T}) \cdot \mathbf{V}_{\{0\} \cup \mathcal{C} \cup T}^{-1} \cdot \mathbf{v}_j$ return pd_j	ChalO ($\mu = [m]$) $r \leftarrow \mathbb{Z}_q$ $[c_0] := [r]$ $[c_1] := [p] \cdot r + [m]$ $\text{ct} := ([c_0], [c_1])$ $L_{\text{Chal}} \leftarrow L_{\text{Chal}} \cup \{\text{ct}\}$ $M[\text{ct}] \leftarrow [m]$ return ct EncO ($\mu = [m], \text{rnd} = r$) $[c_0] := [r]$ $[c_1] := [p] \cdot r + [m]$ $\text{ct} := ([c_0], [c_1])$ $L_{\text{Enc}} \leftarrow L_{\text{Enc}} \cup \{\text{ct}\}$ $M[\text{ct}] \leftarrow [m]$ return (ct, r)
---	--

(a) Hyb₃.

ParDecO ($\text{ct} = ([c_0], [c_1]), j$) assert $(\text{ct} \in L_{\text{Enc}} \cup L_{\text{Chal}}) \wedge j \in [k]$ $P[\text{ct}] \leftarrow P[\text{ct}] \cup \{j\}$ if $j \in \mathcal{C}$ then return $\text{pd}_j := [c_0] \cdot l_j$ if $P[\text{ct}] \cup \mathcal{C} \notin \mathbb{A}_{k,t}$ then parse $[m] \leftarrow M[\text{ct}]$ $\text{pd}_j := ([c_1] - [m], [\mathbf{w}^T c_0], \mathbf{l}^T [c_0]) \cdot \mathbf{T}_{\mathcal{C}} \mathbf{v}_j$ $S_{\text{ParDec}}[\text{ct}, j] \leftarrow \text{pd}_j$ else $\text{pd}_i := [c_0] \cdot l_i \quad \forall i \in \mathcal{C}$ parse $\text{pd}_i \leftarrow S_{\text{ParDec}}[\text{ct}, i] \quad \forall i \in T$ parse $[m] \leftarrow M[\text{ct}]$ $\text{pd}_j := ([c_1] - [m], (\text{pd}_j)_{j \in \mathcal{C} \cup T}) \cdot \mathbf{V}_{\{0\} \cup \mathcal{C} \cup T}^{-1} \cdot \mathbf{v}_j$ return pd_j
--

(b) Hyb₄. Other algorithms are unchanged.Fig. 26: Hyb₃ and Hyb₄ for Appendix G.2.

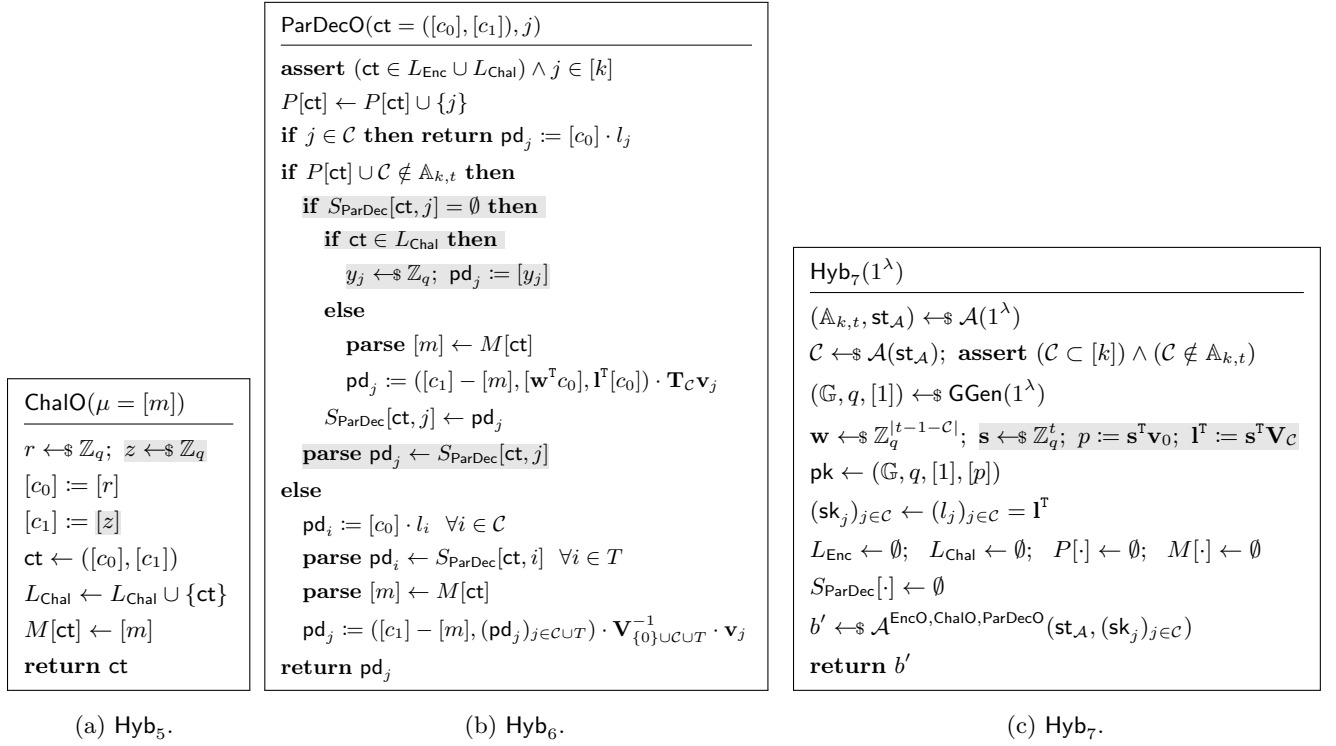


Fig. 27: Hyb₅, Hyb₆ and Hyb₇ for Appendix G.2. Unchanged algorithms are omitted.

Hyb₇(1^λ) $(\mathbb{A}_{k,t}, \mathcal{C}, \text{st}_A) \leftarrow \mathcal{A}(1^\lambda)$ assert $(\mathcal{C} \subset [k]) \wedge (\mathcal{C} \notin \mathbb{A}_{k,t})$ $(\mathbb{G}, q, [1]) \leftarrow \mathcal{G}\text{Gen}(1^\lambda)$ $\mathbf{s} \leftarrow \mathbb{Z}_q^t$; $p := \mathbf{s}^\top \mathbf{v}_0$; $\mathbf{l}^\top := \mathbf{s}^\top \mathbf{V}_C$ $\text{pk} := (\mathbb{G}, q, [1], [p])$ $(\text{sk}_j)_{j \in \mathcal{C}} := (l_j)_{j \in \mathcal{C}} = \mathbf{l}^\top$ $L_{\text{Enc}} \leftarrow \emptyset$; $L_{\text{Chal}} \leftarrow \emptyset$; $P[\cdot] \leftarrow \emptyset$; $M[\cdot] \leftarrow \emptyset$ $S_{\text{Enc}}[\cdot] \leftarrow \emptyset$; $S_{\text{ParDec}}[\cdot] \leftarrow \emptyset$ $\mathbf{w} \leftarrow \mathbb{Z}_q^{t-1- \mathcal{C} }$; $\text{st}_S \leftarrow (\text{pk}, (\text{sk}_j)_{j \in \mathcal{C}}, \mathbf{w})$ $b' \leftarrow \mathcal{A}^{\text{EncO}, \text{ChalO}, \text{ParDecO}}(\text{st}_A, (\text{sk}_j)_{j \in \mathcal{C}})$ return b'	ParDecO($\text{ct} = ([c_0], [c_1]), j$) assert $(\text{ct} \in L_{\text{Enc}} \cup L_{\text{Chal}}) \wedge j \in [k]$ $P[\text{ct}] \leftarrow P[\text{ct}] \cup \{j\}$ if $P[\text{ct}] \cup \mathcal{C} \in \mathbb{A}_{k,t} \wedge \text{ct} \in L_{\text{Chal}}$ then parse $[m] \leftarrow M[\text{ct}]$ if $j \in \mathcal{C}$ then $\text{pd}_j := [c_0] \cdot l_j$ else $\text{pd}_i := [c_0] \cdot l_i \quad \forall i \in \mathcal{C}$ parse $\text{pd}_i \leftarrow S_{\text{ParDec}}[\text{ct}, i] \quad \forall i \in T \quad // T = t - 1 - \mathcal{C} $ $\text{pd}_j := ([c_1] - [m], (\text{pd}_j)_{j \in \mathcal{C} \cup T}) \cdot \mathbf{V}_{\{0\} \cup \mathcal{C} \cup T}^{-1} \cdot \mathbf{v}_j$ else if $j \in \mathcal{C}$ then $\text{pd}_j := [c_0] \cdot l_j$ elseif $S_{\text{ParDec}}[\text{ct}, j] = \emptyset$ then if $\text{ct} \in L_{\text{Chal}}$ then $y_j \leftarrow \mathbb{Z}_q$; $\text{pd}_j := [y_j]$ else parse $[m] \leftarrow S_{\text{Enc}}[\text{ct}]$ if $P[\text{ct}] \cup \mathcal{C} \notin \mathbb{A}_{k,t}$ then $\text{pd}_j := ([c_1] - [m], \mathbf{w}^\top [c_0], \mathbf{l}^\top [c_0]) \cdot \mathbf{T}_C \mathbf{v}_j$ else $\text{pd}_i := [c_0] \cdot l_i \quad \forall i \in \mathcal{C}$ parse $\text{pd}_i \leftarrow S_{\text{ParDec}}[\text{ct}, i] \quad \forall i \in T \quad // T = t - 1 - \mathcal{C} $ $\text{pd}_j := ([c_1] - [m], (\text{pd}_j)_{j \in \mathcal{C} \cup T}) \cdot \mathbf{V}_{\{0\} \cup \mathcal{C} \cup T}^{-1} \cdot \mathbf{v}_j$ $S_{\text{ParDec}}[\text{ct}, j] \leftarrow \text{pd}_j$ parse $\text{pd}_j \leftarrow S_{\text{ParDec}}[\text{ct}, j]$ return pd_j																
<table border="0"> <tr> <td>ChalO($\mu = [m]$)</td><td>EncO($\mu = [m], \text{rnd} = r$)</td></tr> <tr> <td>$r \leftarrow \mathbb{Z}_q$; $z \leftarrow \mathbb{Z}_q$</td><td>$[c_0] := [r]$</td></tr> <tr> <td>$[c_0] := [r]$</td><td>$[c_1] := [p]r + [m]$</td></tr> <tr> <td>$[c_1] := [z]$</td><td>$\text{ct} \leftarrow ([c_0], [c_1])$</td></tr> <tr> <td>$\text{ct} \leftarrow ([c_0], [c_1])$</td><td>$S_{\text{Enc}}[\text{ct}] \leftarrow [m]$</td></tr> <tr> <td>$L_{\text{Chal}} \leftarrow L_{\text{Chal}} \cup \{\text{ct}\}$</td><td>$L_{\text{Enc}} \leftarrow L_{\text{Enc}} \cup \{\text{ct}\}$</td></tr> <tr> <td>$M[\text{ct}] \leftarrow [m]$</td><td>$M[\text{ct}] \leftarrow [m]$</td></tr> <tr> <td>return ct</td><td>return (ct, r)</td></tr> </table>	ChalO ($\mu = [m]$)	EncO ($\mu = [m], \text{rnd} = r$)	$r \leftarrow \mathbb{Z}_q$; $z \leftarrow \mathbb{Z}_q$	$[c_0] := [r]$	$[c_0] := [r]$	$[c_1] := [p]r + [m]$	$[c_1] := [z]$	$\text{ct} \leftarrow ([c_0], [c_1])$	$\text{ct} \leftarrow ([c_0], [c_1])$	$S_{\text{Enc}}[\text{ct}] \leftarrow [m]$	$L_{\text{Chal}} \leftarrow L_{\text{Chal}} \cup \{\text{ct}\}$	$L_{\text{Enc}} \leftarrow L_{\text{Enc}} \cup \{\text{ct}\}$	$M[\text{ct}] \leftarrow [m]$	$M[\text{ct}] \leftarrow [m]$	return ct	return (ct, r)	
ChalO ($\mu = [m]$)	EncO ($\mu = [m], \text{rnd} = r$)																
$r \leftarrow \mathbb{Z}_q$; $z \leftarrow \mathbb{Z}_q$	$[c_0] := [r]$																
$[c_0] := [r]$	$[c_1] := [p]r + [m]$																
$[c_1] := [z]$	$\text{ct} \leftarrow ([c_0], [c_1])$																
$\text{ct} \leftarrow ([c_0], [c_1])$	$S_{\text{Enc}}[\text{ct}] \leftarrow [m]$																
$L_{\text{Chal}} \leftarrow L_{\text{Chal}} \cup \{\text{ct}\}$	$L_{\text{Enc}} \leftarrow L_{\text{Enc}} \cup \{\text{ct}\}$																
$M[\text{ct}] \leftarrow [m]$	$M[\text{ct}] \leftarrow [m]$																
return ct	return (ct, r)																

Fig. 28: Hyb₇ for Appendix G.2 (rewritten). The code in gray can be interpreted as a simulator.

H Summary of Results

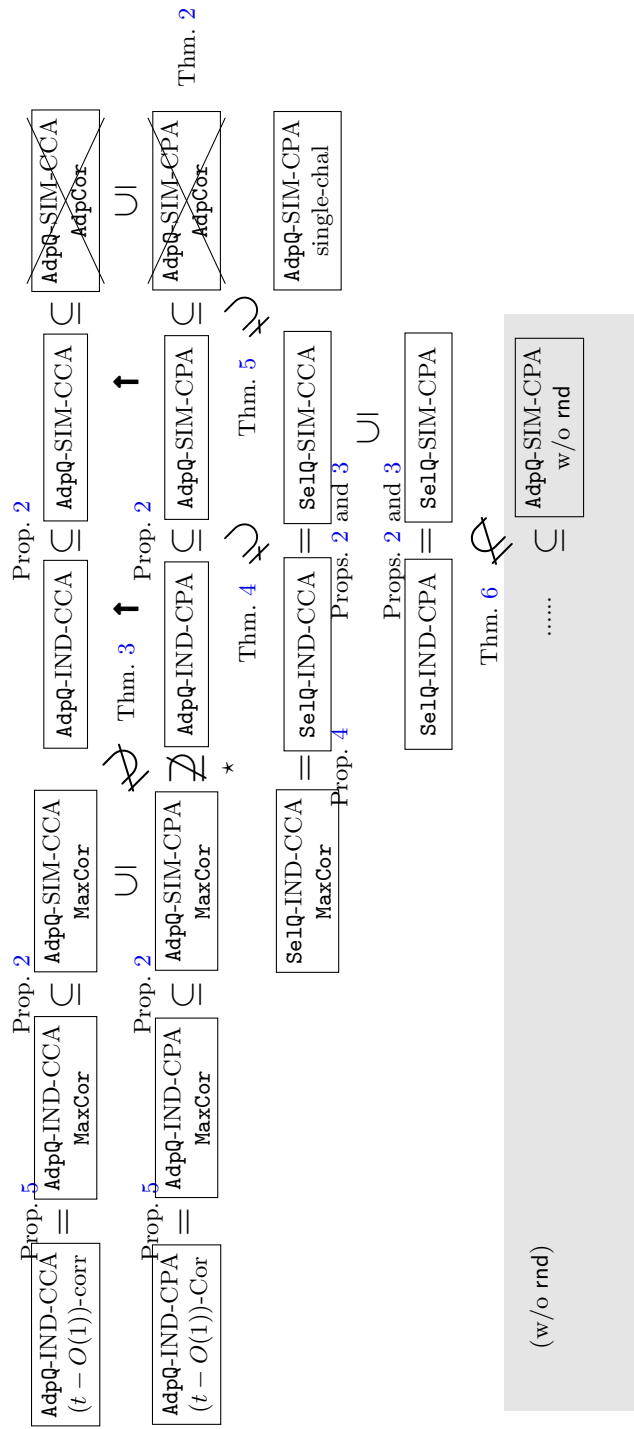


Fig. 29: Summary of results. The above considers static corruption (**Scor**) unless otherwise specified. Relation without mark: trivial. *: implied by Thm. 3. **↑**: Transformation from Theorem 10 applies. “w/o rnd” zone: EncO does not output rnd.